UNIVERSIDAD SIGLO 21

Licenciatura en Informática

Seminario de Práctica de Informática

Sistema de Gestión de Inventario y Punto de Venta
Para Kiosco "JRMARKET"

Autor: Cristian Javier Palmero
Legajo: INF275-11987
Docente/Tutor: Pablo Alejandro Virgolini
Fecha: 10 de Octubre de 2026

Introducción.....5
Antecedentes..... 5
Descripción del Área Programática..... 5
Formulación de la Problemática.....6
Justificación..... 7
Definiciones del Proyecto y Sistema..... 7
Objetivo General del Proyecto..... 7
Objetivos Específicos del Proyecto..... 7
Objetivo General del Sistema..... 8
Límites..... 8
Alcance..... 8
Involucrados con el Sistema..... 8
No Contempla.....9
Elicitación.....9
Actividad del Cliente y Marco Normativo.....9
Marco Regulatorio y Legal..... 9
Elicitación de TIC (Tecnologías de la Información y la Comunicación).....10
Hardware..... 10
Herramientas de Modelado y Diseño..... 12
Tipos y Arquitecturas de Servidores de Bases de Datos..... 13
Análisis de la Competencia.....14
Relevamiento..... 18
Relevamiento Estructural..... 18
Relevamiento Funcional..... 18
Proceso de Negocio.....20
Proceso de Venta de Productos.....20
Proceso de Compra de Productos..... 21
Diagnóstico..... 23
Proceso de Venta de Productos.....23
Proceso de Compra de Productos..... 23
Propuesta de Solución.....24
Propuesta..... 24
Listado de Requerimientos Funcionales.....25
Listado de Requerimientos no Funcionales.....25
Listado de Requerimientos Candidatos.....26
Listado de Casos de Uso..... 26
Desarrollo del Prototipo.....28
Diagrama de Casos de Uso..... 28
Comunicaciones..... 28
Descripción..... 28
Políticas de Seguridad.....29
Descripción de Casos de Uso.....30

Registrar Producto..... 30
Modificar Producto..... 31
Dar de Baja Producto..... 32
Registrar Venta de Productos..... 33
Registrar Compra de Productos..... 36
Interfaz Gráfica..... 38
GUI-01..... 38
GUI-02..... 39
GUI-03..... 40
GUI-04..... 41
GUI-05..... 42
GUI-06..... 43
GUI-07..... 44
GUI-08..... 44
Etapa de Análisis..... 45
Diagrama de Secuencia del Análisis - Venta de Productos..... 46
Diagrama de Secuencia del Análisis - Compra de Productos..... 47
Etapa de Diseño..... 48
Diagrama de Clases..... 48
Etapa de Implementación..... 49
Tabla de Requerimientos del Sistema e Implementación Física..... 49
Diagrama de Despliegue..... 50
Etapa de Pruebas..... 50
Plan de Pruebas..... 50
Objetivos del Plan..... 50
Alcance..... 51
Entorno de Pruebas..... 51
Matriz de Plan de Pruebas..... 51
Modelo de Pruebas: Especificación y Procedimiento..... 52
Prueba de Caja Negra: Análisis de Valores Límite y Clases de Equivalencia..... 52
Prueba de Caja Blanca: Complejidad Ciclomática y Cobertura Lógica..... 53
Tratamiento de Defectos (Workflow y Gestión de Incidentes)..... 55
Flujo de Estados del Defecto (Workflow)..... 55
Plantilla de Registro de Defectos..... 56
Evaluación de las Pruebas y Reporte de Resultados..... 58
Resumen Cuantitativo de Ejecución..... 58
Métricas y Distribución de Defectos..... 58
Criterio de Completitud y Dictamen de Calidad..... 58
Base de Datos..... 59
Definición de Base de Datos para el Sistema..... 59
Base de Datos MySQL..... 59
Componentes del Modelo Relacional..... 59

Diagrama Entidad-Relación....60
El Lenguaje SQL....61
Diagrama Entidad-Relación....63
Sentencias DDL y DML....64
Creación de Tablas y Base de Datos....64
Inserción de Datos de Prueba....66
Consultas de Objetos de Base de Datos del Negocio....68
Modificación y borrado de registros....70
Etapa de Construcción de Código Java....71
Aplicación de los Pilares de la Programación Orientada a Objetos....72
1. Abstracción: Superclase Comprobante....72
2. Herencia y Reutilización: Clases Compra y Venta....73
3. Polimorfismo Dinámico y Sobrescritura de Métodos....74
4. Encapsulamiento, Ocultamiento e Invariantes de Negocio....75
Métodos Constructores e Inicialización de Estados....76
Estructuras de Control, Colecciones Dinámicas y Algoritmos de Búsqueda....76
Gestión de Errores....77
Menú Principal, Navegación y Control de Acceso (RBAC)....78
Verificación Operacional y Capturas del Sistema en Ejecución....78
Patrones de Diseño, Persistencia JDBC y Gestión de Archivos....84
Patrón Arquitectónico Modelo-Vista-Controlador (MVC)....84
Responsabilidades de los Componentes en JRMARKET....85
Conexión y Persistencia Relacional con MySQL mediante JDBC Puro....86
Gestión de la Conexión y Disponibilidad del Canal (ConexionBD.java)....86
Consultas Parametrizadas y Prevención de Inyecciones SQL....88
Transaccionalidad Atómica y Consistencia ACID....89
Estructuras de Datos: Arreglos, Colecciones ArrayList e Iteradores....90
Arreglos Unidimensionales Nativos (Array[])....90
Colecciones Dinámicas (ArrayList<E>)....90
Recorrido Seguro Mediante Iteradores (Iterator<E>)....91
Tratamiento y Gestión Robusta de Excepciones en Java....91
Manipulación de Archivos de Texto Mediante Flujos java.io....94
Manejo del Sistema de Archivos con la Clase File....94
Flujos de Bytes con Salida Formateada (FileOutputStream y PrintStream)....94
Flujos de Caracteres con Buffer (FileWriter y BufferedWriter)....95
Entrega del Proyecto Integrador, Repositorios y Presentación Audiovisual....97
Bibliografía....99

Introducción

En este trabajo se presenta el análisis de la situación problemática que afecta a la gestión comercial, operativa y de control del comercio minorista "**JRMARKET**", proponiendo el desarrollo e implementación de un Sistema de Gestión de Inventario y Punto de Venta de escritorio para dar una solución informática integral.

Antecedentes

El Kiosco "JRMARKET" es un emprendimiento comercial familiar de proximidad fundado hace dos años (a mediados de 2024) en una zona urbana de mucha circulación peatonal. La empresa está constituida por dos socios propietarios, quienes cumplen un doble rol dentro de la organización: actúan como administradores del negocio y, a su vez, operan de forma rotativa en el mostrador en la atención diaria al público.

A pesar de su corta trayectoria, el comercio experimentó un rápido crecimiento y diversificó su catálogo inicial para responder a la demanda de la zona. Además de artículos tradicionales de kiosco y almacén como golosinas, bebidas frías, cigarrillos, lácteos, snacks y comestibles de primera necesidad, incorporó de otro tipo de rubro y de alta demanda tales como accesorios de telefonía móvil (cargadores, cables de carga USB, auriculares), artículos de juguetería, y productos de bazar menor.

En la actualidad, en un radio menor a 5 cuadras, el negocio convive con 3 comercios competidores directos de similares características y con el mismo tipo de gestión, sin herramientas tecnológicas de soporte, basando su administración diaria en cuadernos manuscritos y la memoria de sus dueños. Ante este escenario y debido a la rápida expansión de su inventario multirubro en tan solo dos años de actividad, la administración del Kiosco "JRMARKET" ha decidido profesionalizar y modernizar su gestión comercial mediante la implementación de software a medida, buscando optimizar los tiempos de atención, eliminar errores de cobro y obtener ventajas competitivas frente a los competidores de la zona.

Descripción del Área Programática

El local cuenta con un único puesto de atención al público en el mostrador principal. Como se mencionó anteriormente, este puesto de trabajo carece por completo de equipamiento informático. La totalidad de la operatoria diaria se apoya únicamente en una libreta de papel, lapicera, una calculadora de mano básica y un cartel con el Alias de Mercado Pago pegado al lado de la caja.

Dada esa mecánica, los procesos comerciales, de inventario y de conciliación dejan en evidencia las siguientes problemáticas en el día a día:

- **Incertidumbre y pérdida de tiempo en la consulta de precios:** Al contar con un inventario tan diversificado, los socios y vendedores no logran memorizar la totalidad de los precios ni sus actualizaciones, lo que genera demoras en la respuesta al cliente, transmite falta de profesionalismo y ocasiona molestias.
- **Cálculo manual y cobro con calculadora:** En cada venta se suma cada uno de los ítems que componen una compra de manera manual con una calculadora. En ventas con varios

productos, cualquier distracción obliga a reiniciar la cuenta, aumentando el riesgo de cobrar de menos o dar mal el vuelto en operaciones en efectivo.

- **Fricción en cobros con Mercado Pago mediante Alias:** Para cobrar de forma digital, el comercio no dispone de un código QR generado por sistema. El cliente debe tipear manualmente el Alias bancario o el CVU en su propia aplicación móvil, ingresar el importe y mostrar la pantalla con el comprobante de transferencia. Luego, el socio de turno debe sacar su teléfono personal del bolsillo, entrar a la app de Mercado Pago y esperar a que la notificación de dinero entrante impacte en su cuenta, generando demoras en los tiempos de atención en horas pico.
- **Registro omitido e inconsistencias de caja:** Las ventas se anotan a mano en una libreta. Cuando hay varios clientes esperando en el mostrador, anotar cada monto se vuelve inviable en la práctica, por lo que muchas transacciones quedan sin registrar.
- **Desconocimiento total del inventario y compras a ciegas:** No existe un control de existencias. Los pedidos a distribuidores y preventistas se deciden mirando las estanterías "a ojo" minutos antes de que llegue el repartidor, provocando quiebres de stock recurrentes en productos de alta demanda y capital inmovilizado en artículos de baja rotación.

Formulación de la Problemática

¿Es factible el desarrollo e implementación de un sistema de información de escritorio en Java con persistencia en MySQL que permita a los integrantes del Kiosco "JRMARKET" registrar ventas en tiempo real, controlar medios de pago, automatizar el descuento de inventario y generar reportes precisos de caja?

Justificación

El desarrollo del presente proyecto es relevante para el comercio "JRMARKET" porque permitirá ordenar, agilizar y transparentar sus operaciones diarias de venta, consulta de precios y control de inventario. Al centralizar un catálogo multirubro diverso (golosinas, bebidas, comestibles, accesorios de telefonía y regalería), los socios propietarios y el personal de atención obtendrán información confiable en tiempo real sobre el stock disponible, precios actualizados y totales recaudados por turno, eliminando demoras en la atención, reduciendo los errores de cobro o /vuelto y asegurando la reposición oportuna de mercadería mediante alertas de stock mínimo.

Su impacto tecnológico consistirá en migrar la administración del negocio hacia un sistema informático, el cual permitirá dejar atrás de manera definitiva las herramientas analógicas que tienen, garantizando rapidez, consistencia en las transacciones y un cierre de caja más exacto, otorgando al comercio ventaja competitiva directa frente a los demás kioscos de la zona.

Definiciones del Proyecto y Sistema

Objetivo General del Proyecto

Desarrollar e implementar un sistema de información de escritorio para la gestión de inventario y punto de venta (POS) en el Kiosco "JRMARKET", utilizando el lenguaje Java y el motor de base de datos MySQL, que permita automatizar el registro de ventas, la consulta de precios, el control de stock y los cierres de caja.

Objetivos Específicos del Proyecto

- Mantener un registro histórico detallado de todas las transacciones comerciales realizadas en el establecimiento.
- Permitir la gestión y conformación dinámica de los productos que integran la compra de un cliente en el punto de venta (agregar, modificar cantidad o remover ítems).
- Administrar el catálogo de productos de diferentes tipos, estructurado por categorías comerciales.
- Permitir la búsqueda ágil de productos en el punto de venta mediante el ingreso manual de su código numérico o mediante su nombre o descripción.
- Automatizar el descuento de existencias de stock de los productos tras la confirmación de una venta.
- Automatizar el incremento de existencias de stock de los productos mediante el registro de compras de mercadería.
- Impedir la venta de productos que no cuenten con existencias disponibles en el inventario mediante la emisión de alertas y bloqueo de la operación.
- Implementar alertas visuales preventivas para aquellos artículos que alcancen o se encuentren por debajo de su nivel de stock mínimo configurado.

- Registrar las ventas permitiendo al operador seleccionar el tipo de medio de pago utilizado por el cliente (Efectivo, Transferencia).
- Permitir que el operador realice el cierre de caja de su turno.

Objetivo General del Sistema

Diseñar e implementar una aplicación de escritorio para procesar ventas en mostrador, gestionar las altas y bajas de existencias en el inventario, registrar medios de pago y emitir reportes de balance y arqueo de caja manual.

Límites

El sistema inicia con el alta de un artículo en el catálogo o el registro del ingreso de mercadería para elevar existencias, y finaliza con la confirmación de la venta en el mostrador, la deducción automática del stock en el inventario y la emisión del cierre de caja manual del turno correspondiente.

Alcance

- Alta, baja y modificación de productos.
- Búsqueda de artículos por código o nombre o descripción.
- Actualización de stock.
- Descuento automático de existencias tras cada venta concretada.
- Bloqueo operativo ante intentos de venta de productos sin existencias disponibles.
- Alertas de productos en estado crítico de reposición.
- Carga y modificación de ítems en la transacción de venta.
- Cálculo del subtotal por ítem y del total general de la operación.
- Selección simple del tipo de medio de pago para registrar la operación.
- Registro histórico de las transacciones de ventas efectuadas.
- Generación del reporte de cierre de caja por turno a solicitud del operador, totalizando ventas discriminadas por ti po de medio de pago.
- Control de Permisos de Usuarios y Login.

Involucrados con el Sistema

Si bien los socios propietarios cumplen simultáneamente las tareas de venta y de reposición en turnos distintos, en el presente trabajo ambas responsabilidades se documentan por separado, previendo una eventual delegación futura de la atención al público.

- **Socios Propietarios (Administradores / Operadores):** Definen precios, dan de alta nuevos rubros, registran el ingreso de compras de mercadería al inventario, auditan los cierres de caja y atienden las ventas en sus respectivos turnos.
- **Vendedor:** Opera la terminal de venta en el mostrador, consulta precios, registra ventas y ejecuta el cierre manual de su turno.
- **Repositor:** Genera listas de compras de productos, realiza las compras semanales de reposición y despacha en el depósito.

No Contempla

- Facturación electrónica.
- Integración con Mercado Pago (el cobro por transferencia se registra manualmente en la misma venta).
- Módulo de pasarela de pago.
- Liquidación de sueldos o contabilidad impositiva.
- Venta online por plataforma e-commerce.
- Entregas a domicilio.

Elicitación

Actividad del Cliente y Marco Normativo

El comercio minorista de kiosco y minimarket se rige por normas comerciales estándar y directivas municipales de salubridad y bromatología para la venta de alimentos envasados, bebidas y golosinas. Requiere una alta rotación de inventario con productos que poseen fechas de vencimiento y precios variables según la inflación del mercado.

Marco Regulatorio y Legal

La actividad cotidiana y el flujo de comercialización del negocio están regidos por un marco normativo multidimensional que abarca disposiciones del orden nacional, provincial y municipal:

- **Expendio de Bebidas Alcohólicas:** Rige la prohibición de venta a menores de 18 años a nivel federal (Ley N° 24.788, 1997), restricción horaria estricta que prohíbe el expendio al público en kioscos y despensas entre las 23:00 y las 08:00 horas (Ley N° 7.646, 2007).
- **Venta y Precios de Tabaco:** La comercialización de cigarrillos prohíbe el fraccionamiento en unidades sueltas y la venta a menores (Ley N° 26.687, 2011). Asimismo, las resoluciones de la Agencia de Recaudación y Control Aduanero (ARCA) exigen a los minoristas respetar las listas oficiales de precios de venta al público fijadas por las tabacaleras, sin recargos en mostrador.
- **Salubridad Alimentaria:** La exhibición y conservación de comestibles y bebidas se rige por el Código Alimentario Argentino (Ley N° 18.284, 1971), requiriendo control de fechas de vencimiento y conservación de cadena de frío.
- **Transparencia en Precios y Medios de Cobro:** La Ley de Defensa del Consumidor (Ley N° 24.240, 1993) obliga a exhibir el precio final de contado de manera clara e inequívoca, coincidiendo con el valor cobrado en caja. Tributariamente, el negocio opera bajo el Régimen Simplificado (Monotributo de ARCA e Ingresos Brutos de ATM Mendoza), rigiendo la obligación de aceptar transferencias y pagos electrónicos.

Elicitación de TIC (Tecnologías de la Información y la Comunicación)

Siguiendo las pautas metodológicas de la ingeniería de software, se estudian las herramientas y tecnologías aplicables para la etapa de análisis, diseño y posterior construcción:

Hardware

Respecto al Hardware que debería emplearse se encontraron las siguientes opciones:

- **Computadoras Portátiles (Laptops / Notebooks)**
    - Concepto: Es un sistema informático autónomo que integra en un chasis único la pantalla, el teclado, el panel táctil (trackpad), los componentes de procesamiento interno y una batería de respaldo.
    - Ventajas:
        - Portabilidad y movilidad: Permite trasladar el equipo fuera del establecimiento al concluir el turno o guardarlo con facilidad.
        - Respaldo de energía incorporado: La batería interna permite continuar operando ante cortes breves del suministro eléctrico general, posibilitando guardar datos y cerrar transacciones de forma segura.
        - Espacio reducido: Ocupa una superficie mínima en el mostrador y disminuye el cableado a la vista.
    - Desventajas:
        - Capacidad de actualización limitada: La mayoría de sus componentes esenciales vienen integrados o soldados directamente a la placa madre (como la memoria RAM en equipos modernos), impidiendo ampliaciones futuras.
        - Menor rendimiento por inversión: Conforme a los análisis comparativos de hardware, un equipo portátil suele costar más que una computadora de escritorio de potencia y almacenamiento equivalentes.
        - Vulnerabilidad en mostrador: Al encontrarse el teclado integrado sobre los circuitos principales, caídas accidentales de líquidos o el desgaste mecánico intensivo de las teclas pueden dañar la placa base y requerir reparaciones complejas.
- **Computadoras de Escritorio (Desktop PC)**
    - Concepto: Es una estación de trabajo modular orientada a operar de manera fija, que separa la torre o gabinete (donde se ubica la placa madre, procesador, disco y fuente de energía) de los dispositivos externos de visualización y entrada (monitor, teclado y ratón).
    - Ventajas:
        - Mejor relación precio-rendimiento y refrigeración: Al contar con un chasis espacioso y ventiladores de mayor diámetro, permite un flujo de aire continuo que evita la limitación térmica (thermal throttling) ante jornadas comerciales continuadas de 12 a 16 horas.
        - Facilidad de mantenimiento y actualización: Permite sustituir o mejorar de manera individual unidades de disco, memoria RAM o fuentes de alimentación con componentes estándar del mercado.

        - Periféricos externos reemplazables: El teclado y el ratón se vinculan por puertos independientes (USB), lo que permite sustituir de inmediato y a bajo costo un periférico dañado por el uso continuo de mostrador sin interrumpir el funcionamiento de la computadora.
    - Desventajas:
        - Falta de movilidad: Requiere una ubicación física fija en el mueble de caja y una canalización ordenada de los cables.
        - Dependencia del suministro eléctrico: No posee batería interna ante interrupciones de energía, requiriendo un estabilizador o sistema de alimentación ininterrumpida (UPS) externo para evitar apagados imprevistos.

En el caso del Kiosco "JRMARKET", es recomendable utilizar una computadora de escritorio estándar en el mostrador principal, debido a la facilidad de recambio de componentes internos si alguno falla. Además, al tratarse de un puesto fijo, la relación precio-beneficio favorece a un equipo de escritorio frente a un portátil. Sin embargo, están plasmadas las ventajas y desventajas de una notebook por si se pensara elegir una.

Infraestructura de Red y Conectividad

Respecto a la conexión a Internet y la estructuración de la red para la transmisión de datos, se analizan los siguientes tipos:

- **Acceso por ADSL (Línea Digital de Suscriptor Asimétrica)**
    - *Concepto:* Servicio tradicional de acceso a Internet de banda ancha provisto a través del par de cobre de la línea telefónica analógica.
    - *Funcionamiento:* Utiliza frecuencias superiores a las empleadas para la voz mediante técnicas de multiplexación, permitiendo hablar por teléfono y transferir datos simultáneamente. Requiere un módem con filtro (*splitter*) en el local y un multiplexor DSLAM en la central telefónica para separar el tráfico hacia la red pública.
- **Acceso por Cablemódem (HFC - Híbrido Fibra-Coaxial)**
    - *Concepto:* Servicio brindado por empresas de televisión por cable mediante redes mixtas que combinan troncales de fibra óptica con distribución final domiciliaria en cable coaxial.
    - *Funcionamiento:* El módem en el local se conecta a la bajada coaxial, modulando los datos en frecuencias que no interfieren con la transmisión de señales de televisión sobre el mismo conductor físico.
- **Acceso por Fibra Óptica (FTTH - Fibra hasta el Hogar / Local)**
    - *Concepto:* Medio físico de transmisión guiado que transporta pulsos de luz modulados a través de filamentos delgados de vidrio o silicio puro.
    - *Funcionamiento:* La fibra óptica ingresa de forma directa al local comercial y se conecta a una Terminal de Red Óptica (ONT/Módem), brindando anchos de banda simétricos de alta velocidad, latencia mínima y total inmunidad ante el ruido electromagnético generado por motores o transformadores de heladeras comerciales.
- **Acceso por Red de Telefonía Móvil Celular (4G LTE / 5G)**

    - *Concepto:* Transmisión de datos sin cables soportada sobre redes de antenas y celdas geográficas de telefonía móvil.
    - *Funcionamiento:* El enlace se efectúa mediante módems inalámbricos o teléfonos móviles provistos de tarjetas SIM de datos, permitiendo conectividad inmediata sin tendidos físicos de cable, aunque la estabilidad y la velocidad efectiva quedan condicionadas a la cobertura regional y la saturación de la celda de la compañía proveedora.
- **Red de Área Local Cableada (LAN Ethernet - IEEE 802.3)**
    - *Concepto:* Red de comunicación que interconecta equipos y periféricos dentro del local comercial mediante cables físicos.
    - *Funcionamiento:* Emplea cableado de par trenzado UTP Categoría 6 con conectores RJ-45 interconectado al router, garantizando transferencias a velocidad Gigabit constantes, sin latencia ni riesgo de caídas de señal.
- **Red de Área Local Inalámbrica (WLAN / Wi-Fi - IEEE 802.11)**
    - *Concepto:* Sistema de transmisión de datos local que prescinde de cables físicos, empleando radiofrecuencia en bandas de 2.4 GHz y 5 GHz emitidas desde un Access Point o router. Ofrece movilidad pero puede sufrir atenuaciones al atravesar tabiques o estructuras metálicas como heladeras exhibidoras.
- **Red de Área de Almacenamiento (SAN - Storage Area Network)**
    - *Concepto:* Red especializada de alta velocidad diseñada para interconectar múltiples servidores a matrices consolidadas de almacenamiento masivo de discos.
    - *Funcionamiento:* Los servidores acceden a los bloques de almacenamiento compartidos como si se tratara de unidades de disco locales, orientándose a grandes centros de cómputo que requieren redundancia masiva y recuperación rápida ante desastres.

En el caso del Kiosco 'JRMARKET', la comunicación entre la aplicación de ventas y el motor de base de datos se realiza internamente dentro de la misma computadora mediante la interfaz local (localhost / 127.0.0.1), lo que garantiza tiempos de respuesta en milisegundos y operatividad continua sin depender de ninguna red para registrar ventas. No obstante, para vincular el equipo de mostrador con el módem/router comercial (con acceso por Fibra Óptica FTTH o Cablemódem) se recomienda una conexión cableada LAN Ethernet, asegurando un enlace a Internet estable para tareas administrativas de los socios y dejando la infraestructura lista ante una eventual terminal adicional a futuro. En paralelo, la red Wi-Fi del router permite a los dueños verificar en sus teléfonos la acreditación de pagos por transferencia.

Herramientas de Modelado y Diseño

Lenguaje Unificado de Modelado (UML): Kendall y Kendall (2011) señalan que UML es el estándar para modelar sistemas orientados a objetos. Para este dominio resultan de interés:

- **Diagrama de Casos de Uso:** Permite identificar actores (vendedor, administrador), casos de uso que son funcionalidades que el sistema ofrece y sus relaciones.
- **Modelo de Dominio:** es una representación conceptual de los objetos, conceptos y relaciones importantes de un determinado problema o negocio. Describe qué elementos

existen en el ámbito que se está analizando, sin explicar todavía cómo se programará el sistema.

- **Diagrama de Clases:** Es un diagrama de UML que representa la estructura estática de un sistema. Muestra las clases que lo componen, sus características y la forma en que se relacionan.
- **Diagramas de Actividades:** Útiles para representar los flujos de negocio, tales como el flujo cobro o descuento de stock.

Tipos y Arquitecturas de Servidores de Bases de Datos

Conforme a lo documentado por Servercore, un servidor es un programa o dispositivo que ofrece funcionalidad a otros programas denominados clientes bajo el modelo cliente-servidor. Un servidor de base de datos es un software especializado encargado de mantener y compartir datos estructurados en tablas a través de la red, garantizando la integridad de las transacciones y atendiendo peticiones concurrentes.

En cuanto al entorno de alojamiento y despliegue de la base de datos, se analizan dos esquemas arquitectónicos:

- **Alojamiento On-Premise**
    - *Concepto:* La base de datos y su sistema gestor se ejecutan en servidores y equipos que el negocio posee y controla físicamente dentro de sus instalaciones.
    - *Ventajas:*
        - Control total y soberanía de los datos: La organización mantiene el control directo de su información, configuraciones de seguridad y políticas de acceso.
        - Operación continua sin depender de Internet: Al ejecutarse en el equipo local mediante la dirección de bucle local (localhost o IP 127.0.0.1) mediante conectores directos como JDBC, las ventas, consultas de precios y actualización de existencias funcionan con tiempos de respuesta en milisegundos y sin riesgo de interrupción ante caídas del servicio de Internet del local.
        - Ausencia de costos recurrentes de suscripción: Elimina el pago continuo de abonos mensuales a proveedores de infraestructura externa.
    - *Desventajas:*
        - Responsabilidad de administración y respaldos: El local debe asumir la ejecución de copias de seguridad manuales o programadas en soportes externos para prevenir pérdidas ante daños físicos en el equipo.
- **Alojamiento en la Nube**
    - *Concepto:* La base de datos se delega a la infraestructura de un proveedor externo de servicios en la nube (como AWS, Google Cloud o Microsoft Azure), accediendo mediante Internet.
    - *Ventajas:*
        - Accesibilidad remota: Permite consultar los registros e inventarios desde cualquier ubicación geográfica.
        - Mantenimiento y respaldos delegados: Las tareas de parches del sistema y copias de seguridad automáticas quedan a cargo del proveedor del servicio.

    - *Desventajas:*
        - Dependencia crítica de conectividad: Cualquier corte, caída o intermitencia en el proveedor de Internet paraliza el punto de venta e impide procesar transacciones en el mostrador.
        - Costos continuos: Exige el pago regular de suscripciones mensuales y tarifas por consumo de almacenamiento y transferencia de datos.

En el caso del Kiosco "JRMARKET", es recomendable utilizar una arquitectura de servidor local on-premise / standalone (instalando el motor relacional MySQL en la misma computadora del mostrador), dado que el local opera con un único punto de venta activo. Esta configuración elimina gastos recurrentes de almacenamiento en la nube, simplifica la administración técnica y asegura que la caja y el control de inventario permanezcan totalmente operativos ante eventuales cortes del enlace a Internet.

Análisis de la Competencia

Se realizó una investigación del mercado de software de gestión comercial y puntos de venta (POS) en la República Argentina, identificando tres productos representativos de los distintos paradigmas de despliegue:

- **Dux Software (Solución SaaS / Nube):**
    - Descripción y Alcance: Sistema de gestión comercial y punto de venta operado íntegramente bajo la modalidad Software as a Service (SaaS) alojado en la nube. Está orientado a micro y pequeñas empresas comerciales, integrando ventas, control de inventario en tiempo real, sincronización con tiendas electrónicas y facturación electrónica autorizada por ARCA.
    - Modelo de Costos y Limitaciones: Opera mediante planes de suscripción mensual recurrente (Plan Inicial desde $31.500 + IVA por mes con tope de 250 comprobantes mensuales, escalando a planes Estándar y Pro de hasta $97.500 + IVA mensuales). Su mayor vulnerabilidad para el mostrador de proximidad es su arquitectura 100% web: exige una conexión a Internet continua y estable en la caja, quedando inoperativo ante cortes del proveedor de telecomunicaciones o degradación de la señal.
- **Dragonfish - Zoo Logic (Sistema Comercial Tradicional On-Premise / Híbrido):**
    - Descripción y Alcance: Software de punto de venta y gestión comercial consolidado en el segmento minorista y cadenas comerciales de Argentina. Se instala en estaciones de trabajo locales y permite administrar inventarios con variables complejas (talles, colores, códigos de barras), controladores fiscales, emisión de comprobantes, cuentas corrientes y módulos de Business Intelligence integrados con su plataforma central zNube.
    - Modelo de Costos y Limitaciones: Aunque provee alta estabilidad local, su diseño está pensado para tiendas de indumentaria, zapaterías o cadenas con estructuras contables avanzadas. Presenta una interfaz de usuario cargada de opciones y ventanas secundarias que atentan contra la velocidad de tipeo requerida en un

kiosco, demandando además licencias comerciales corporativas y abonos mensuales de soporte técnico y mantenimiento.

- **FácilVirtual Kioscos (Software Enlatado de Escritorio para Kioscos):**
    - Descripción y Alcance: Aplicación empaquetada desarrollada específicamente para el rubro de kioscos y almacenes de barrio en Argentina. Se instala localmente bajo entorno Windows (exigiendo requerimientos de hardware mínimos como 2 GB de RAM y 100 MB de disco) e incluye funcionalidades básicas como lectura de código de barras, listados de reposición, cálculo de vuelto en pantalla y avisos de stock mínimo.
    - Modelo de Costos y Limitaciones: Se comercializa bajo licencias fijas (promociones de adquisición que rondan desde $59.999 hasta más de $180.000 según módulos fiscales incorporados). Presenta rigidez en su diseño cerrado: carece de adaptabilidad para configurar flujos transaccionales híbridos personalizados (diferenciación inmediata entre efectivo y transferencias con registro del alias de los socios) y sus reportes de caja no se ajustan a la dinámica de turnos rotativos que requiere el local.

Los criterios seleccionados responden a las necesidades insatisfechas del Kiosco "JRMARKET" y a los requerimientos no funcionales del mostrador:

- **Disponibilidad e Independencia de Internet:** Capacidad de procesar ventas, consultar precios y actualizar inventario cuando se interrumpe la conexión externa del comercio.
- **Control Transaccional de Stock:** Registro automatizado de bajas por ventas y altas por compras en tiempo real, con detección de faltantes.
- **Alertas Visuales de Reposición Crítica:** Aviso inmediato en la pantalla operativa cuando las existencias igualan o descienden por debajo del umbral mínimo de seguridad.
- **Búsqueda Ágil de Artículos:** Rapidez para localizar productos mediante el ingreso manual de su código o por su nombre sin saturar de campos modales al operador.
- **Selección del Medio de Pago:** Registro directo y flexible del tipo de medio de pago utilizado por el cliente para el balance de caja.
- **Estructura de Costos:** Nivel de inversión inicial, pagos recurrentes periódicos e indexaciones por inflación.
- **Curva de Aprendizaje y Usabilidad:** Nivel de complejidad de la interfaz para operadores no técnicos durante horarios pico de despacho.

Criterio Evaluado	Método Manual Actual	Dux Software	Dragonfish - Zoo Logic	FácilVirtual Kioscos	Sistema a Medida Propuesto
Instalación local	No (Soporte físico en papel)	No (Vía web)	Sí (Motor SQL y terminales)	Sí (Asistente estándar Windows)	Sí (Base de datos local en misma PC)
Compatibilidad multiplataforma	No aplica (Manual)	Sí (Cualquier navegador web)	No (Solo entornos Windows)	No (Solo entornos Windows)	Sí (Java; desplegado en Windows)
Mantenimiento y actualizaciones	No (Inexistente)	Sí (Automático por el proveedor)	Parcial (Parches manuales y soporte técnico)	Parcial (Sujeto a vigencia de licencia)	Sí (Evolutivo a cargo del desarrollador)
Independencia de Internet	Sí	No	Sí	Sí	Sí
Cero costo de licenciamiento continuo	Sí	No (Abono mensual indexable)	No (Licencia inicial alta + abonos)	Parcial (Pago inicial por máquina)	Sí (Desarrollo propio sin mensualidades)
Registro de venta en mostrador	Parcial (Registro manual	Sí	Sí	Sí	Sí

	incompleto)				
Selección de Medio de Pago	No	Sí	Sí	Sí	Sí
Búsqueda de artículos	No	Sí (Por código o texto)	Sí (Por código y filtros avanzados)	Sí (Por lector óptico o descripción)	Sí (Por código de producto o nombre)
Descuento automático de stock	No	Sí	Sí	Sí	Sí
Alerta visual de stock crítico	No	No	No	Sí	Sí
Registro de compras de mercadería	Sí (Manualmente)	Sí	Sí	Sí	Sí
Cierre de caja	Sí (Manualmente)	Sí	Sí	Sí	Sí (Cierre por turno)
Curva de aprendizaje y usabilidad	Nula	Media	Alta	Media	Baja (Interfaz simple)

Relevamiento

Relevamiento Estructural

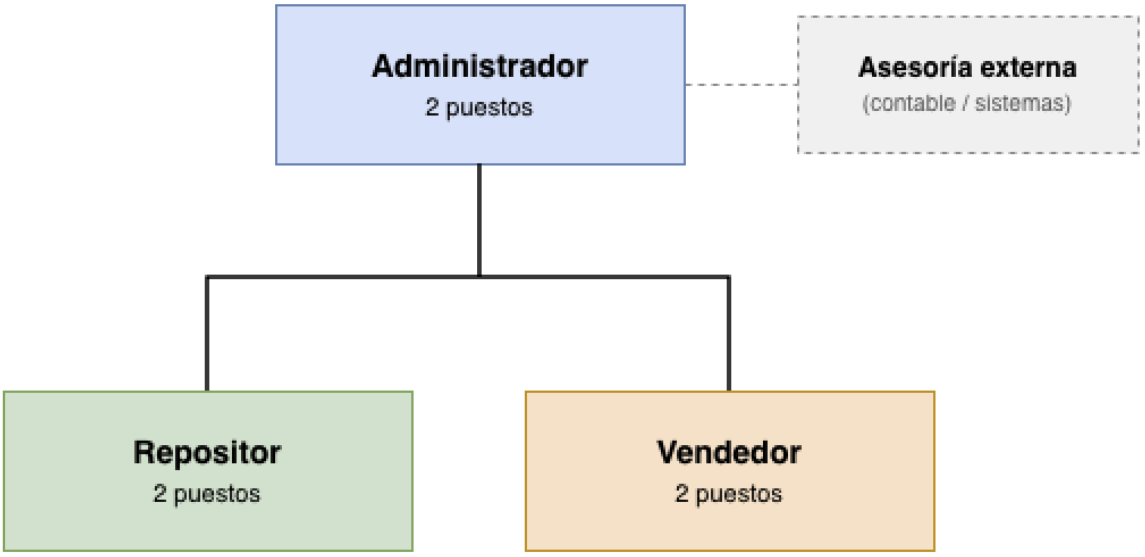
Ubicación geográfica:
El Kiosco "JRMARKET" se encuentra ubicado físicamente en la calle San Miguel 575, en el departamento de Las Heras, Provincia de Mendoza. El inmueble comercial está emplazado sobre una arteria urbana de alta circulación peatonal y vehicular, colindando de forma directa con un gimnasio de alta concurrencia diaria. El establecimiento cuenta con un salón principal de ventas con un mostrador único de atención al público al frente, stands modulares y estanterías de reducidas dimensiones para la exhibición interna de mercadería variada (golosinas, comestibles envasados, artículos de perfumería menor, juguetería y accesorios de telefonía móvil), además de dos unidades de frío comercial: una heladera exhibidora vertical para bebidas y una heladera horizontal de piso para lácteos, productos congelados y bebidas de reserva. En la parte posterior del inmueble dispone de una habitación independiente que funciona como depósito trasero para el almacenamiento de bultos cerrados y mercadería de reposición.

Equipamientos existentes:
Actualmente el Kiosco "JRMARKET" carece por completo de equipamiento informático en su puesto de atención. La totalidad de las tareas comerciales se realizan de forma manual en el mostrador, disponiendo de un cajón manual para el resguardo físico del dinero en efectivo y el cambio operativo (con un fondo fijo inicial de $20.000 en billetes de baja denominación), una libreta de papel para el registro manuscrito de ventas, una lapicera, una calculadora de mano básica y un cartel con el Alias de Mercado Pago pegado junto a la caja para cobros por transferencia digital. El local no posee una red informática cableada ni servidores; su conectividad externa se apoya en un módem(router) comercial con conexión a Internet y enlace Wi-Fi, utilizado por los socios en sus teléfonos celulares personales para verificar la acreditación de pagos digitales.

Relevamiento Funcional

Organigrama:

- **Nivel Directivo (Administración General):** Integrado por el cargo de Administrador, correspondiente a los socios propietarios, quienes ejercen la conducción general del comercio, definen políticas de precios, gestionan los recursos económicos y coordinan las operaciones.
- **Órgano de Staff (Asesoría Externa):** Unidad de asistencia técnica externa en materia Contable y de Sistemas, brindando soporte a la administración en operaciones contables.
- **Nivel Operativo (Abastecimiento y Ventas):**
    - Repositor: Responsable de las compras mayoristas, la recepción de mercadería, el acopio en el depósito trasero y la reposición física continua en las estanterías y heladeras del salón de ventas.
    - Vendedor: Responsable de la atención al público en mostrador durante los turnos comerciales, la búsqueda de artículos en el sistema, el registro de la venta y el cobro en caja.



Fuente: Elaboración propia.
Figura 1: Organigrama funcional del comercio JRMARKET.

Funciones de las áreas:

- Administrador:
    - Definir y actualizar los precios de venta al público
    - Calcular márgenes de ganancia sobre los distintos productos del catálogo.
    - Administrar los fondos globales del negocio y planificar la distribución mensual de ingresos.
    - Supervisar y auditar los cierres de caja rendidos al término de cada turno de mostrador.
    - Autorizar la incorporación o baja de rubros comerciales en el catálogo.
    - Gestionar y archivar la documentación comercial y tributaria de compras (Facturas "B" de mayoristas bajo el régimen simplificado de Monotributo y ATM Mendoza).
- Repositor:
    - Relevar el estado físico del inventario e identificar necesidades de reposición de mercadería crítica.
    - Efectuar las compras presenciales de fin de semana (sábados o domingos) en distribuidores mayoristas de la zona (Tadicor y Oscar David).
    - Retirar los fondos asignados de la caja común para abonar a los proveedores mayoristas y exigir los comprobantes Factura "B".
    - Cotejar que los artículos físicos recibidos coincidan en cantidad y descripción con el detalle de la factura de compra mayorista.
    - Descargar, ordenar y clasificar los bultos cerrados, bebidas y cajas dentro del depósito trasero del local.

    - Trasladar periódicamente la mercadería desde el depósito hacia el salón de ventas para abastecer estanterías, stands y vitrinas.
    - Cargar y acondicionar diariamente las bebidas y productos lácteos en la heladera vertical y en la heladera de piso, preservando la cadena de frío.
    - Controlar fechas de vencimiento de productos en góndolas y depósito.
- Vendedor:
    - Atender al público en el mostrador único durante la franja horaria asignada (Turno Mañana de 09:00 a 15:00 hs o Turno Tarde/Noche de 15:00 a 24:00 hs).
    - Realizar la búsqueda y consulta de precios en el sistema mediante el ingreso manual del código numérico o la digitación del nombre/descripción del artículo.
    - Conformar la transacción comercial agregando, modificando cantidades o eliminando productos solicitados por el cliente.
    - Registrar la venta seleccionando en pantalla el tipo de medio de pago utilizado por el cliente (Efectivo o Transferencia).
    - Custodiar el cajón de dinero, resguardar el fondo fijo inicial de $20.000 y entregar el vuelto exacto en operaciones en efectivo.
    - Constatar en el teléfono celular la notificación visual de acreditación cuando el cliente abona mediante transferencia digital.
    - Notificar al Repositor sobre quiebres de existencias detectados en estanterías y heladeras durante la atención.
    - Ejecutar el cierre y balance de caja por turno al concluir la jornada de servicio, rindiendo los importes recaudados discriminados por tipo de medio de pago a la administración o al siguiente operador.

Proceso de Negocio

Proceso de Venta de Productos

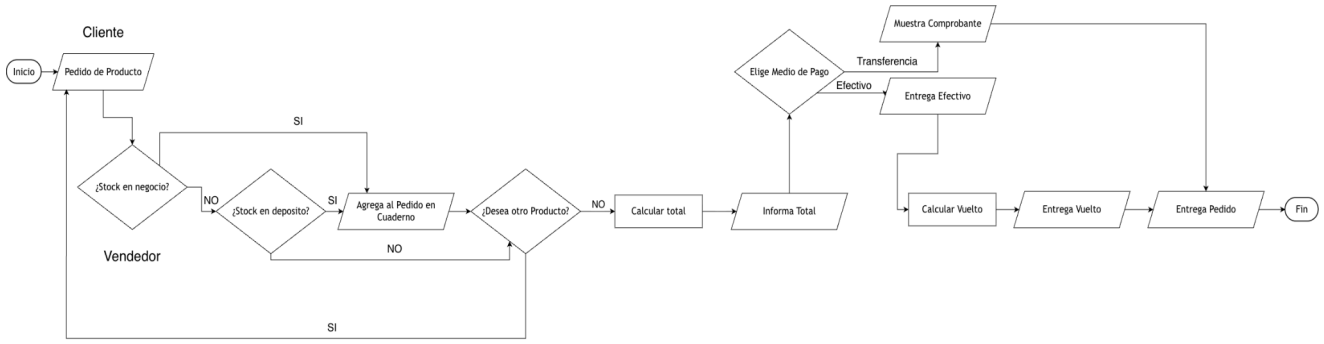
Roles:

- Cliente: Concurre al mostrador, solicita los productos que desea adquirir, define el medio de pago, entrega el dinero o exhibe el comprobante de transferencia y recibe el pedido.
- Vendedor: Atiende al cliente, verifica la disponibilidad de stock en salón y depósito, anota los ítems en el cuaderno, calcula e informa el total, procesa el cobro (calculando y entregando vuelto o verificando comprobante) y entrega la mercadería.

Descripción:

1. El Cliente realiza el pedido de un producto en el mostrador.
2. El Vendedor verifica si existe stock del artículo en el negocio:
    a. Si hay stock: El Vendedor agrega el producto al pedido en el cuaderno y pasa al paso 4.
    b. Si no hay stock: El Vendedor verifica si existe stock en el depósito trasero.

3. El Vendedor revisa la disponibilidad en el depósito:
    a. Si hay stock en depósito: El Vendedor agrega el producto al pedido en el cuaderno.
    b. Si no hay stock en depósito: No se agrega el artículo y se continúa directamente al paso 4.
4. El Vendedor consulta al Cliente si desea solicitar otro producto:
    a. Si el Cliente desea otro producto: El flujo regresa al paso 1 para procesar el nuevo ítem.
    b. Si el Cliente no desea otro producto: El Vendedor procede al cálculo del importe total de la compra.
5. El Vendedor calcula el total acumulado de los productos registrados e informa el monto al Cliente.
6. El Cliente elige el medio de pago a utilizar:
    a. Si elige Efectivo:
        i. El Cliente entrega el dinero en efectivo.
        ii. El Vendedor calcula el vuelto correspondiente.
        iii. El Vendedor realiza la entrega del vuelto al Cliente.
        iv. El Vendedor entrega el pedido al Cliente.
    b. Si elige Transferencia:
        i. El Cliente realiza la transferencia.
        ii. Muestra el comprobante digital en su teléfono.
        iii. El Vendedor verifica el comprobante.
        iv. Entrega el pedido al Cliente.
7. Finaliza el proceso.



Fuente: Elaboración propia.
Figura 2: Diagrama de actividades del Proceso de Venta de Productos.

Proceso de Compra de Productos

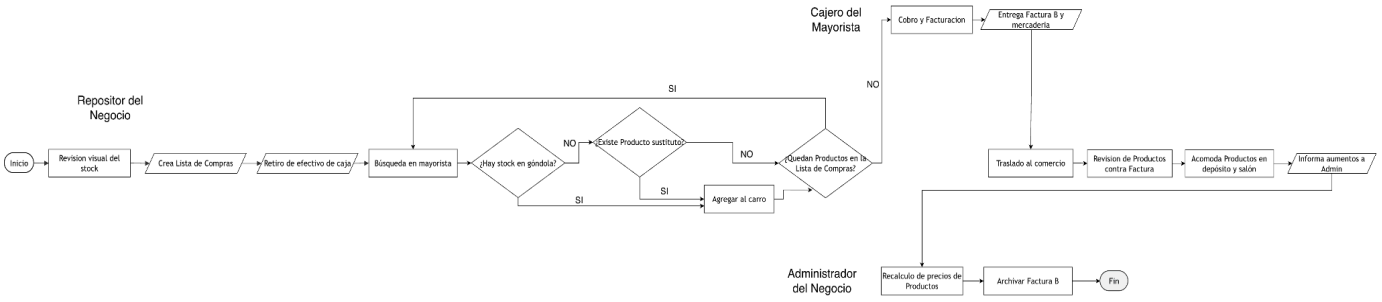
Roles:

- Repositor: Es el encargado de relevar el stock faltante, armar la lista de compras, retirar el dinero, acudir al mayorista, adquirir y transportar la mercadería, controlar los bultos contra la factura, reponer los productos en salón y depósito, y reportar los aumentos de costos.
- Cajero del Mayorista: Registra los artículos en la línea de cajas del distribuidor mayorista, realiza el cobro en efectivo, emite la Factura "B" y entrega los productos.

- Administrador: Es el responsable de recibir la información sobre los aumentos, recalcular manualmente los precios de venta al público para el salón y archivar físicamente la factura.

Descripción:

1. El Repositor inspecciona visualmente las estanterías del salón y del depósito trasero para identificar artículos agotados o próximos a acabarse.
2. El Repositor elabora una lista de compras manuscrita en papel con los artículos a reabastecer.
3. El Repositor retira de la caja del comercio el dinero en efectivo necesario para la compra.
4. El Repositor va al distribuidor mayorista y busca los artículos anteriormente anotados en la lista:
    a. Si algún Producto de la lista no está en el mayorista: Se busca un Producto sustituto.
        i. Si existe sustituto se incorpora al carro de compras.
        ii. Si no, no se adquiere y se continúa con la compra.
    b. Si el Producto de la lista existe, se agrega al carro de compras.
5. El Cajero del Mayorista registra los productos en la línea de cajas, cobra el importe en efectivo, emite la Factura "B" y entrega la mercadería.
6. El Repositor traslada los artículos hacia el kiosco.
7. El Repositor coteja físicamente las cantidades y productos recibidos contra el detalle impreso en la Factura "B".
8. El Repositor acomoda los bultos en el depósito trasero y traslada las unidades necesarias para abastecer las estanterías y heladeras del salón de ventas.
9. El Repositor se comunica con el Administrador para avisar de aumentos en determinados Productos.
10. El Administrador calcula, basado en el porcentaje de aumento de los Productos, los nuevos precios de venta al público.
11. El Administrador archiva la Factura "B" en formato físico y finaliza el proceso.



Fuente: Elaboración propia.

Figura 3: Diagrama de actividades del Proceso de Compra de Productos.

Diagnóstico

Proceso de Venta de Productos

Problemas:

- **Demoras por verificación física de existencias:** Cuando un artículo no está visible en el salón, el vendedor debe pausar la atención y trasladarse al depósito para constatar la disponibilidad, prolongando la espera del cliente.
- **Errores en precios y cálculo de totales/vueltos:** La determinación de precios de memoria o en papel, junto con la suma en calculadora de mano y el cálculo mental del vuelto, ocasiona diferencias de dinero en la caja diaria.
- **Registro manuscrito incompleto en horarios pico:** Durante momentos de alta afluencia, el asiento en el cuaderno se interrumpe o se posterga, dejando ventas sin registrar y desajustando el arqueo de caja.
- **Desactualización del stock y recuento manual:** Al no existir un descuento automático al momento de la venta, el control de inventario debe realizarse manualmente mediante el conteo y cálculo de los productos anotados en el cuaderno, provocando errores de registro, desconocimiento de existencias reales y quiebres imprevistos de mercadería.
- **Imposibilidad de conocer la rentabilidad y márgenes de ganancia:** Al registrar únicamente importes brutos sin vincular cada venta al costo del artículo ni guardar un histórico de la cotización aplicada, se desconoce si cada operación genera ganancia o pérdida real.

Causas:

- Ausencia de un catálogo digital centralizado que informe en tiempo real las existencias disponibles tanto en salón como en depósito.
- Ausencia de un software de Punto de Venta (POS) que aplique automáticamente los precios vigentes y calcule con exactitud los totales y vueltos.
- Dependencia exclusiva de métodos analógicos (cuaderno de papel, lapicera y calculadora) para el asentamiento de las transacciones comerciales.
- Inexistencia de una herramienta informática que descuente de forma automática el stock de los productos comercializados al confirmar la venta.
- Falta de un registro digital transaccional que almacene los costos y precios de venta por producto para el cálculo automático de márgenes comerciales.

Proceso de Compra de Productos

Problemas:

- **Compras imprecisas basadas en estimaciones visuales:** Al no contar con alertas automáticas de inventario ni parámetros de stock mínimo, la lista de compras depende exclusivamente de la memoria y la inspección ocular del repositor, provocando sobrestock de artículos de baja rotación o falta en los productos esenciales.

- **Falta de actualización de existencias:** Las compras a mayoristas no se registran en ninguna herramienta de gestión; la mercadería ingresa directamente a las estanterías sin actualizar un stock centralizado, perpetuando el desconocimiento sobre las cantidades reales disponibles.
- **Pérdida de margen por actualización manual y desfasada de precios:** La comunicación verbal de aumentos de costos y el cálculo artesanal de los nuevos precios de venta genera retrasos en la modificación de los valores en mostrador, provocando que se continúen vendiendo artículos a precios desactualizados por debajo del margen de ganancia requerido.
- **Ausencia de historial y control de compras:** El respaldo del aprovisionamiento reside únicamente en las Facturas "B" archivadas en papel, imposibilitando consultar de forma rápida la evolución histórica de costos por mayorista o auditar los egresos de dinero de la caja común.

Causas:

- Inexistencia de un módulo de inventario que identifique automáticamente los productos que están cerca de agotarse o agotados.
- Falta de una funcionalidad de registro de compras que permita cargar los comprobantes comerciales (Factura "B") e incremente de forma automática el stock de los productos adquiridos.
- Ausencia de un cálculo automático de precios de venta vinculado al costo de adquisición y al margen de ganancia definido por la administración.
- Dependencia de registros manuscritos y archivo físico de comprobantes sin ningún tipo de sistematización digital.

Propuesta de Solución

Propuesta

Diseñar e implementar un sistema de escritorio integral que centralice y automatice las operaciones diarias del comercio JRMARKET. La solución incorpora un subsistema de Seguridad y Control de Acceso que autentica a los usuarios mediante credenciales individuales y aplica un esquema de autorización basado en roles (Administrador, Vendedor y Repositor), garantizando que cada colaborador acceda exclusivamente a las funciones que le corresponden según su responsabilidad operativa.
Sobre esta base de seguridad, el sistema articula cuatro áreas operativas fundamentales:

- **Ventas y Cobros:** Automatiza la atención al público permitiendo la búsqueda ágil de artículos por código o catálogo, verificación de existencias en tiempo real, cálculo de totales y vuelto en efectivo, registro de transferencias y descuento atómico del inventario comercializado, almacenando costos y precios para auditar la rentabilidad del negocio.
- **Abastecimiento y Compras Mayoristas:** Detecta productos en nivel crítico para generar la lista sugerida de compras y registra el ingreso formal de mercadería con comprobante de proveedor, actualizando el stock disponible.
- **Pantalla de Gestión de Productos:** Centraliza la consulta de precios, rubros, marcas y existencias físicas, alertando sobre quiebres de inventario.

- **Cierre de Turno y Arqueo:** Concilia los cobros registrados en el sistema contra el dinero físico rendido por el vendedor al culminar su jornada, justificando diferencias y cerrando la sesión de caja de manera auditable.

Listado de Requerimientos Funcionales

- RF01: El sistema debe permitir dar de alta usuarios.
- RF02: El sistema debe permitir modificar usuarios.
- RF03: El sistema debe permitir dar de baja usuarios.
- RF04: El sistema debe permitir asignar roles a usuarios.
- RF05: El sistema debe permitir iniciar sesión con usuario y contraseña.
- RF06: El sistema debe permitir cerrar sesión.
- RF07: El sistema debe controlar el acceso a los módulos según el rol del usuario.
- RF08: El sistema debe permitir registrar marcas.
- RF09: El sistema debe permitir modificar marcas.
- RF10: El sistema debe permitir dar de baja marcas.
- RF11: El sistema debe permitir dar de alta proveedores mayoristas.
- RF12: El sistema debe permitir modificar proveedores mayoristas.
- RF13: El sistema debe permitir dar de baja proveedores mayoristas.
- RF14: El sistema debe permitir registrar tipos de productos.
- RF15: El sistema debe permitir modificar tipos de productos.
- RF16: El sistema debe permitir dar de baja tipos de productos.
- RF17: El sistema debe permitir dar de alta productos.
- RF18: El sistema debe permitir modificar productos.
- RF19: El sistema debe permitir dar de baja productos.
- RF20: El sistema debe permitir consultar stock de productos.
- RF21: El sistema debe permitir consultar el catálogo de productos.
- RF22: El sistema debe alertar por stock crítico de productos.
- RF23: El sistema debe generar lista sugerida de compras.
- RF24: El sistema debe permitir la emisión de una lista de compras.
- RF25: El sistema debe permitir registrar compras de mercadería.
- RF26: El sistema debe actualizar el stock por compra.
- RF27: El sistema debe verificar la disponibilidad del stock.
- RF28: El sistema debe calcular vuelto.
- RF29: El sistema debe descontar stock por venta.
- RF30: El sistema debe registrar las ventas de productos.
- RF31: El sistema debe realizar cierre de turno y arqueo de caja.
- RF32: El sistema debe permitir la conciliación del saldo de caja.

Listado de Requerimientos no Funcionales

- RNF01: El sistema debe operar bajo una arquitectura de escritorio local desarrollada en Java sobre el sistema operativo Windows.

- RNF02: El sistema debe estar conectado a una base de datos MySQL local, garantizando su funcionamiento ininterrumpido sin dependencia de conexión a Internet.
- RNF03: La interfaz debe ser intuitiva y permitir la operación completa de venta y navegación mediante atajos de teclado (Enter, Esc, Supr, F1 a F6) y teclado numérico para agilizar la atención al público.
- RNF04: Los tiempos de respuesta del sistema para la consulta de productos en catálogo, búsqueda de artículos y confirmación de transacciones deben ser inferiores a 1 segundo.
- RNF05: Las contraseñas de los usuarios deben almacenarse de forma cifrada.
- RNF06: El sistema debe garantizar transaccionalidad atómica y consistencia de datos (ACID).

Listado de Requerimientos Candidatos

- RC01: El sistema debe permitir el procesamiento y registro de pagos mediante tarjetas de débito y crédito.
- RC02: El sistema debe permitir la carga automatizada de comprobantes fiscales (Factura "B") mayoristas mediante importación de archivos o escaneo de datos impositivos.
- RC03: El sistema debe permitir la integración con Mercado Pago.
- RC04: El sistema debe permitir la captura e identificación automática de productos mediante la lectura de código de barras.
- RC05: El sistema debe permitir la exportación de los reportes de cierre de turno, balances comerciales e historial de ventas en formato pdf.

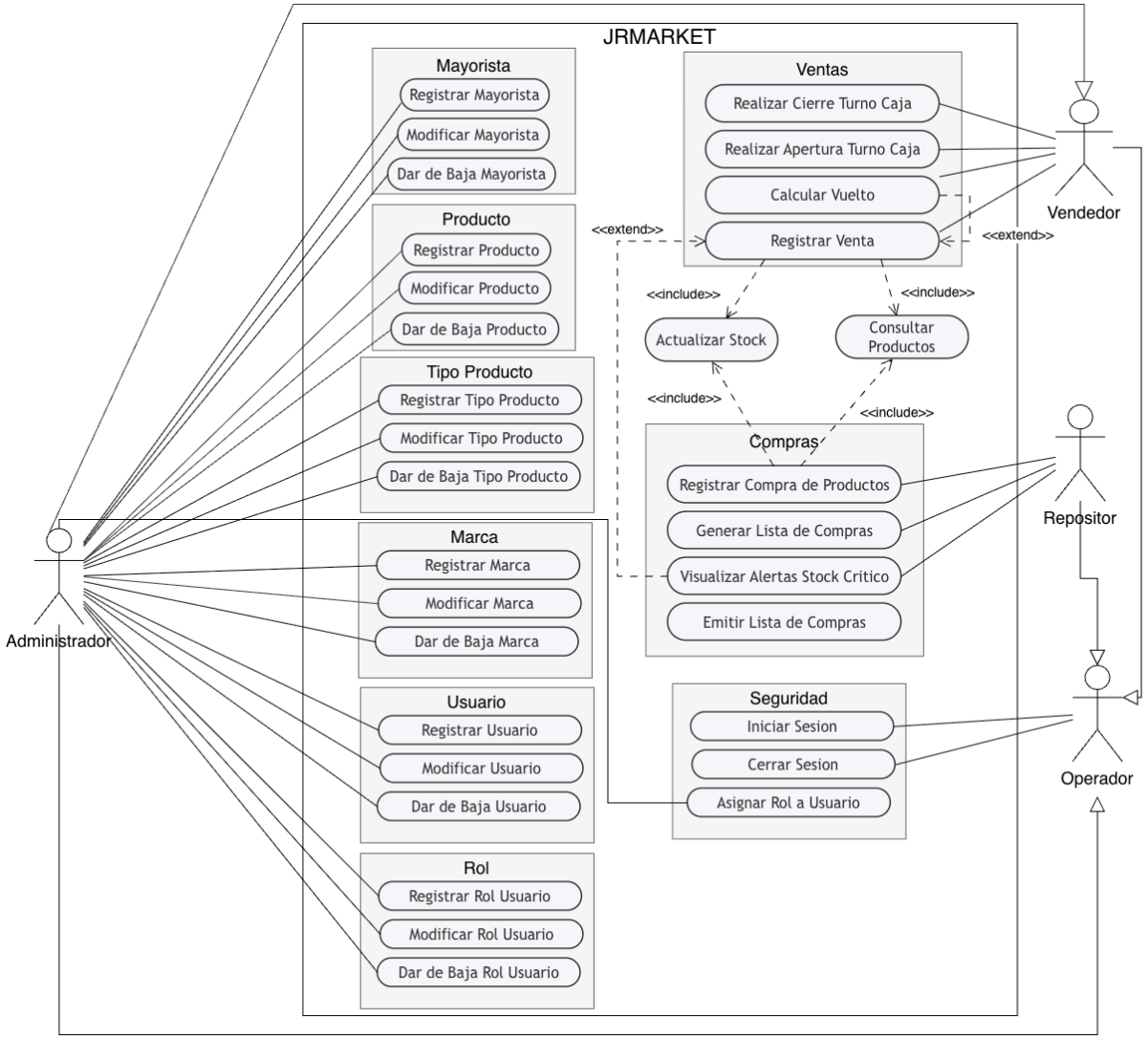
Listado de Casos de Uso

- CU001: Iniciar Sesión
- CU002: Cerrar Sesión
- CU003: Registrar Rol de Usuario
- CU004: Modificar Rol de Usuario
- CU005: Dar de Baja Rol de Usuario
- CU006: Registrar Usuario
- CU007: Modificar Usuario
- CU008: Dar de Baja Usuario
- CU009: Registrar Marca
- CU010: Modificar Marca
- CU011: Dar de Baja Marca
- CU012: Registrar Tipo de Producto
- CU013: Modificar Tipo de Producto
- CU014: Dar de Baja Tipo de Producto
- CU015: Registrar Mayorista
- CU016: Modificar Mayorista
- CU017: Dar de Baja Mayorista
- CU018: Registrar Producto
- CU019: Modificar Producto
- CU020: Dar de Baja Producto
- CU021: Consultar Productos

- CU022: Visualizar Alertas de Stock Crítico
- CU023: Generar Lista Sugerida de Compras
- CU024: Emitir / Imprimir Lista de Compras
- CU025: Registrar Compra de Productos
- CU026: Actualizar Stock
- CU027: Realizar Apertura de Turno de Caja
- CU028: Registrar Venta de Productos
- CU029: Verificar Disponibilidad de Stock
- CU030: Calcular Vuelto
- CU031: Realizar Cierre de Turno y Arqueo
- CU032: Conciliar Saldo de Caja

Desarrollo del Prototipo

Diagrama de Casos de Uso



Fuente: Elaboración propia.
Figura 4: Diagrama de Casos de uso general.

Comunicaciones

Descripción

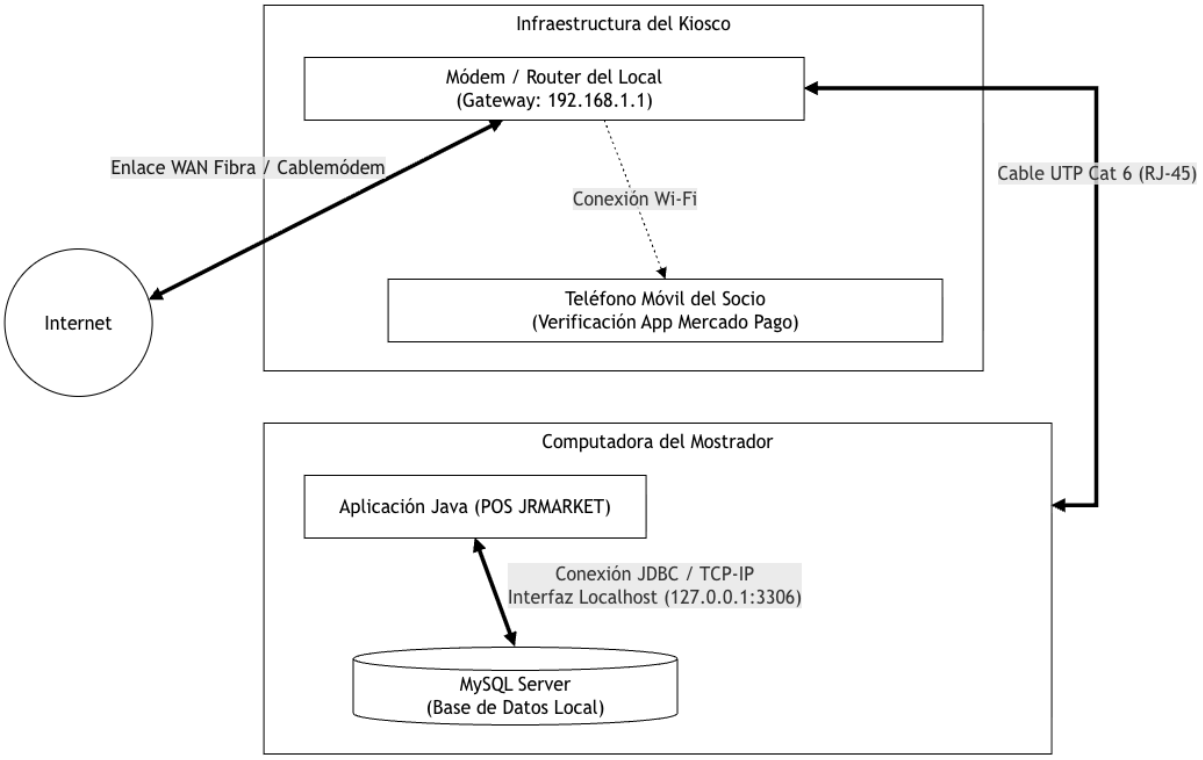
El sistema adopta una arquitectura de escritorio local monopuesto (Standalone / On-Premise) instalada en la computadora del mostrador:

- Capa de Datos y Persistencia: El motor de base de datos relacional (MySQL Server) reside en el Servidor Local y escucha peticiones entrantes a través del protocolo TCP/IP en el puerto estándar 3306.
- Capa de Aplicación y Clientes: La terminal de trabajo, que funciona como, Puesto Mostrador de Ventas y Puesto de Gestión de Compras y Reposición, ejecutan de forma local la

aplicación de escritorio empaquetada en Java. Además contiene la comunicación con la base de datos se realiza mediante el controlador oficial MySQL Connector/J (JDBC), gestionando un pool de conexiones optimizado para minimizar la latencia.

Políticas de Seguridad

Al tratarse de una arquitectura monopuesto local, el servicio de base de datos MySQL opera configurado exclusivamente sobre la dirección local, bind-address = 127.0.0.1, manteniendo el puerto 3306 cerrado a toda conexión entrante desde la red externa. La red inalámbrica del comercio cuenta con cifrado estándar WPA2-PSK (AES) provisto por el módem/router del proveedor Movistar, permitiendo a los propietarios conectar sus dispositivos móviles de manera segura para la constatación de cobros digitales sin exponer los datos del sistema de mostrador.



Fuente: Elaboración propia.

Figura 5: Diagrama de Comunicaciones y Conectividad Local.

Descripción de Casos de Uso

Registrar Producto

CU018	**Registrar Producto**
Versión	1.00 06/09/2026
Objetivos asociados	Ingresar la información de un nuevo producto al sistema.
Actor Principal	Administrador
Actor Secundario	No aplica.
Precondición	● El usuario debe estar registrado, con atributos suficientes para realizar las acciones y haber iniciado sesión en el sistema (CU001). ● Las marcas (CU009) y los tipos de producto (CU012) deben encontrarse registrados en la base de datos.
Postcondición	El producto queda cargado en el sistema y habilitado para la venta y compra.

Curso Normal	Curso Alternativo
1) El Administrador accede a la pantalla de Pantalla de Gestión de Productos desde el Menú Principal o presionando la tecla rápida F3.	
2) El operador presiona el botón "[F5] Nuevo Producto" (o presiona F5).	
3) El sistema despliega la ventana modal Formulario de Producto con los campos vacíos y el foco en el campo 'Código de Barras'.	
4) El operador completa los campos solicitados (código de barras, descripción, marca, tipo de producto, precio de costo, precio de venta y stock mínimo).	4.1) El operador presiona el botón "Cancelar" o la tecla Esc. 4.1.1) Se descartan los cambios y se vuelve al paso 2.
5) Se presiona el botón "Guardar" o [Enter].	5.1) Faltan datos en alguno de los campos obligatorios o el formato es incorrecto. 5.1.1) Se vuelve al paso 4. 5.2) El código de barras ingresado ya está siendo utilizado por un producto cargado. 5.2.1) Se vuelve al paso 4.
6) Fin del caso de uso.	

Parámetro	Detalle
Frecuencia esperada	Carga inicial masiva de 85 artículos; posteriormente 2 casos de uso por semana.
Importancia	Muy Importante

Modificar Producto

CU019	**Modificar Producto**
Versión	1.00 06/09/2026
Objetivos asociados	Actualizar la información, precios y parámetros de stock de un producto cargado en el sistema.
Actor Principal	Administrador
Actor Secundario	No aplica.
Precondición	● El usuario debe estar registrado, con atributos suficientes para realizar las acciones y haber iniciado sesión en el sistema (CU001). ● El producto a modificar debe estar cargado en el sistema.
Postcondición	Se actualiza la información del producto en la base de datos.

Curso Normal	**Curso Alternativo**
1) El Administrador accede a la pantalla de Pantalla de Gestión de Productos desde el Menú Principal o presionando F3.	
2) El operador localiza el producto mediante el campo de filtro rápido y lo selecciona en la grilla.	3.1) El producto buscado no aparece en la lista. 3.1.1) El producto no ha sido cargado. 3.1.2) Fin del caso de uso.
3) El operador presiona el botón "[F6] Modificar" (o presiona F6 o doble clic en la fila).	
4) El sistema despliega la ventana modal Formulario de Producto precargando los datos del artículo seleccionado.	
5) El operador modifica los campos requeridos (precios, costos o stock mínimo).	5.1) El operador presiona el botón "Cancelar" o la tecla Esc. 5.1.1) Se descartan las modificaciones y se vuelve al paso 3.

6) Se presiona el botón "Guardar".	6.1) Faltan datos en alguno de los campos obligatorios o el formato es incorrecto. 6.1.1) Se vuelve al paso 5.
7) Fin del caso de uso.	

Parámetro	Detalle
Frecuencia esperada	10 a 20 casos de uso por semana (actualizaciones de precios y costos de reposición).
Importancia	Muy Importante

Dar de Baja Producto

CU020	**Dar de baja Producto**
Versión	1.00 06/09/2026
Objetivos asociados	Inhabilitar un producto para que no pueda comercializarse en el sistema.
Actor Principal	Administrador
Actor Secundario	No aplica.
Precondición	● El usuario debe estar registrado, con atributos suficientes para realizar las acciones y haber iniciado sesión en el sistema (CU001). ● El producto a dar de baja debe estar cargado en el sistema.
Postcondición	Se actualiza la información del producto en la base de datos.

Curso Normal	**Curso Alternativo**
1) El Administrador accede a la pantalla de "Pantalla de Gestión de Productos" desde el "Menú Principal" o presionando "F3".	
2) El operador busca y selecciona en la grilla el producto a inhabilitar.	
3) El operador presiona el botón "[Supr] Dar de Baja" (o presiona la tecla Supr).	3.1) El producto buscado no aparece en la lista. 3.1.1) El producto no ha sido cargado. 3.1.2) Fin del caso de uso.
4) El sistema despliega un cuadro de diálogo de confirmación: "¿Está seguro de que desea dar	

de baja el producto seleccionado?".	
5) El operador presiona el botón "Aceptar".	5.1) El operador presiona el botón "Cancelar" o la tecla Esc. 5.1.1) Se descartan las modificaciones y se vuelve al paso 3.
6) Fin del caso de uso.	

Parámetro	**Detalle**
Frecuencia esperada	1 a 2 casos de uso por mes (artículos discontinuados o vencidos comercialmente).
Importancia	Muy Importante

Registrar Venta de Productos

CU028	**Registrar Venta de Productos**
Versión	1.00 06/09/2026
Objetivos asociados	Procesar la venta de mercadería en mostrador, verificando el stock disponible, liquidando subtotales, registrando el medio de pago, calculando el vuelto, emitiendo el comprobante y descontando existencias.
Actor Principal	Vendedor, Administrador
Actor Secundario	No Aplica.
Precondición	● El usuario debe estar registrado, con atributos suficientes para realizar las acciones y haber iniciado sesión en el sistema (CU001). ● El turno de caja del operador debe encontrarse en estado abierto (CU027). ● Los productos deben estar registrados y con stock disponible en el sistema.
Postcondición	Queda registrada la venta en el sistema en estado "Cobrada", se asienta su historial de estados, se descuenta el inventario de los artículos involucrados (CU026) y se acumula el monto al turno de caja activo.

Curso Normal	**Curso Alternativo**

1) El operador accede a la función "Mostrador de Ventas y Cobro" desde el "Menú Principal" o presionando la tecla rápida F1.	
2) En el campo "Buscar Producto", el operador ingresa el código de barras o descripción del artículo y define la cantidad requerida (por defecto 1).	2.1) El operador no recuerda el código del producto y presiona el botón "[F4] Explorar Catálogo" o la tecla F4. 2.1.1) El sistema despliega la ventana de catálogo general. 2.1.2) El operador busca el producto, lo selecciona y presiona "Aceptar". 2.1.3) El sistema transfiere el código del producto al mostrador y continúa en el paso 3.
3) El operador presiona la tecla Enter.	3.1) El código ingresado no existe en el catálogo. 3.1.1) El sistema muestra el mensaje de error: "Producto no encontrado". 3.1.2) Se vuelve al paso 2.
4) El sistema verifica la disponibilidad de stock del producto ingresado (CU029).	4.1) No existe stock disponible suficiente para cubrir la cantidad solicitada (CU029). 4.1.1) El sistema emite una alerta indicando la existencia física actual. 4.1.2) Se vuelve al paso 2 para ajustar la cantidad o cancelar el ingreso del ítem.
5) El sistema agrega el artículo a la grilla de detalle de venta mostrando código, marca, descripción, cantidad, precio unitario y subtotal, actualizando el "TOTAL A COBRAR".	5.1) El operador desea eliminar un artículo de la grilla. 5.1.1) Selecciona la fila del producto y presiona el botón "[Supr] Quitar Ítem Seleccionado" o la tecla Supr. 5.1.2) El sistema remueve el artículo y recalcula el "TOTAL A COBRAR". Se vuelve al paso 2. 5.2) El operador cancela la transacción en curso. 5.2.1) Presiona el botón "[Esc] Cancelar Venta Completa" o la tecla Esc. 5.2.2) El sistema asienta la venta con estado "Cancelada" en el historial de estados, limpia la grilla y vuelve al paso 2.
6) El operador selecciona el medio de pago en el selector "Medio de Pago" (Efectivo o Transferencia).	6.1) El cliente abona mediante "Transferencia". 6.1.1) El sistema deshabilita el campo "Paga con" y el cálculo de vuelto, habilitando el campo "Nro. Comprobante". 6.1.2) El operador ingresa el número de cupón o transacción bancaria. 6.1.3) Se continúa en el paso 9.
7) En el campo "Paga con: $", el operador ingresa el monto entregado por el cliente.	7.1) El importe ingresado en "Paga con: $" es menor que el "TOTAL A COBRAR".

	7.1.1) El sistema mantiene deshabilitado el botón "Confirmar" y notifica el saldo restante. 7.1.2) Se vuelve al paso 7.
8) El sistema calcula automáticamente el valor en "VUELTO" (CU030).	
9) El operador presiona el botón "Confirmar".	
10) El sistema registra la venta en estado "Cobrada", genera el registro en el historial de estados, descuenta el stock de los productos vendidos (CU026) y suma el importe al balance del turno de caja activo.	
11) El sistema limpia los campos de la interfaz dejando el foco disponible para una nueva transacción.	
12) Fin del caso de uso.	

Parámetro	**Detalle**
Frecuencia esperada	50 a 100 casos de uso por jornada comercial.
Importancia	Muy Importante

Registrar Compra de Productos

CU025	**Registrar Compra de Productos**
Versión	1.00 06/09/2026
Objetivos asociados	Registrar los comprobantes de compra de mercadería adquirida a proveedores mayoristas, detallando los artículos, cantidades y costos unitarios, para actualizar e incrementar automáticamente las existencias de inventario en el sistema.
Actor Principal	Repositor, Administrador
Actor Secundario	No Aplica.
Precondición	● El usuario debe estar registrado, con atributos suficientes para realizar las acciones y haber iniciado sesión en el sistema (CU001). ● El proveedor mayorista debe encontrarse previamente registrado en el sistema (CU015). ● Los productos a abastecer deben existir registrados en el catálogo maestro (CU018).
Postcondición	Queda registrada la compra en la base de datos, se asienta el detalle de ítems, se incrementa automáticamente el stock de los artículos (CU026) y se actualiza el costo de reposición.

Curso Normal	**Curso Alternativo**
1) El operador ingresa en el módulo "Gestión de Compras y Reposición" (CU025) (GUI-06) desde el Menú Principal (GUI-02) o presionando la tecla rápida F4.	
2) El sistema despliega la pestaña "Registrar Compra de Mercadería" (GUI-06).	
3) El operador selecciona el proveedor de la lista desplegable "Mayorista".	3.1) El mayorista no figura en la lista: 3.1.1) El operador debe registrar previamente al proveedor mayorista en el módulo correspondiente (CU015). 3.1.2) Fin del caso de uso.
4) El operador ingresa el número de comprobante fiscal en el campo "Nro. Ticket / Factura".	4.1) El número de comprobante se encuentra vacío al intentar confirmar: 4.1.1) El sistema resalta el campo con la advertencia: "Debe ingresar el número de ticket o factura del comprobante". Se vuelve al paso 4.
5) En el campo "Buscar Producto", el operador ingresa el código de barras o la descripción del	5.1) El operador no recuerda el código del producto y presiona el botón "[F4] Explorar

artículo y define la cantidad adquirida en el campo "Cant".	Catálogo" o la tecla F4: 5.1.1) El sistema abre la ventana modal de catálogo general (CU021) (GUI-04). 5.1.2) El operador busca el producto en la grilla, lo selecciona y presiona "Aceptar". 5.1.3) El sistema transfiere el código del artículo a la pantalla de compra (GUI-06) y continúa en el paso 6.
6) El operador presiona la tecla Enter.	6.1) El producto ingresado no existe en el sistema: 6.1.1) El sistema emite el mensaje: "Producto no encontrado. Debe dar de alta el producto previamente en el catálogo (CU018)". 6.1.2) Se vuelve al paso 5.
7) El sistema consulta y verifica el producto en el catálogo (CU021).	
8) El sistema agrega el artículo a la grilla de compras exhibiendo código, marca, descripción, unidades compradas, costo unitario y subtotal, actualizando el "TOTAL TICKET MAYORISTA".	8.1) El operador desea remover un ítem de la grilla: 8.1.1) Selecciona la fila del producto y presiona el botón "[Supr] Quitar Ítem Seleccionado" o la tecla Supr. 8.1.2) El sistema elimina la fila y recalcula el "TOTAL TICKET MAYORISTA". Se vuelve al paso 5. 8.2) El operador cancela la carga de la compra: 8.2.1) Presiona el botón "[Esc] Limpiar Formulario" o la tecla Esc. 8.2.2) El sistema vacía la grilla de detalle, restablece los campos y regresa al paso 3.
9) El operador presiona el botón "Confirmar".	9.1) Intento de confirmación con grilla vacía: 9.1.1) El sistema mantiene deshabilitado el botón "Confirmar" hasta que se incorpore al menos un ítem a la lista. 9.1.2) Se vuelve al paso 5.
10) El sistema almacena la transacción Compra junto con su DetalleCompra en la base de datos.	
11) El sistema actualiza e incrementa de forma automática el stock actual de cada producto registrado (CU026).	
12) El sistema emite el mensaje: "Compra de mercadería registrada con éxito", limpia los campos del formulario (GUI-06) y deja la interfaz disponible para un nuevo registro.	

13) Fin del caso de uso.	

Parámetro	Detalle
Frecuencia esperada	1 o 2 casos de uso por semana (dependiendo de la necesidad de reabastecimiento).
Importancia	Muy Importante

Interfaz Gráfica

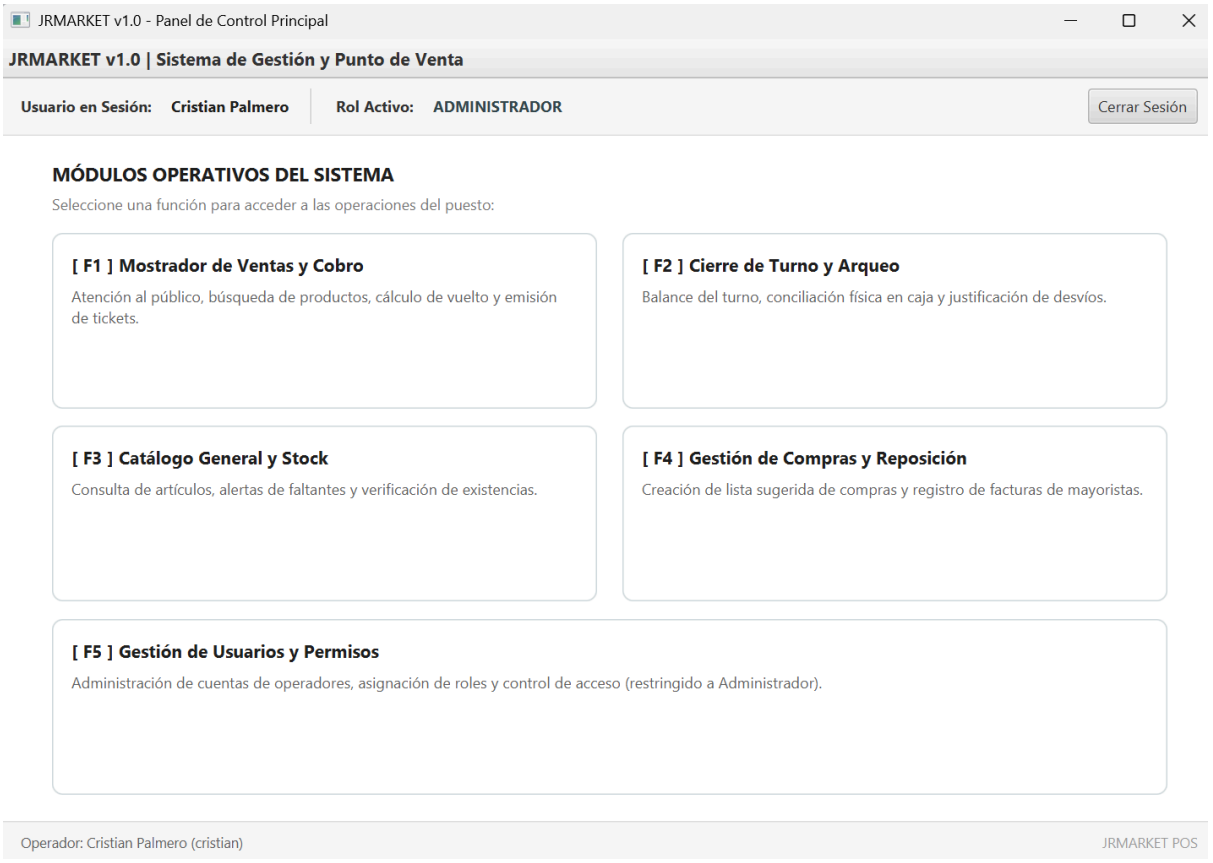
GUI-01



Fuente: Elaboración propia.

Figura 6: Pantalla de Inicio de Sesión.

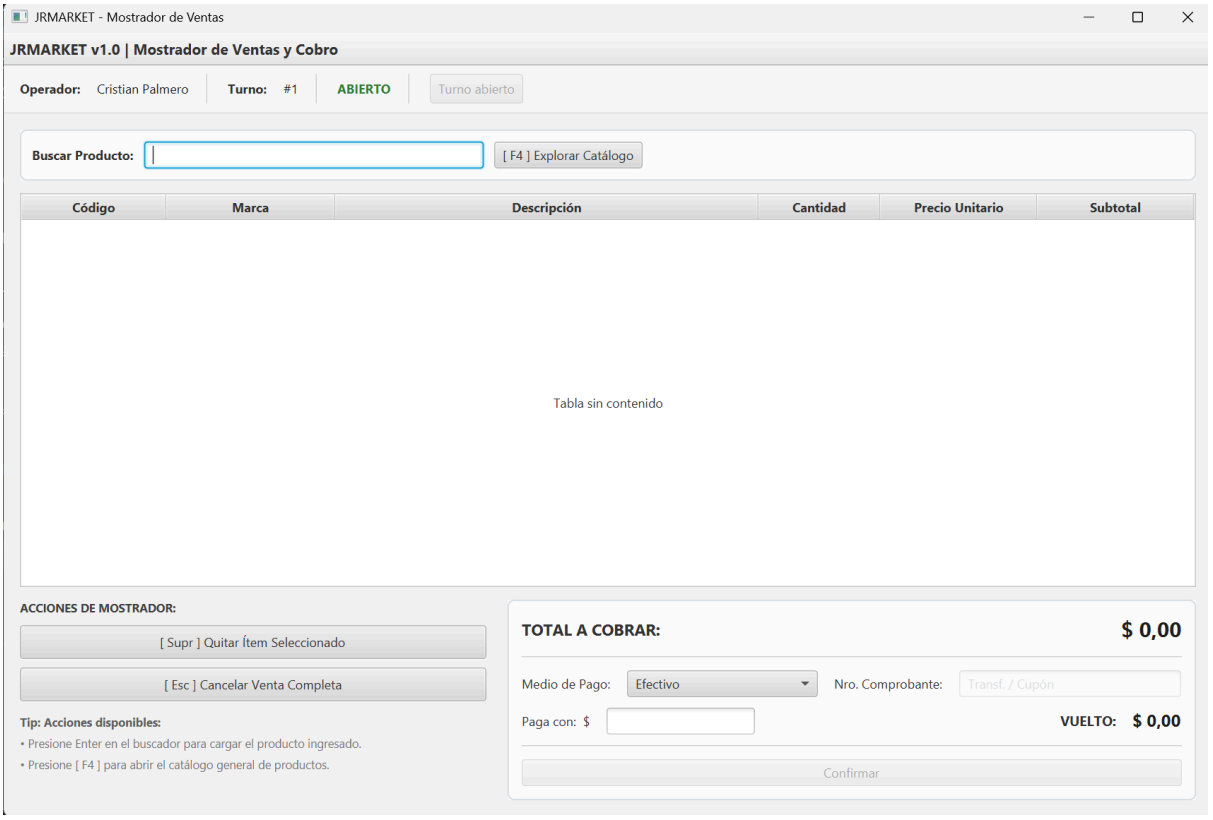
GUI-02



Fuente: Elaboración propia.

Figura 7: Pantalla de Menú Principal.

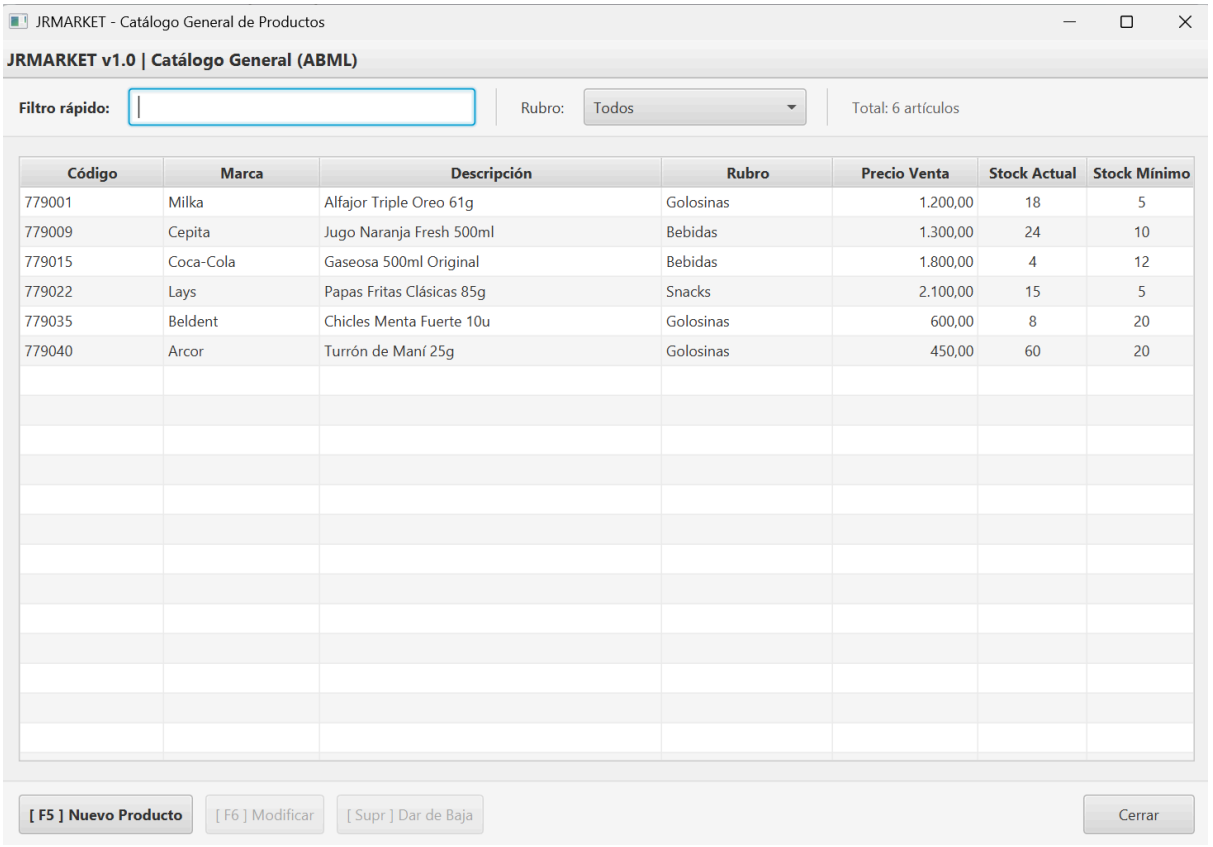
GUI-03



Fuente: Elaboración propia.

Figura 8: Pantalla de Mostrador de Ventas y Cobros.

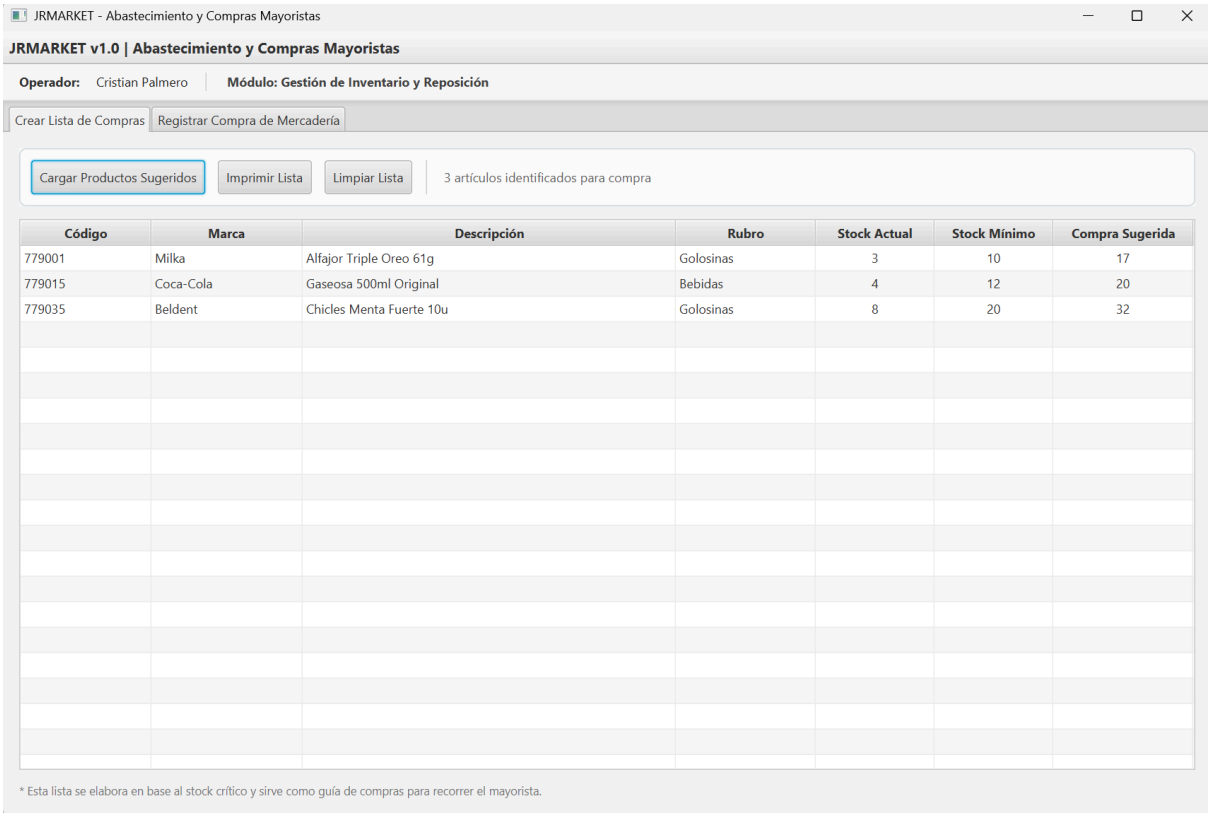
GUI-04



Fuente: Elaboración propia.

Figura 9: Pantalla de Gestión de Productos.

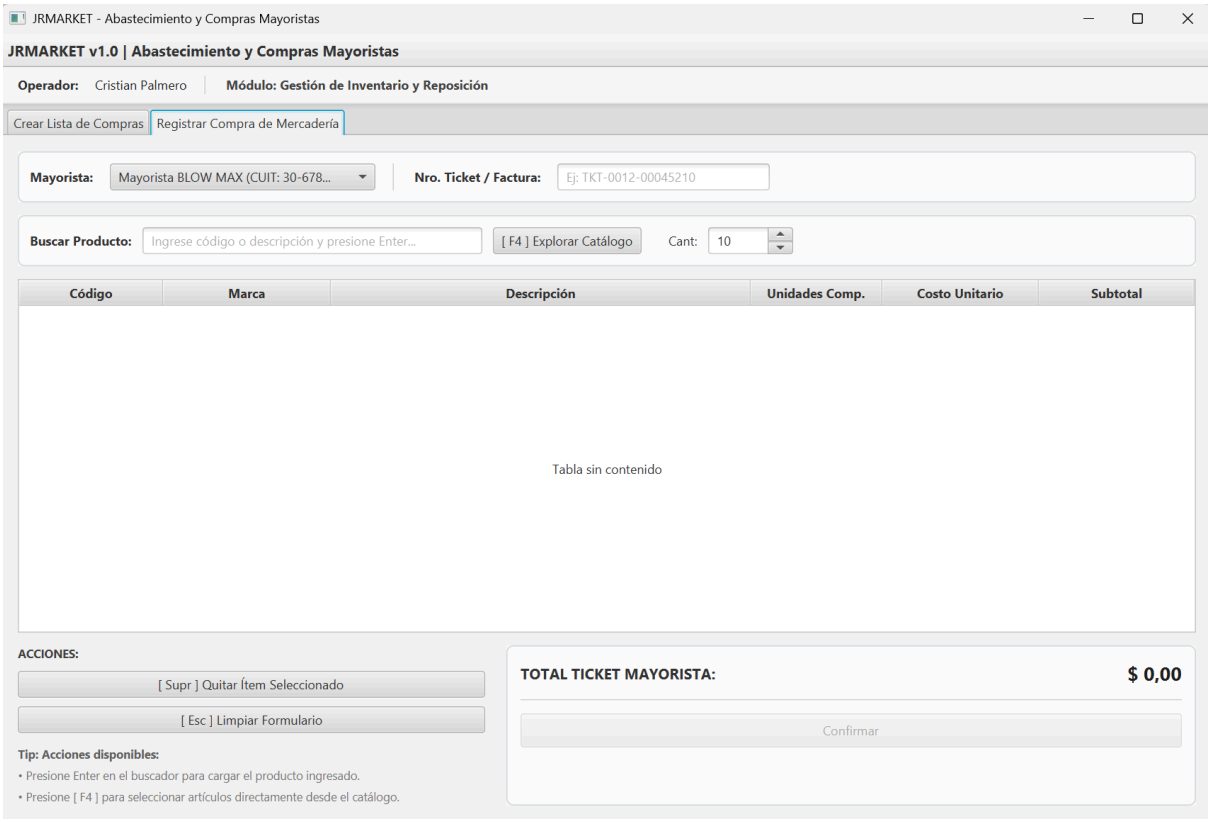
GUI-05



Fuente: Elaboración propia.

Figura 10: Pantalla de Creación de Lista de Compras.

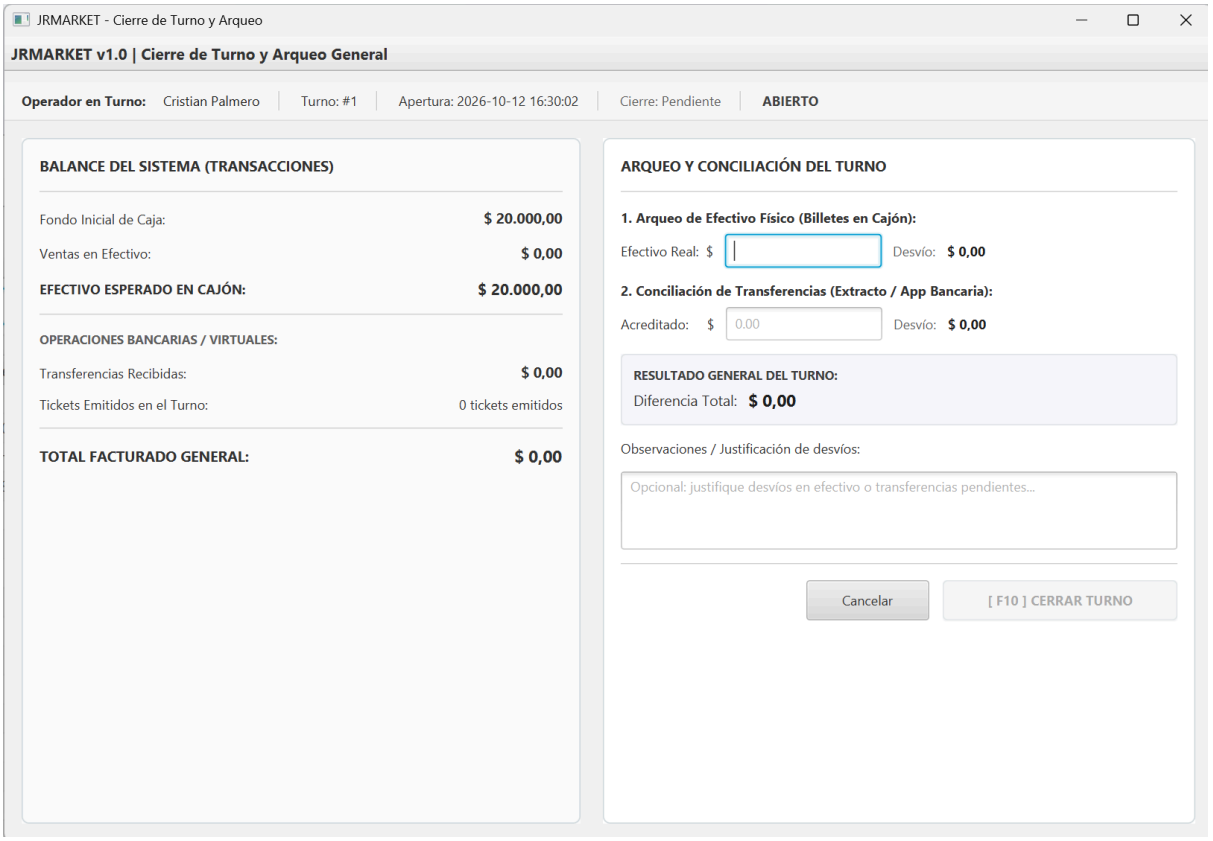
GUI-06



Fuente: Elaboración propia.

Figura 11: Pantalla de Registro de Compra de Productos.

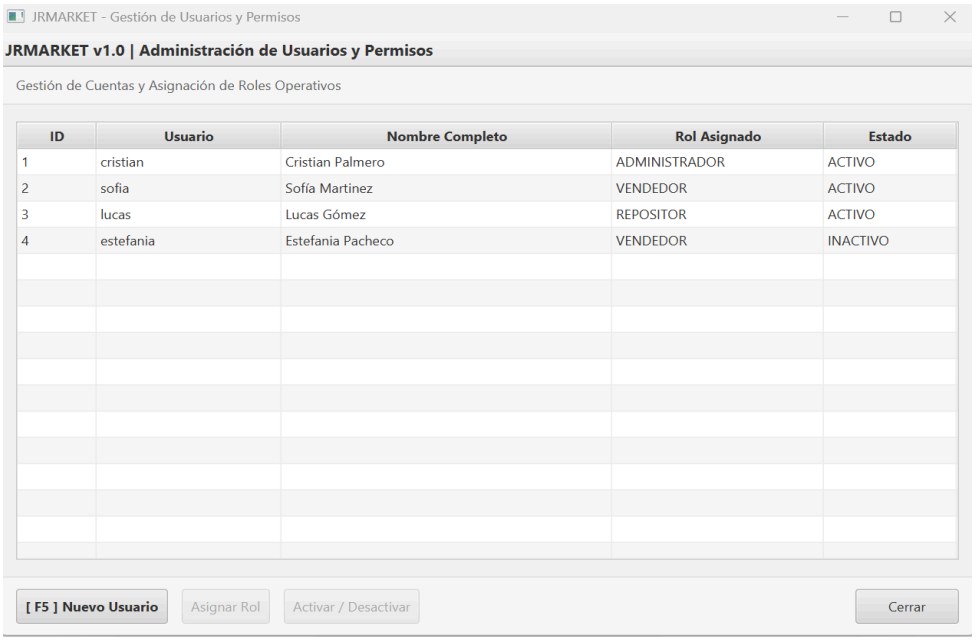
GUI-07



Fuente: Elaboración propia.

Figura 12: Pantalla de Cierre de Turno y Conciliación (Arqueo).

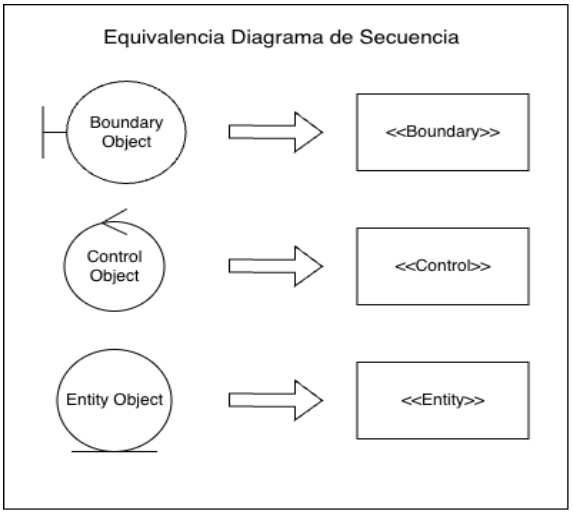
GUI-08



Fuente: Elaboración propia.

Figura 13: Pantalla de Gestión de Usuarios y Asignación de Roles.

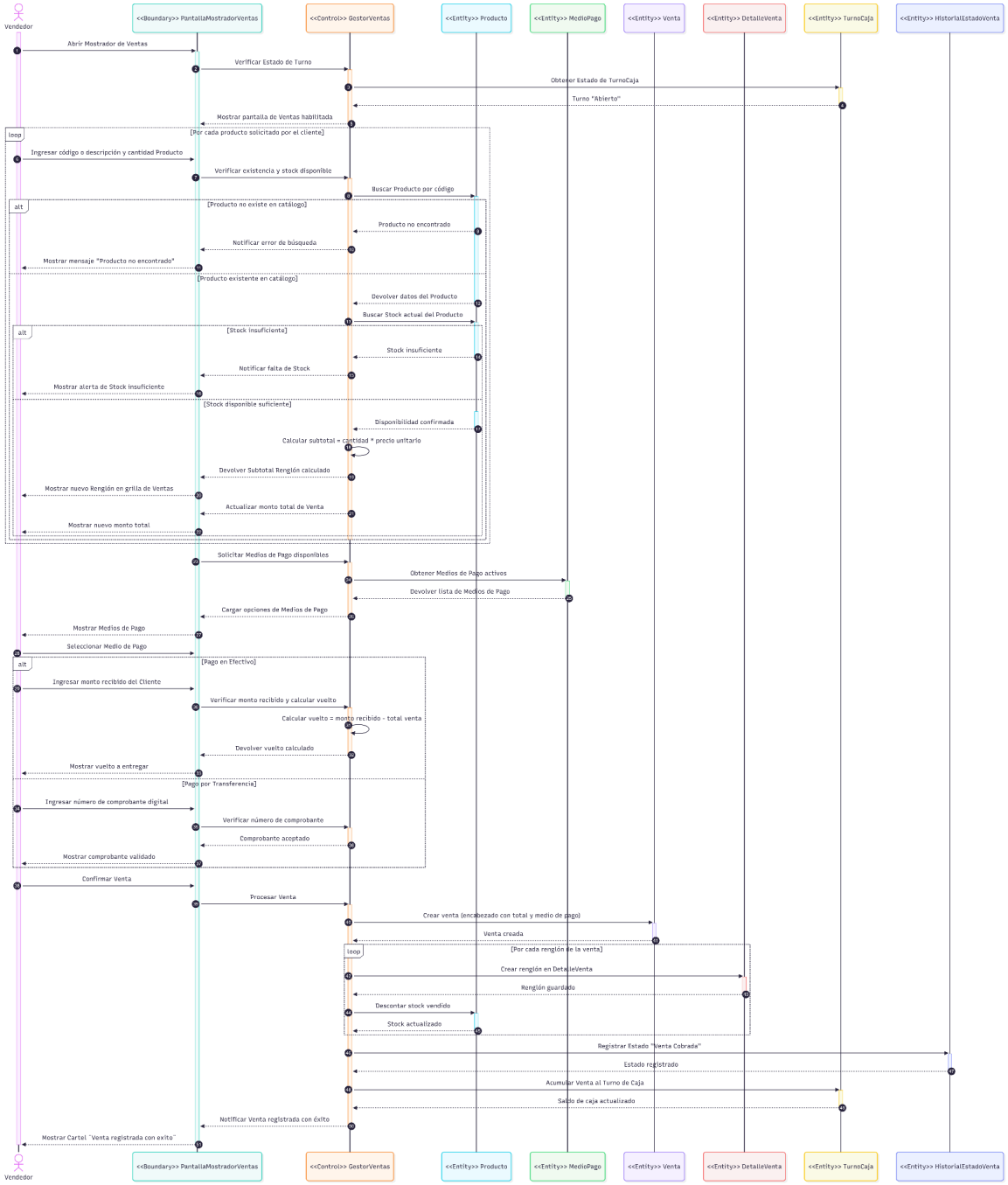
Etapa de Análisis



Fuente: Elaboración propia.

Figura 14: Equivalencia de estereotipos del análisis contra cómo fueron diagramados.

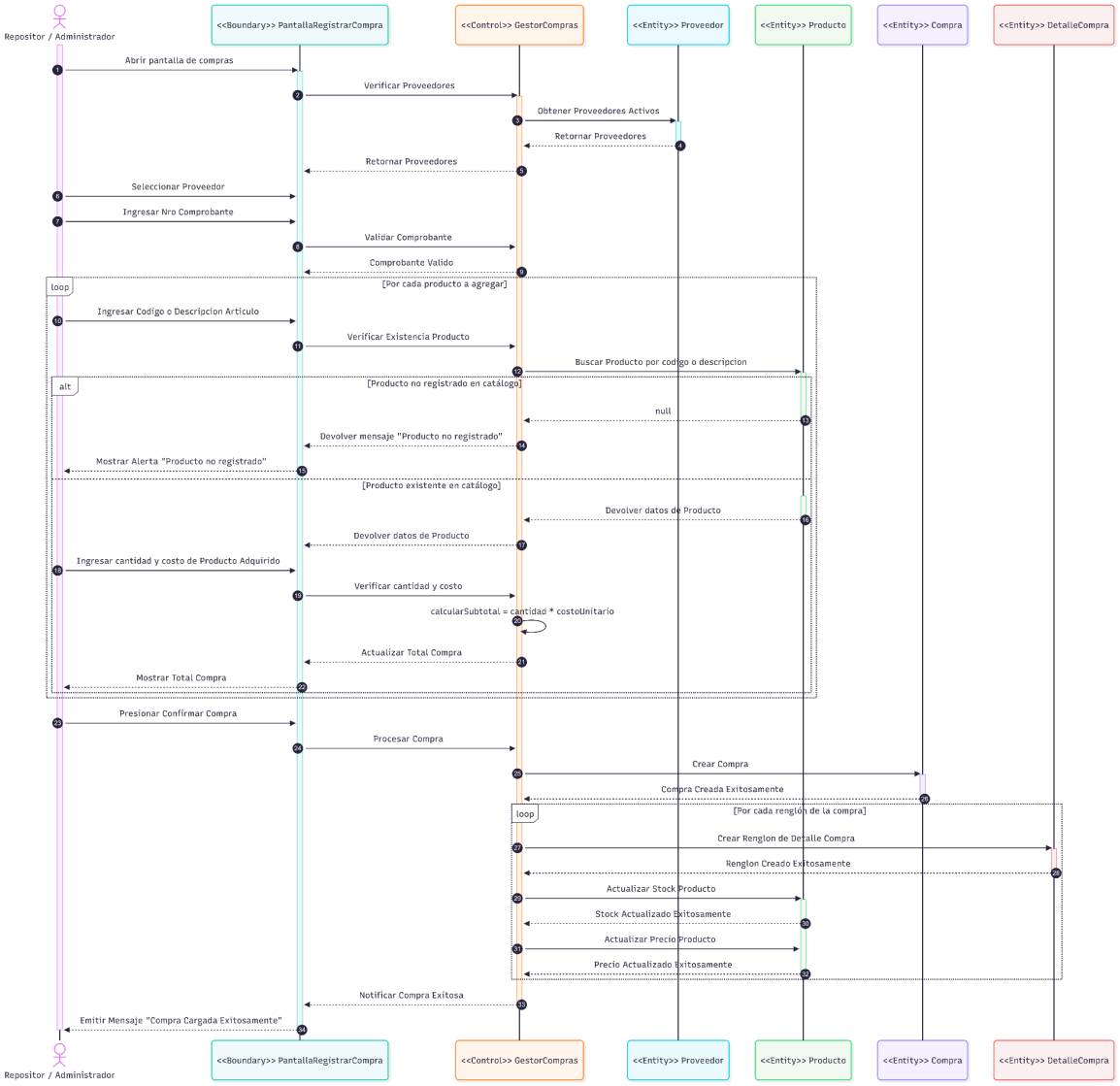
Diagrama de Secuencia del Análisis - Venta de Productos



Fuente: Elaboración propia.

Figura 15: Diagrama de Secuencia del Análisis - Venta de Productos.

Diagrama de Secuencia del Análisis - Compra de Productos

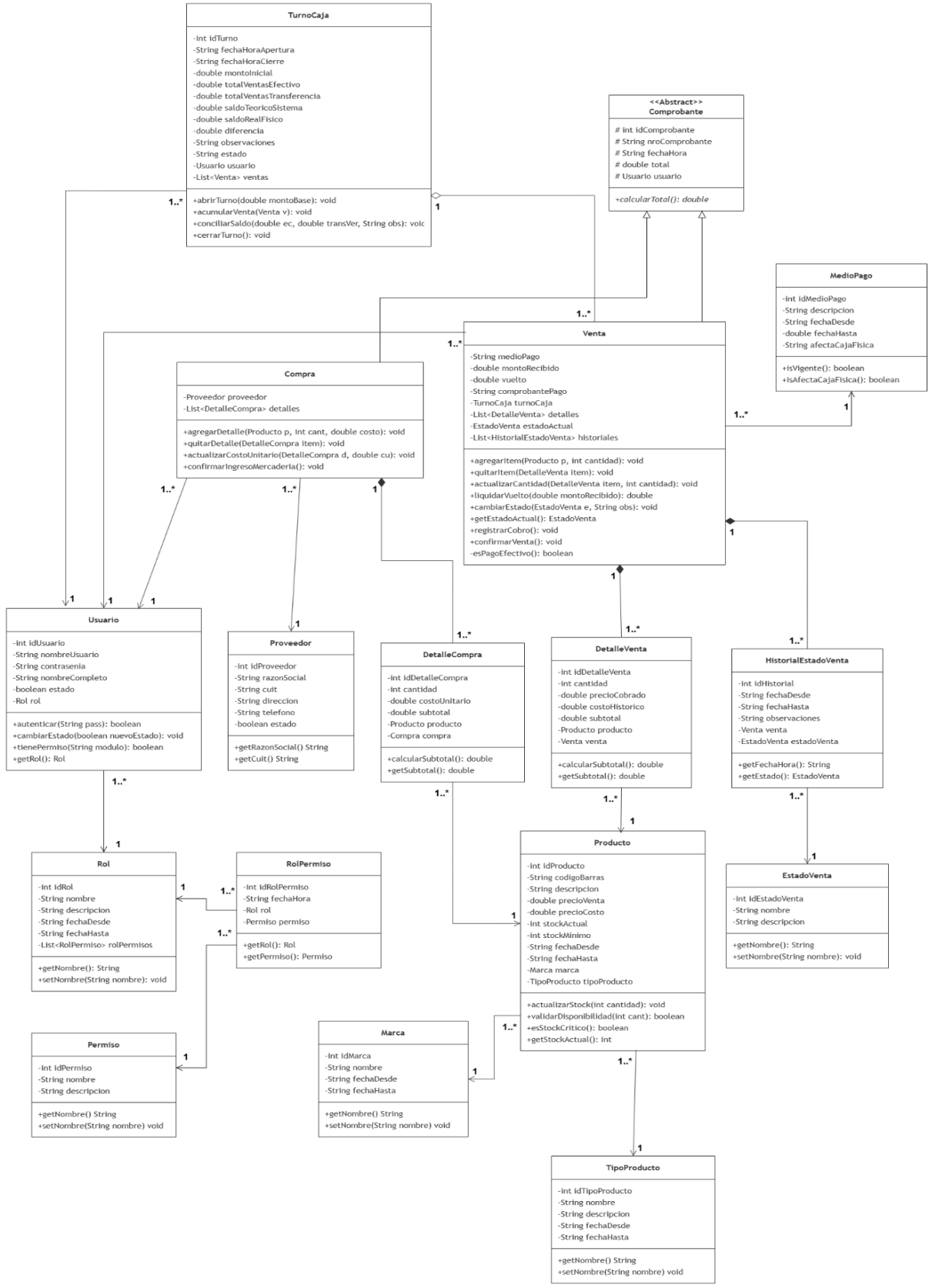


Fuente: Elaboración propia.

Figura 16: Diagrama de Secuencia del Análisis - Compra de Productos.

Etapa de Diseño

Diagrama de Clases



Fuente: Elaboración propia.
Figura 17: Diagrama de Clases general.

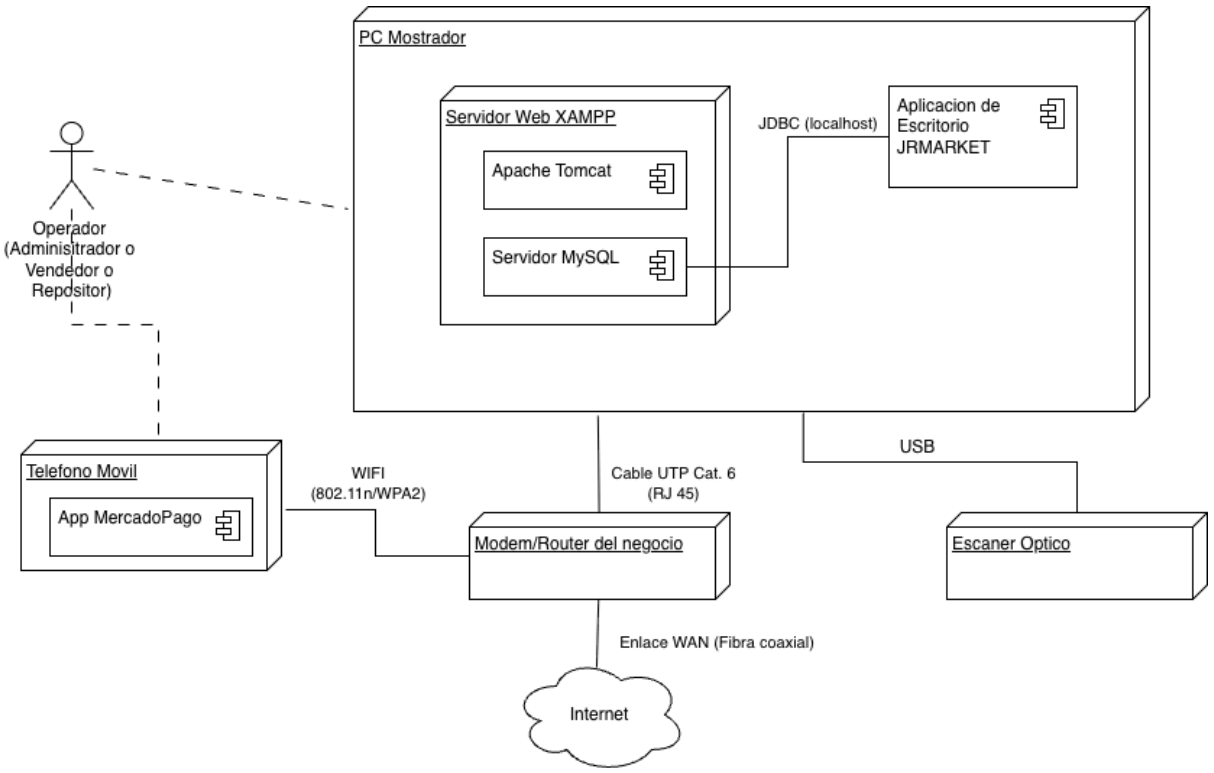
Etapa de Implementación

En esta etapa del Proceso Unificado de Desarrollo (PUD), los modelos lógicos de análisis y diseño se materializan en artefactos físicos ejecutables y configuraciones de hardware concretas. El Diagrama de Despliegue modela la arquitectura física del sistema, especificando cómo se distribuyen los componentes de software (ejecutable Java, bibliotecas y base de datos) sobre los nodos de cómputo y periféricos del establecimiento comercial, así como los protocolos mediante los cuales interactúan. A continuación, se formaliza la correspondencia entre los Requerimientos No Funcionales (RNF) aprobados para JRMARKET y las decisiones de arquitectura de software y hardware:

Tabla de Requerimientos del Sistema e Implementación Física

Requerimiento	**Clasificación**	**Descripción Técnica de Implementación**
RNF01	Plataforma	Binario .jar sobre Java Runtime Environment 17 LTS bajo Microsoft Windows 10/11 (64 bits), desplegado en el nodo Terminal de Mostrador.
RNF02	Persistencia y Entorno	Motor MySQL Server gestionado localmente sobre XAMPP en el puerto 3306 (127.0.0.1), garantizando autonomía total de la conexión a Internet.
RNF03	Interfaz y Periféricos	Interfaz de mostrador navegable por teclado USB (F1 a F6, numérico) y compatible con lector de código de barras USB bajo protocolo estándar HID.
RNF04	Rendimiento	Tiempos de respuesta inferiores a 1 segundo soportados en el controlador JDBC MySQL Connector/J con consultas parametrizadas.
RNF05	Seguridad	Almacenamiento de contraseñas con función hash criptográfica SHA-256 / BCrypt y cumplimiento de la Ley N° 25.326 de Protección de Datos Personales.
RNF06	Integridad	Transacciones atómicas ACID mediante tablas con motor InnoDB en MySQL, protegiendo existencias y caja ante cierres imprevistos.

Diagrama de Despliegue



Fuente: Elaboración propia.
Figura 18: Diagrama de Despliegue del Sistema.

Etapa de Pruebas

La presente sección formaliza el proceso de aseguramiento de calidad y verificación para el software de escritorio del comercio "JRMARKET", integrando los lineamientos de la cátedra de Seminario de Práctica de la Universidad Siglo 21 y la teoría general de testing de sistemas.

Plan de Pruebas

Objetivos del Plan

- Verificar y validar que las funcionalidades críticas del Punto de Venta (POS) y Gestión de Inventario cumplan con los requerimientos funcionales especificados en la propuesta.
- Garantizar la integridad de los datos transaccionales (descuento automático de existencias, cálculo de cobros y liquidación de arqueos) en el motor de base de datos relacional local MySQL.

- Evaluar el comportamiento del sistema ante condiciones de entrada erróneas, datos fuera de rango y flujos alternativos en la interfaz gráfica de usuario.

Alcance

- Módulos bajo prueba:
    - Módulo de Mostrador de Ventas.
    - Módulo de Gestión de Inventario y Catálogo de Productos.
    - Módulo de Compras y Abastecimiento de Productos.
    - Módulo de Cierre de Caja.
    - Módulo de Seguridad y Acceso.
- Elementos fuera de alcance:
    - Conectividad a pasarelas de pago externas o servidores de facturación electrónica externa de ARCA.
    - Automatización mediante robots de pruebas o frameworks de stress distribuido.

Entorno de Pruebas

- Hardware de prueba: Computadora de escritorio dedicada bajo arquitectura monopuesto.
- Plataforma y Software: Microsoft Windows 11 Pro (64 bits), Java Development Kit (JDK) 17 LTS, MySQL Server 8.0 desplegado localmente vía entorno XAMPP.
- Aislamiento: Base de datos de prueba independiente poblada con el catálogo inicial de 85 artículos comerciales del kiosco, evitando operar sobre datos de producción.

Matriz de Plan de Pruebas

A continuación, se define la matriz consolidada de pruebas que asocia los casos de uso principales con sus correspondientes tipos y técnicas de prueba:

Caso de Uso Asociado	**Código de Prueba**	**Nivel / Tipo de Prueba**	**Técnica Aplicada**	**Objetivo y Observación**
CU028: Registrar Venta de Productos	PR-SIS-01	Sistema / Funcional	Caja Negra: Partición de Equivalencias y Análisis de Frontera	Validar el procesamiento del cobro en efectivo y cálculo de vuelto con límites de importes.
CU029: Verificar Disponibilidad de Stock	PR-INT-01	Integración / Estructural	Caja Blanca: Cobertura de Caminos Básicos (McCabe)	Validar la lógica interna del método Java que verifica stock y define el estado transaccional.
CU025: Registrar Compra de Productos	PR-SIS-02	Sistema / Funcional	Caja Negra: Basada en Casos de Uso	Validar el incremento atómico del inventario y actualización del costo de reposición tras asentar la factura.

CU018: Registrar Producto	PR-SIS-03	Sistema / Robustez	Caja Negra: Partición de Equivalencias	Comprobar el bloqueo ante códigos de barra duplicados o precios de costo negativos.
CU001: Iniciar Sesión	PR-SIS-04	Aceptación / Seguridad	Caja Negra: Entradas inválidas y control de acceso	Verificar la autenticación con credenciales cifradas y asignación correcta de interfaz según rol operativo.

Modelo de Pruebas: Especificación y Procedimiento

Prueba de Caja Negra: Análisis de Valores Límite y Clases de Equivalencia

- Caso de Uso: CU028 - Registrar Venta de Productos (Subfunción: CU030 - Calcular Vuelto).
- Componente / Interfaz: GUI-03: Mostrador de Ventas y Cobros.
- Variable analizada: Monto abonado por el cliente (montoAbonado) en relación con el importe total liquidado en la venta (totalVenta), fijado para el escenario de prueba en $1.500,00.

Definición de Clases de Equivalencia y Valores Límite

- Clase Válida 1 (CE1): montoAbonado > totalVenta. Cobro exitoso y cálculo de vuelto mayor a cero (montoAbonado - totalVenta).
- Clase Válida 2 (CE2): montoAbonado == totalVenta. Cobro exacto exitoso, vuelto generado igual a $0,00.
- Clase Inválida 1 (CI1): 0 < montoAbonado < totalVenta. Pago insuficiente; el sistema bloquea la confirmación de la transacción e informa el saldo restante requerido.
- Clase Inválida 2 (CI2): montoAbonado <= 0. Entrada no permitida; mensaje de error y foco retenido en el campo de entrada.
- Clase Inválida 3 (CI3): Caracteres alfanuméricos o campo nulo. Validación de tipo; rechazo inmediato por formato inválido.

Casos de Prueba Derivados (PR-SIS-01)

Total de la transacción fijado en: $1.500,00

ID Caso de Prueba	Condición de Prueba	Valor de Entrada (montoAbonado)	Resultado Esperado	Clase Evaluada
CP-BB-01	Límite inferior inválido	$1.499,99	Botón "Confirmar" deshabilitado; aviso visual: "Monto insuficiente. Saldo restante: $0,01".	CI1
CP-BB-02	Límite exacto	$1.500,00	Botón "Confirmar"	CE2

	de pago		habilitado; Vuelto calculado: $0,00; transacción lista para emitir ticket.	
CP-BB-03	Límite superior inmediato	$1.500,01	Botón "Confirmar" habilitado; Vuelto calculado: $0,01.	CE1
CP-BB-04	Valor nominal superior	$2.000,00	Botón "Confirmar" habilitado; Vuelto calculado: $500,00.	CE1
CP-BB-05	Valor límite nulo / negativo	-$50,00	Cuadro de alerta: "El importe ingresado debe ser positivo"; campo restablecido a 0.	CI2

Procedimiento de Prueba Asociado

- Código de Procedimiento: PROC-SIS-01.
- Precondiciones:
    - Sesión de usuario activa con rol "Vendedor" o "Administrador".
    - Turno de caja en estado "Abierto" con fondo inicial configurado.
    - Carrito de venta cargado con productos disponibles totalizando exactamente $1.500,00.
- Pasos de Ejecución:
    - Acceder a la interfaz GUI-03: Mostrador de Ventas y Cobros (o presionar la tecla rápida F1).
    - Seleccionar "Efectivo" en el menú desplegable "Medio de Pago".
    - Ingresar en el campo de texto "Paga con: $" el valor estipulado en la columna Valor de Entrada del caso de prueba a evaluar.
    - Presionar la tecla Enter o tabular al siguiente control.
    - Inspeccionar visualmente el campo de salida "VUELTO" y el estado de habilitación del botón "[Enter] Confirmar".
    - Si el botón queda habilitado y se presiona confirmar, constatar en la base de datos MySQL (tabla: ventas) el registro del cobro y el descuento físico en tabla: productos.
    - Contrastar los valores obtenidos con el Resultado Esperado y consignar el estado (Aprobado / Fallido) en la bitácora de ejecución.

Prueba de Caja Blanca: Complejidad Ciclomática y Cobertura Lógica

- Caso de Uso: CU029 - Verificar Disponibilidad de Stock / CU028 - Procesar Venta.
- Módulo: ControladorVentas.java.
- Método bajo análisis: Lógica encargada de evaluar si una venta puede confirmarse según la existencia física de mercadería y el método de pago seleccionado:

Código a Analizar:

public int procesarVenta(int stockDisponible, int cantidadRequerida, String medioPago, double montoAbonado, double totalVenta) {

```
    // Nodo 1
    if (cantidadRequerida <= 0 || cantidadRequerida > stockDisponible) {
        // Nodo 2
        return -1; // Error: Stock insuficiente o cantidad inválida
    }
    // Nodo 3
    if ("EFECTIVO".equalsIgnoreCase(medioPago)) {
        // Nodo 4
        if (montoAbonado < totalVenta) {
            // Nodo 5
            return -2; // Error: Monto en efectivo insuficiente
        } else {
            // Nodo 6
            return 1; // Venta exitosa en efectivo con vuelto
        }
    } else {
        // Nodo 7
        return 2; // Venta exitosa vía Transferencia (Mercado Pago / Banco)
    }
    // Nodo 8 (Salida común)
}
```

Grafo de Flujo Lógico

- Nodo 1: Evaluación de la condición de existencias: cantidadRequerida <= 0 || cantidadRequerida > stockDisponible.
- Nodo 2: Retorno de código de error -1 (rama verdadera de fallo de stock).
- Nodo 3: Rama falsa de stock. Evaluación del predicado: medioPago == "EFECTIVO".
- Nodo 4: Rama verdadera de efectivo. Evaluación del predicado: montoAbonado < totalVenta.
- Nodo 5: Retorno de error -2 (monto insuficiente).
- Nodo 6: Retorno de éxito 1 (venta cobrada en efectivo).
- Nodo 7: Rama falsa de efectivo (pago por Transferencia). Retorno de éxito 2.
- Nodo 8: Fin del método.

Cálculo de la Complejidad Ciclomática de McCabe

Aplicando la métrica formal basada en predicados simples,
Complejidad Ciclomática: V(G) = P + 1

Donde los puntos de decisión con predicados simples son:

- cantidadRequerida <= 0
- cantidadRequerida > stockDisponible
- "EFECTIVO".equalsIgnoreCase(medioPago)

- montoAbonado < totalVenta

Total de condiciones simples P = 4 -> **V(G) = 4 + 1 = 5**

El conjunto básico de caminos independientes que deben ejecutarse al menos una vez para garantizar el 100% de cobertura lógica es de 5 caminos.

Definición de Caminos Independientes y Casos de Prueba (PR-INT-01)

- Camino 1 (Stock inválido/nulo):
    - Trayectoria: 1 -> 2 -> 8.
    - Entrada: stockDisponible = 10, cantidadRequerida = 0, medioPago = "EFECTIVO", montoAbonado = 1000, totalVenta = 1000.
    - Resultado esperado: Retorno -1.
- Camino 2 (Quiebre de stock):
    - Trayectoria: 1 -> 2 -> 8 (evaluando segunda condición lógica).
    - Entrada: stockDisponible = 2, cantidadRequerida = 5, medioPago = "EFECTIVO", montoAbonado = 1000, totalVenta = 1000.
    - Resultado esperado: Retorno -1.
- Camino 3 (Efectivo con importe insuficiente):Trayectoria: 1 -> 3 -> 4 -> 5 -> 8.
    - Entrada: stockDisponible = 10, cantidadRequerida = 1, medioPago = "EFECTIVO", montoAbonado = 800, totalVenta = 1000.
    - Resultado esperado: Retorno -2.
- Camino 4 (Efectivo exitoso con cálculo de vuelto):Trayectoria: 1 -> 3 -> 4 -> 6 -> 8.
    - Entrada: stockDisponible = 10, cantidadRequerida = 1, medioPago = "EFECTIVO", montoAbonado = 1500, totalVenta = 1000.
    - Resultado esperado: Retorno 1.
- Camino 5 (Pago por Transferencia exitoso):Trayectoria: 1 -> 3 -> 7 -> 8.
    - Entrada: stockDisponible = 10, cantidadRequerida = 1, medioPago = "TRANSFERENCIA", montoAbonado = 0, totalVenta = 1000.
    - Resultado esperado: Retorno 2.

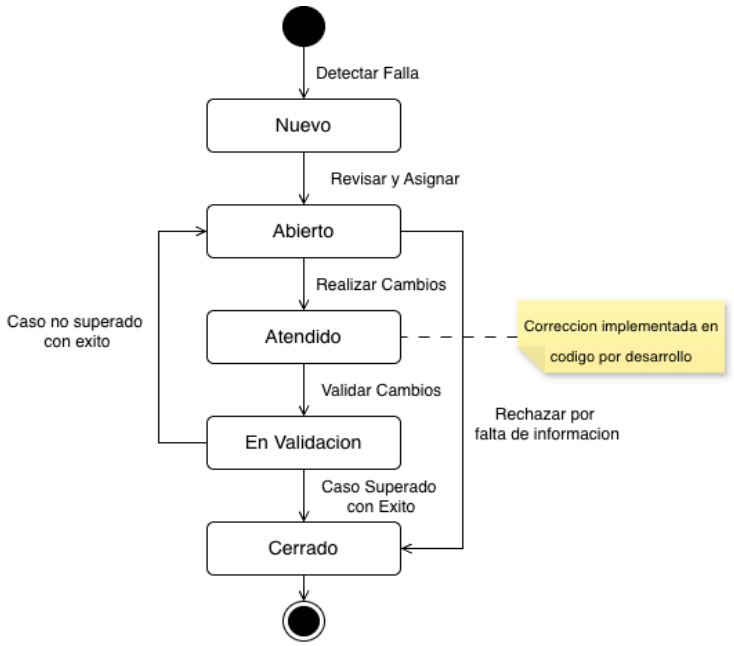
Tratamiento de Defectos (Workflow y Gestión de Incidentes)

Flujo de Estados del Defecto (Workflow)

Cuando una discrepancia entre el resultado esperado y el resultado observado es detectada durante el ciclo de pruebas, el incidente se gestiona mediante el siguiente ciclo de vida formal:

- Nuevo: Defecto registrado con sus pasos de reproducción y evidencias asociadas.
- Abierto: Incidente clasificado y priorizado para intervención directa en la capa de software afectada.
- Atendido: El código ha sido modificado y compilado, el artefacto queda a la espera del proceso de testing por parte del evaluador.
- En validación: Los cambios fueron implementados en el paso anterior y proceden a ser evaluados para asegurar su calidad y cumplimiento de objetivos.

- Cerrado: La validación confirma que la falla desapareció y que las pruebas de regresión no detectaron defectos nuevos, es decir, un caso de éxito. O un caso abierto con gran falta de información debe cerrarse.
- Reabierto: La falla persiste tras la verificación del fix, o bien, se detectaron nuevas fallas regresando el caso al estado "Abierto".



Fuente: Elaboración propia.
Figura 19: Diagrama de Estados de tratamiento de defectos.

Plantilla de Registro de Defectos

Campo	**Especificación del Registro**
ID del Defecto	Identificador alfanumérico único (ej: DEF-001).
Caso de Prueba Asociado	Código del caso de prueba que evidenció la falla (ej: CP-BB-01).
Módulo / Interfaz	Pantalla o subsistema afectado (GUI-03: Mostrador de Ventas).
Severidad	Impacto técnico: Crítica / Mayor / Moderada / Menor.
Prioridad	Urgencia de corrección: Alta / Media / Baja.
Entorno de Detección	Versión de compilación, SO y motor de BD (ej: v1.0.0, Java 17, MySQL 8.0).
Reportado por / Fecha	Nombre del responsable y fecha del hallazgo.
Descripción Breve	Resumen conciso del comportamiento anómalo detectado.
Pasos para Reproducir	Secuencia numerada exacta de acciones para disparar el fallo.

Resultado Obtenido	Comportamiento o valor erróneo devuelto por el sistema.
Resultado Esperado	Comportamiento o valor estipulado en la especificación.
Estado Actual	Estado dentro del workflow.
Resolución	Detalle técnico de la solución aplicada en el código fuente.

Campo	**Especificación del Registro**
ID del Defecto	DEF-001
Caso de Prueba Asociado	CP-BB-01
Módulo / Interfaz	GUI-03: Mostrador de Ventas y Cobros
Severidad	Mayor
Prioridad	Alta
Entorno de Detección	Windows 11, JDK 17, MySQL 8.0
Reportado por / Fecha	Cristian Palmero - 20/09/2026
Descripción Breve	Habilitación indebida del botón "Confirmar" ante montos abonados por el Cliente inferiores al total de la Venta.
Pasos para Reproducir	1. Cargar una venta por $1.500,00. 2. Seleccionar medio de pago "Efectivo". 3. Tipear $1.499,99 en el campo "Paga con: $" 4. Presionar la tecla Enter o presionar boton “Confirmar”.
Resultado Obtenido	El botón "Confirmar" permaneció activo y el sistema computó un vuelto erróneo de -$0,01, permitiendo asentar la transacción en la base de datos con saldo negativo.
Resultado Esperado	El botón "Confirmar" debe bloquearse y el sistema debe advertir: "Monto insuficiente. Saldo restante: $0,01".
Estado Actual	Cerrado (Superó Validacion)
Resolución	Se corrigió la condición lógica en el oyente de teclado de la interfaz (VentaView.java), agregando la cláusula condicional btnConfirmar.setEnabled(montoAbonado >= totalVenta).

Evaluación de las Pruebas y Reporte de Resultados

Resumen Cuantitativo de Ejecución

Durante la ejecución del ciclo de pruebas de la entrega, se aplicó la batería de pruebas diseñada sobre los casos de uso nucleares del sistema JRMARKET:

- Casos de prueba planificados: 24.
- Casos de prueba ejecutados: 24 (100% de cobertura del plan).
- Casos exitosos en primera ejecución (Passed): 21 (87,5%).
- Casos fallidos inicialmente (Failed): 3 (12,5%).
- Defectos reportados y tratados: 3 (DEF-001, DEF-002, DEF-003).
- Defectos corregidos y validados mediante retesting: 3 (100% de resolución).
- Tasa final de éxito tras pruebas de regresión: 100%.

Métricas y Distribución de Defectos

Clasificación de Defecto	**Cantidad Detectada**	**Cantidad Resuelta**	**Estado Final**
Fallas en Validación de Datos (GUI / Límites)	1 (DEF-001)	1	Cerrado
Inconsistencia de Stock en Cancelación	1 (DEF-002)	1	Cerrado
Formato de Cifrado en Credenciales	1 (DEF-003)	1	Cerrado
Total	**3**	**3**	**100% Subsanados**

Criterio de Completitud y Dictamen de Calidad

El ciclo de pruebas se da por concluido satisfactoriamente al haberse alcanzado los criterios de salida definidos en la planificación: el 100% de los casos de prueba prioritarios fueron ejecutados, no restan defectos de severidad crítica ni mayor en estado abierto, y se comprobó la estabilidad de las consultas relacionales en MySQL y de las transacciones en memoria.

El software evidencia robustez operativa y consistencia de datos, encontrándose apto para continuar hacia la etapa de codificación e integración integral en lenguaje Java.

Base de Datos

Definición de Base de Datos para el Sistema

Base de Datos MySQL

Una base de datos es definida formalmente como “un conjunto de datos estructurados y definidos a través de un proceso específico, que busca evitar la redundancia y que se almacenará en algún medio de almacenamiento masivo, como un disco” (Reinosa et al., 2012). El motor relacional MySQL nació en el ecosistema del software libre y se consolidó como el estándar más extendido de la industria debido a su portabilidad, bajo consumo de recursos de procesamiento y sencillez de configuración e instalación.

Para el sistema de punto de venta e inventario del kiosco JRMARKET, la adopción de MySQL Server desplegado localmente en la computadora de mostrador responde a la necesidad crítica de autonomía operativa (RNF02): permite persistir y consultar transacciones comerciales con tiempos de respuesta en milisegundos a través del controlador oficial JDBC (Connector/J) bajo el protocolo TCP/IP (localhost:3306), garantizando que la facturación de mostrador y el descuento de inventario funcionen ininterrumpidamente sin depender de una conexión a Internet. Asimismo, la configuración del motor de almacenamiento transaccional InnoDB brinda soporte estricto a la integridad referencial mediante claves foráneas y asegura atomicidad, consistencia, aislamiento y durabilidad (propiedades ACID) en cada operación de venta y compra registrada.

Componentes del Modelo Relacional

El modelo de datos relacional representa la realidad operativa del negocio de una única forma en el nivel lógico visible para el usuario y el desarrollador: a través de una estructura tabular bidimensional conformada por filas y columnas con valores atómicos (Reinosa et al., 2012).

Formalmente, a esta estructura matemática se la denomina relación, mientras que en el ámbito de la implementación física en el motor de base de datos se la identifica como tabla. La tabla constituye la materialización de una entidad del dominio relevado del kiosco.

idProducto	idMarca	idTipoProducto	codigoBarras	descripcion	precio	stockActual	stockMinimo	fechaDesde	fechaHasta
1	4	1	7790040134011	Alfajor Guaymallén Triple Chocolate	500.00	24	10	2026-09-28 11:40:07	*NULL*
2	2	2	7791813421153	Gaseosa Coca Cola 500ml	1200.00	3	6	2026-09-28 11:40:07	*NULL*
3	1	1	7790580123456	Caramelos Menthoplus Menta Fuerte	450.00	2	5	2026-09-28 11:40:07	*NULL*
4	3	3	7033061234567	Bolígrafo Bic Cristal Azul	350.00	15	5	2026-09-28 11:40:07	*NULL*

Fuente: Elaboración propia.
Figura 17: Ejemplo de estructura tabular SQL del objeto Producto.

Los conceptos fundamentales que rigen el modelo relacional implementado en el sistema son los siguientes:

- Relación: Representación tabular bidimensional compuesta por un conjunto de filas (tuplas) y columnas (atributos). Cada relación modela una entidad específica del kiosco (como

productos, ventas, proveedores, turnos de caja, compra) y debe cumplir con la restricción de unicidad de sus registros.

- Fila o Tupla: Es cada ocurrencia o hecho individual almacenado dentro de la relación que contiene datos de la realidad comercial. Por ejemplo, los datos correspondientes al artículo “Alfajor Guaymallén Triple Chocolate” constituyen una tupla dentro de la relación Producto. Cada tupla representa una combinación única e irrepetible de datos.
- Cuerpo: Es el conjunto completo de tuplas que contiene la relación en un momento dado. En el ejemplo ilustrativo de la “Tabla Cabecera del objeto Producto” , el cuerpo de la relación Producto está conformado por 2 tuplas.
- Columna o Atributo: Propiedad o característica formal que describe a una entidad del dominio. Los elementos que integran la estructura no pueden repetirse dentro de la misma relación. En Producto, los atributos son idProducto, idMarca, idTipoProducto, codigoBarras, descripcion, precio, stockActual, stockMinimo, fechaDesde y fechaHasta.
- Cabecera: Es el esquema formal constituido por el conjunto de nombres de los atributos que definen a la relación.

idProducto	idMarca	idTipoProducto	codigoBarras	descripcion	precio	stockActual	stockMinimo	fechaDesde	fechaHasta

Tabla Cabecera del objeto Producto
Fuente: Elaboración propia.

- Dato: Es el valor elemental e indivisible almacenado en la intersección exacta entre una tupla y un atributo. Por ejemplo, el valor 7790040134011 es el dato atómico correspondiente al código de barras del primer registro de la relación.
- Grado: Es el número total de atributos o columnas que componen la relación. Representa una propiedad estática que únicamente se altera si se modifica la estructura del diseño de la base de datos. En la relación Producto presentada, el grado es 10.
- Cardinalidad: Es el número de tuplas o filas presentes en el cuerpo de la relación en un instante determinado. Constituye una propiedad dinámica, ya que varía continuamente ante las operaciones de inserción y baja de artículos. La cardinalidad actual de la Tabla 1 es 2.
- Dominio: Es el conjunto acotado de valores válidos y homogéneos que puede adoptar un atributo específico. Por ejemplo, el atributo stockActual pertenece al dominio de los números enteros mayores o iguales a cero, mientras que precio asume valores dentro del dominio de los números decimales positivos.

Diagrama Entidad-Relación

Un diagrama entidad-relación (DER) es un modelo gráfico utilizado para especificar la estructura lógica de los datos y plasmar formalmente la manera en que los elementos del sistema se vinculan

entre sí. En el marco de la ingeniería de software, el DER funciona como el plano estructural que traduce las necesidades del negocio en tablas relacionales normalizadas.

Los componentes arquitectónicos que integran el DER son:

- Entidades: Representan objetos del mundo real o conceptos transaccionales con existencia e identidad propia que requieren ser persistidos en la base de datos. Se identifican como rectángulos rotulados con nombres en singular representativos del dominio comercial (Producto, Venta, TurnoCaja, Proveedor).
- Atributos: Son las propiedades que caracterizan y detallan a cada entidad, definiendo su estado. En el modelo relacional físico en notación de pata de gallo (Crow's Foot), los atributos se listan como filas tipadas dentro del cuerpo de la entidad.
- Relaciones y Cardinalidad: Líneas que conectan las entidades para definir las asociaciones de dependencia lógica entre ellas (uno a muchos 1..*, cero a muchos 0..*). La presencia del círculo denota opcionalidad (0), mientras que la barra vertical cruzada denota obligatoriedad (1), indicando la cantidad de instancias que pueden coexistir en la interacción.
- Claves Primarias (PK) y Claves Foráneas (FK): La clave primaria (Primary Key) es el atributo seleccionado como identificador unívoco de cada tupla dentro de la entidad. La clave foránea (Foreign Key) es un atributo dentro de una entidad dependiente cuyos valores referencian directamente a la clave primaria de otra tabla, estableciendo el vínculo de integridad referencial entre ambas.

El Lenguaje SQL

El lenguaje SQL (Structured Query Language) constituye la herramienta formal estandarizada diseñada específicamente para interactuar y administrar sistemas de bases de datos relacionales.

La característica sobresaliente del lenguaje SQL reside en su naturaleza no procedimental (declarativa): las sentencias especifican qué información o modificación se desea obtener, sin describir la secuencia algorítmica paso a paso de cómo debe llevarse a cabo la búsqueda o persistencia física (Reinosa et al., 2012). El motor relacional MySQL asume la resolución interna a través de su optimizador de consultas, el cual evalúa los índices creados, los caminos de acceso físico y las estadísticas de los metadatos almacenados en el diccionario de datos para ejecutar la instrucción en el menor tiempo posible.

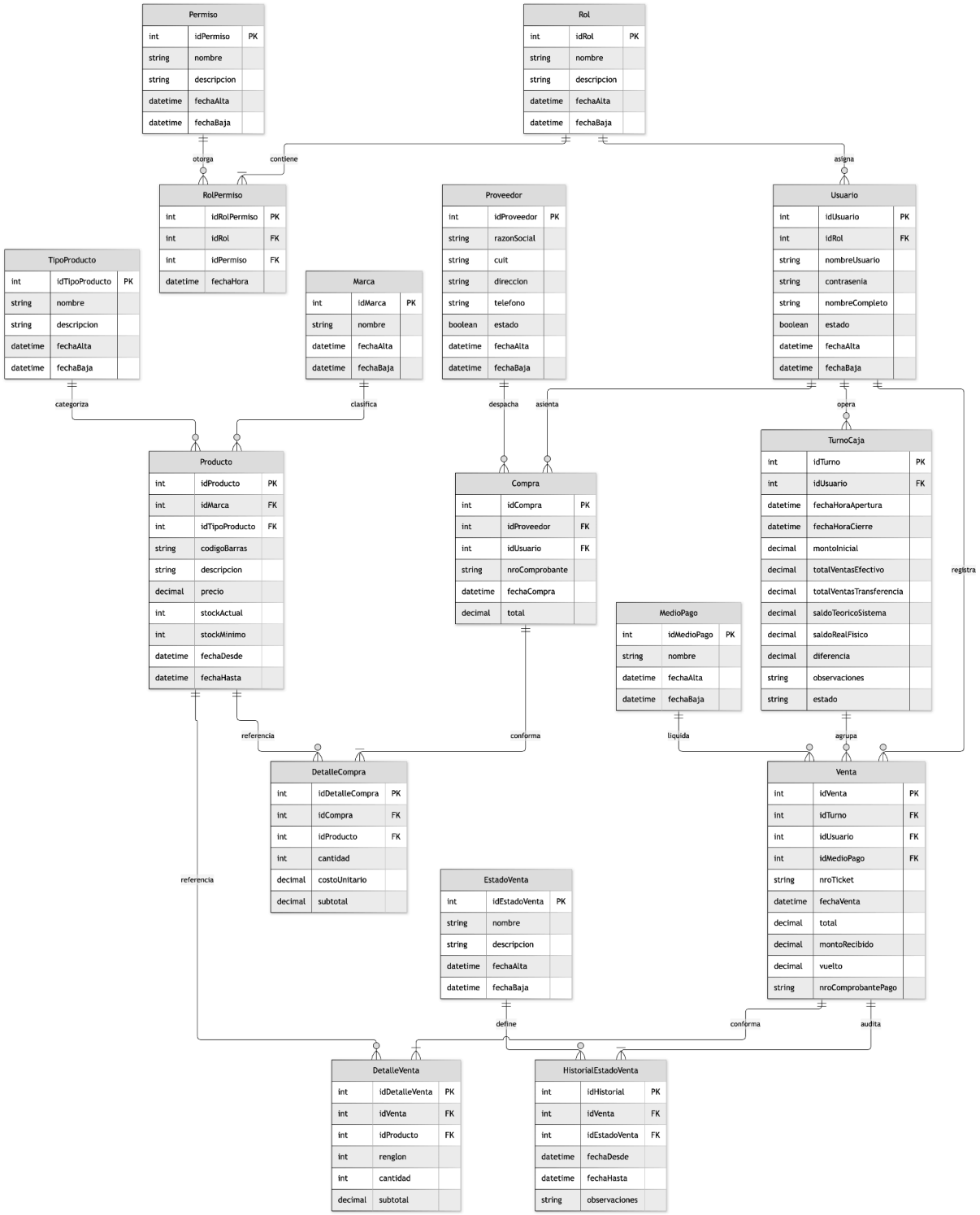
Las principales ventajas del lenguaje SQL en la solución adoptada abarcan:

- Independencia del proveedor: Es un estándar internacional mantenido por ISO/IEC y ANSI, lo que asegura que las habilidades y la lógica de acceso a datos sean transferibles entre diversos motores.
- Simplicidad y expresividad: Emplea un vocabulario acotado basado en términos directos del idioma inglés (SELECT, INSERT, UPDATE, DELETE, CREATE), facilitando la legibilidad y mantenimiento del código.
- Procesamiento de conjuntos: Trata los datos en lotes o grupos de tuplas que satisfacen determinadas condiciones relacionales en una única instrucción atómica, prescindiendo del procesamiento manual registro a registro.

Funcionalmente, el lenguaje SQL clasifica sus instrucciones en subconjuntos de trabajo esenciales:

- DDL (Data Definition Language): Sentencias como CREATE TABLE, ALTER TABLE y DROP TABLE, destinadas a crear, modificar y estructurar los esquemas físicos de las relaciones, sus tipos de datos y restricciones de integridad.
- DML (Data Manipulation Language): Sentencias como SELECT (con cláusulas WHERE, INNER JOIN, GROUP BY), INSERT INTO, UPDATE y DELETE, abocadas a consultar, ingresar, modificar y depurar los datos transaccionales del negocio.

Diagrama Entidad-Relación



Fuente: Elaboración propia.

Figura 20: Diagrama de Entidad-Relación general.

Sentencias DDL y DML

Creación de Tablas y Base de Datos

Para no saturar el cuerpo del informe con las 16 tablas del sistema, se exhiben a continuación las definiciones DDL correspondientes al núcleo transaccional de ventas y control de inventario (Producto, TurnoCaja, Venta y DetalleVenta). El conjunto completo de scripts se encuentra alojado en el repositorio del proyecto y en el entregable comprimido.

```
-- Creación de la base de datos
CREATE DATABASE IF NOT EXISTS `jrmarket` DEFAULT CHARACTER SET utf8mb4 COLLATE
utf8mb4_spanish_ci;
USE `jrmarket`;

-- Tabla: Producto (Catálogo de artículos e inventario físico)
CREATE TABLE `Producto` (
   `idProducto` INT NOT NULL AUTO_INCREMENT,
   `idMarca` INT NOT NULL,
   `idTipoProducto` INT NOT NULL,
   `codigoBarras` VARCHAR(50) NOT NULL UNIQUE,
   `descripcion` VARCHAR(150) NOT NULL,
   `precio` DECIMAL(10,2) NOT NULL,
   `stockActual` INT NOT NULL DEFAULT 0,
   `stockMinimo` INT NOT NULL DEFAULT 5,
   `fechaDesde` DATETIME NOT NULL,
   `fechaHasta` DATETIME NULL,
   PRIMARY KEY (`idProducto`),
   CONSTRAINT `fk_Producto_Marca` FOREIGN KEY (`idMarca`) REFERENCES `Marca` (`idMarca`)
ON UPDATE CASCADE ON DELETE RESTRICT,
   CONSTRAINT `fk_Producto_TipoProducto` FOREIGN KEY (`idTipoProducto`) REFERENCES
`TipoProducto` (`idTipoProducto`) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_spanish_ci;

-- Tabla: TurnoCaja (Apertura, arqueo y liquidación de jornada)
CREATE TABLE `TurnoCaja` (
   `idTurno` INT NOT NULL AUTO_INCREMENT,
   `idUsuario` INT NOT NULL,
   `fechaHoraApertura` DATETIME NOT NULL,
   `fechaHoraCierre` DATETIME NULL,
   `montoInicial` DECIMAL(10,2) NOT NULL,
   `totalVentasEfectivo` DECIMAL(12,2) NOT NULL DEFAULT 0.00,
   `totalVentasTransferencia` DECIMAL(12,2) NOT NULL DEFAULT 0.00,
   `saldoTeoricoSistema` DECIMAL(12,2) NOT NULL DEFAULT 0.00,
```

```
    `saldoRealFisico` DECIMAL(12,2) NULL,
    `diferencia` DECIMAL(10,2) NULL,
    `observaciones` VARCHAR(255) NULL,
    `estado` VARCHAR(20) NOT NULL DEFAULT 'ABIERTO',
    PRIMARY KEY (`idTurno`),
    CONSTRAINT `fk_TurnoCaja_Usuario` FOREIGN KEY (`idUsuario`) REFERENCES `Usuario`
(`idUsuario`) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_spanish_ci;

-- Tabla: Venta (Encabezado del comprobante y liquidación)
CREATE TABLE `Venta` (
    `idVenta` INT NOT NULL AUTO_INCREMENT,
    `idTurno` INT NOT NULL,
    `idUsuario` INT NOT NULL,
    `idMedioPago` INT NOT NULL,
    `nroTicket` VARCHAR(50) NOT NULL UNIQUE,
    `fechaVenta` DATETIME NOT NULL,
    `total` DECIMAL(12,2) NOT NULL,
    `montoRecibido` DECIMAL(12,2) NULL,
    `vuelto` DECIMAL(10,2) NULL,
    `nroComprobantePago` VARCHAR(60) NULL,
    PRIMARY KEY (`idVenta`),
    CONSTRAINT `fk_Venta_TurnoCaja` FOREIGN KEY (`idTurno`) REFERENCES `TurnoCaja`
(`idTurno`) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT `fk_Venta_Usuario` FOREIGN KEY (`idUsuario`) REFERENCES `Usuario` (`idUsuario`)
ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT `fk_Venta_MedioPago` FOREIGN KEY (`idMedioPago`) REFERENCES `MedioPago`
(`idMedioPago`) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_spanish_ci;

-- Tabla: DetalleVenta (Renglones de artículos facturados)
CREATE TABLE `DetalleVenta` (
    `idDetalleVenta` INT NOT NULL AUTO_INCREMENT,
    `idVenta` INT NOT NULL,
    `idProducto` INT NOT NULL,
    `renglon` INT NOT NULL,
    `cantidad` INT NOT NULL,
    `subtotal` DECIMAL(12,2) NOT NULL,
    PRIMARY KEY (`idDetalleVenta`),
    CONSTRAINT `fk_DetalleVenta_Venta` FOREIGN KEY (`idVenta`) REFERENCES `Venta` (`idVenta`)
ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT `fk_DetalleVenta_Producto` FOREIGN KEY (`idProducto`) REFERENCES `Producto`
(`idProducto`) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_spanish_ci;
```

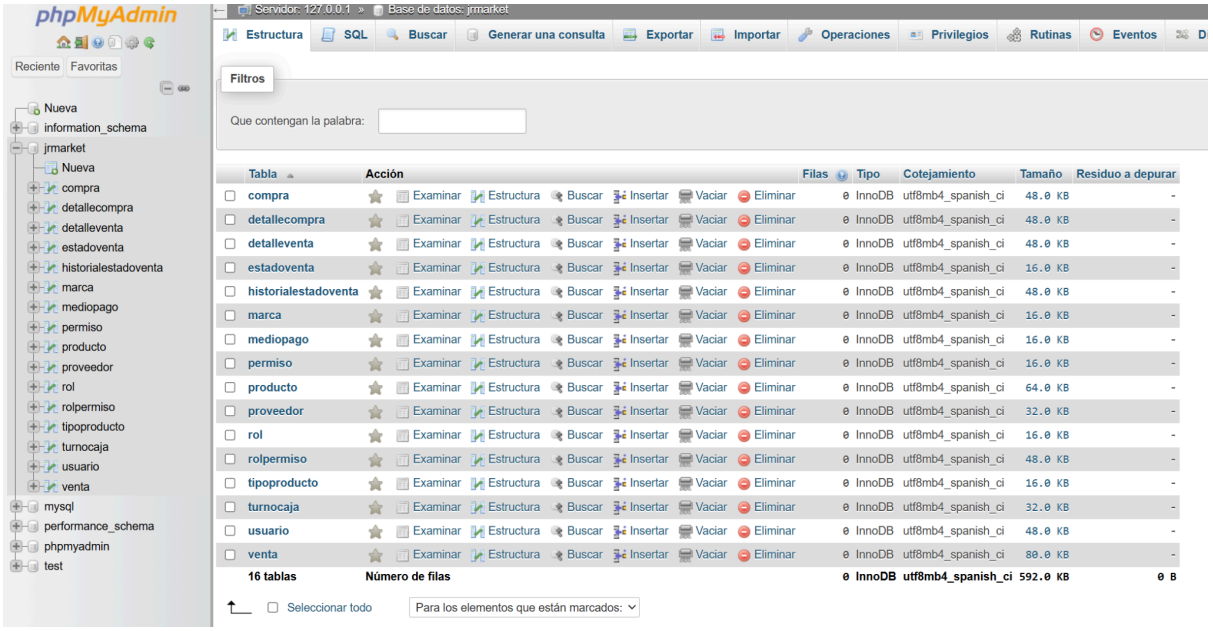


Figura 21: Vista de estructura de tablas de la base de datos jrmarket en phpMyAdmin.
Fuente: Elaboración propia.

Inserción de Datos de Prueba

```
USE `jrmarket`;

-- Inserción de Roles y Usuarios
INSERT INTO `Rol` (`idRol`, `nombre`, `descripcion`, `fechaAlta`) VALUES
(1, 'Administrador', 'Control total de inventario, caja y usuarios', NOW()),
(2, 'Vendedor', 'Atención en mostrador y registro de transacciones', NOW());

INSERT INTO `Usuario` (`idUsuario`, `idRol`, `nombreUsuario`, `contrasenia`, `nombreCompleto`,
`estado`, `fechaAlta`) VALUES
(1, 1, 'admin', '8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918',
'Administrador Kiosco', 1, NOW()),
(2, 2, 'cajero1', '04f8996da763b7a969b1028ee3007569eaf3a635486ddab211d512c85b9df8fb',
'Operador de Turno', 1, NOW());

-- Medios de Pago y Estados
INSERT INTO `MedioPago` (`idMedioPago`, `nombre`, `fechaAlta`) VALUES
(1, 'Efectivo', NOW()),
(2, 'Mercado Pago / Transferencia', NOW());

INSERT INTO `EstadoVenta` (`idEstadoVenta`, `nombre`, `descripcion`, `fechaAlta`) VALUES
(1, 'Cobrada', 'Venta finalizada y liquidada en mostrador', NOW()),
```

```
(2, 'Anulada', 'Comprobante cancelado por error o devolución', NOW());

-- Clasificación de Artículos
INSERT INTO `Marca` (`idMarca`, `nombre`, `fechaAlta`) VALUES
(1, 'Arcor', NOW()),
(2, 'Coca-Cola', NOW()),
(3, 'Bic', NOW()),
(4, 'Guaymallén', NOW());

INSERT INTO `TipoProducto` (`idTipoProducto`, `nombre`, `descripcion`, `fechaAlta`) VALUES
(1, 'Golosinas', 'Chocolates, alfajores y caramelos', NOW()),
(2, 'Bebidas', 'Gaseosas, aguas y jugos refrigerados', NOW()),
(3, 'Librería', 'Artículos escolares y de oficina', NOW());

-- Productos (Catálogo con artículos sobre stock mínimo y en stock crítico)
INSERT INTO `Producto` (`idProducto`, `idMarca`, `idTipoProducto`, `codigoBarras`, `descripcion`,
`precio`, `stockActual`, `stockMinimo`, `fechaDesde`) VALUES
(1, 4, 1, '7790040134011', 'Alfajor Guaymallén Triple Chocolate', 500.00, 24, 10, NOW()),
(2, 2, 2, '7791813421153', 'Gaseosa Coca Cola 500ml', 1200.00, 3, 6, NOW()),      -- Caso: Stock
Crítico
(3, 1, 1, '7790580123456', 'Caramelos Menthoplus Menta Fuerte', 450.00, 2, 5, NOW()),     -- Caso:
Stock Crítico
(4, 3, 3, '7033061234567', 'Bolígrafo Bic Cristal Azul', 350.00, 15, 5, NOW());

-- Apertura de Turno de Caja (Fondo fijo de $20.000)
INSERT INTO `TurnoCaja` (`idTurno`, `idUsuario`, `fechaHoraApertura`, `montoInicial`,
`saldoTeoricoSistema`, `estado`) VALUES
(1, 2, NOW(), 20000.00, 20000.00, 'ABIERTO');

-- Registro de Ventas de Prueba
-- Venta 1: En efectivo (Ticket 0001-00000001)
INSERT INTO `Venta` (`idVenta`, `idTurno`, `idUsuario`, `idMedioPago`, `nroTicket`, `fechaVenta`,
`total`, `montoRecibido`, `vuelto`) VALUES
(1, 1, 2, 1, 'TK-0001-00000001', NOW(), 1000.00, 2000.00, 1000.00);

INSERT INTO `DetalleVenta` (`idDetalleVenta`, `idVenta`, `idProducto`, `renglon`, `cantidad`,
`subtotal`) VALUES
(1, 1, 1, 1, 2, 1000.00); -- 2 Guaymallén a $500

INSERT INTO `HistorialEstadoVenta` (`idHistorial`, `idVenta`, `idEstadoVenta`, `fechaDesde`,
`observaciones`) VALUES
(1, 1, 1, NOW(), 'Venta completada en mostrador');

-- Venta 2: Por transferencia MP (Ticket 0001-00000002)
```

```
INSERT INTO `Venta` (`idVenta`, `idTurno`, `idUsuario`, `idMedioPago`, `nroTicket`, `fechaVenta`,
`total`, `nroComprobantePago`) VALUES
(2, 1, 2, 2, 'TK-0001-00000002', NOW(), 1200.00, 'MP-8923471023');

INSERT INTO `DetalleVenta` (`idDetalleVenta`, `idVenta`, `idProducto`, `renglon`, `cantidad`,
`subtotal`) VALUES
(2, 2, 2, 1, 1, 1200.00); -- 1 Coca Cola a $1200

INSERT INTO `HistorialEstadoVenta` (`idHistorial`, `idVenta`, `idEstadoVenta`, `fechaDesde`,
`observaciones`) VALUES
(2, 2, 1, NOW(), 'Pago verificado mediante billetera virtual');
```

idProducto	idMarca	idTipoProducto	codigoBarras	descripcion	precio	stockActual	stockMinimo	fechaDesde	fechaHasta
1	4	1	7790040134011	Alfajor Guaymallén Triple Chocolate	500.00	24	10	2026-09-28 11:40:07	*NULL*
2	2	2	7791813421153	Gaseosa Coca Cola 500ml	1200.00	3	6	2026-09-28 11:40:07	*NULL*
3	1	1	7790580123456	Caramelos Menthoplus Menta Fuerte	450.00	2	5	2026-09-28 11:40:07	*NULL*
4	3	3	7033061234567	Bolígrafo Bic Cristal Azul	350.00	15	5	2026-09-28 11:40:07	*NULL*

Figura 22: Registros poblados en la tabla Producto desde phpMyAdmin.
Fuente: Elaboración propia.

idVenta	idTurno	idUsuario	idMedioPago	nroTicket	fechaVenta	total	montoRecibido	vuelto	nroComprobantePago
1	1	2	1	TK-0001-00000001	2026-09-28 11:40:07	1000.00	2000.00	1000.00	*NULL*
2	1	2	2	TK-0001-00000002	2026-09-28 11:40:07	1200.00	*NULL*	*NULL*	MP-8923471023

Figura 23: Registros poblados en la tabla Venta desde phpMyAdmin.
Fuente: Elaboración propia.

idDetalleVenta	idVenta	idProducto	renglon	cantidad	subtotal
1	1	1	1	2	1000.00
2	2	2	1	1	1200.00

Figura 24: Registros poblados en la tabla DetalleVenta desde phpMyAdmin.
Fuente: Elaboración propia.

idTurno	idUsuario	fechaHoraApertura	fechaHoraCierre	montoInicial	totalVentasEfectivo	totalVentasTransferencia	saldoTeoricoSistema	saldoRealFisico	diferencia	observaciones	estado
1	2	2026-09-28 11:40:07	*NULL*	20000.00	0.00	0.00	20000.00	*NULL*	*NULL*	*NULL*	ABIERTO

Figura 25: Registros poblados en la tabla TurnoCaja desde phpMyAdmin.
Fuente: Elaboración propia.

Consultas de Objetos de Base de Datos del Negocio

Consulta 1: Resumen de ventas por turno y medio de pago (Conciliación de Caja)
Permite al administrador auditar la recaudación discriminando entre dinero en efectivo ingresado a caja y transacciones electrónicas por transferencia:

```
SELECT
    t.idTurno AS 'Turno N°',
    u.nombreCompleto AS 'Cajero',
    mp.nombre AS 'Medio de Pago',
    COUNT(v.idVenta) AS 'Cant. Transacciones',
    SUM(v.total) AS 'Total Recaudado'
FROM Venta v
INNER JOIN TurnoCaja t ON v.idTurno = t.idTurno
INNER JOIN Usuario u ON v.idUsuario = u.idUsuario
INNER JOIN MedioPago mp ON v.idMedioPago = mp.idMedioPago
WHERE t.idTurno = 1
GROUP BY t.idTurno, u.nombreCompleto, mp.nombre;
```

Turno N°	Cajero	Medio de Pago	Cant. Transacciones	Total Recaudado
1	Operador de Turno	Efectivo	1	1000.00
1	Operador de Turno	Mercado Pago / Transferencia	1	1200.00

Figura 26: Resultado de consulta de ventas por turno y medio de pago desde phpMyAdmin.
Fuente: Elaboración propia.

Consulta 2: Reporte de stock crítico para emisión de orden de compra.
Filtra los artículos cuya existencia física se encuentra en o por debajo del umbral mínimo configurado, uniendo tablas de clasificación para el repositor:

```
SELECT
    p.codigoBarras AS 'Código',
    p.descripcion AS 'Artículo',
    m.nombre AS 'Marca',
    tp.nombre AS 'Categoría',
    p.stockActual AS 'Stock Actual',
    p.stockMinimo AS 'Stock Mínimo',
    (p.stockMinimo - p.stockActual) AS 'Unidades Faltantes'
FROM Producto p
INNER JOIN Marca m ON p.idMarca = m.idMarca
INNER JOIN TipoProducto tp ON p.idTipoProducto = tp.idTipoProducto
WHERE p.stockActual <= p.stockMinimo
ORDER BY (p.stockMinimo - p.stockActual) DESC;
```

Código	Artículo	Marca	Categoría	Stock Actual	Stock Mínimo	Unidades Faltantes
7791813421153	Gaseosa Coca Cola 500ml	Coca-Cola	Bebidas	3	6	3
7790580123456	Caramelos Menthoplus Menta Fuerte	Arcor	Golosinas	2	5	3

Figura 27: Resultado de consulta de reporte de stock crítico para emisión de lista de compras desde phpMyAdmin.
Fuente: Elaboración propia.

Consulta 3: Consulta analítica de ticket fiscal y renglones facturados

Recupera el encabezado y el detalle completo de un comprobante emitido, mostrando cálculo de cantidades y subtotales por artículo:

```
SELECT
   v.nroTicket AS 'Ticket N°',
   v.fechaVenta AS 'Fecha / Hora',
   u.nombreUsuario AS 'Vendedor',
   mp.nombre AS 'Forma de Pago',
   dv.renglon AS 'Renglón',
   p.descripcion AS 'Producto',
   dv.cantidad AS 'Cant.',
   p.precio AS 'Precio Unit.',
   dv.subtotal AS 'Subtotal',
   v.total AS 'Total Comprobante'
FROM Venta v
INNER JOIN Usuario u ON v.idUsuario = u.idUsuario
INNER JOIN MedioPago mp ON v.idMedioPago = mp.idMedioPago
INNER JOIN DetalleVenta dv ON v.idVenta = dv.idVenta
INNER JOIN Producto p ON dv.idProducto = p.idProducto
WHERE v.nroTicket = 'TK-0001-00000001'
ORDER BY dv.renglon ASC;
```

Ticket N°	Fecha / Hora	Vendedor	Forma de Pago	Renglón	Producto	Cant.	Precio Unit.	Subtotal	Total Comprobante
TK-0001-00000001	2026-09-28 11:40:07	cajero1	Efectivo	1	Alfajor Guaymallén Triple Chocolate	2	500.00	1000.00	1000.00

Figura 28: Resultado de ticket fiscal y renglones facturados desde phpMyAdmin.
Fuente: Elaboración propia.

Modificación y borrado de registros

Operación UPDATE: Descuento de stock por venta concretada

Representa la postcondición transaccional del caso de uso Registrar Venta, donde el sistema disminuye el inventario luego de confirmada la Venta:

```
UPDATE Producto
SET stockActual = stockActual - 2
WHERE idProducto = 1;
```

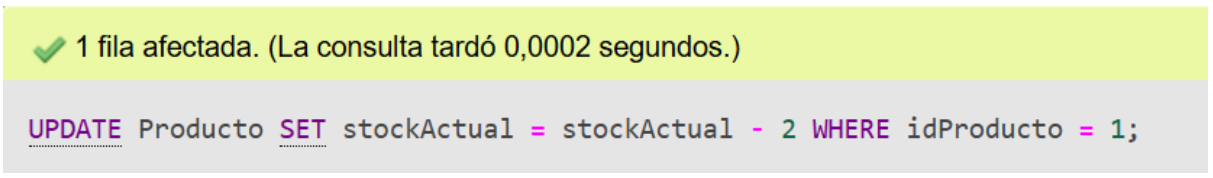


Figura 29: Resultado de ejecución de operación Update desde phpMyAdmin.
Fuente: Elaboración propia.

Operación DELETE y Justificación de Baja Lógica:
Para garantizar la integridad referencial y mantener trazabilidad histórica de los comprobantes emitidos, el sistema implementa borrado lógico en las entidades de catálogo mediante el campo fechaHasta / estado = FALSE. De este modo, un artículo discontinuado no se elimina físicamente si tiene registros asociados en DetalleVenta o DetalleCompra (lo cual violaría la restricción ON DELETE RESTRICT).

Sin embargo, para propósitos de mantenimiento o depuración de registros transitorios no consolidados, se demuestra la sintaxis de eliminación física condicionada:

```
-- Baja Lógica formal del catálogo:
UPDATE Producto
SET fechaHasta = NOW()
WHERE idProducto = 4;

-- Eliminación física de un renglón transitorio (aislado):
DELETE FROM DetalleVenta
WHERE idDetalleVenta = 999;
```

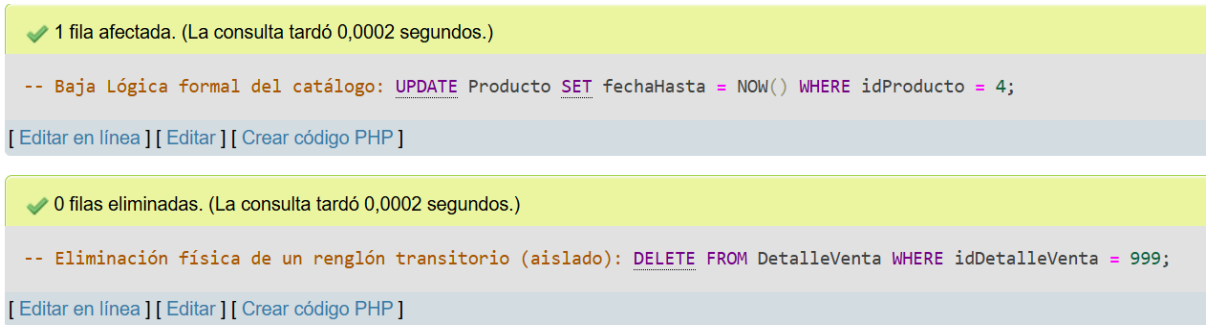


Figura 30: Resultado de ejecución de operación Update y Delete desde phpMyAdmin.
Fuente: Elaboración propia.

Etapa de Construcción de Código Java

En esta etapa del Proceso Unificado de Desarrollo (PUD), las definiciones de los casos de uso, los diagramas de interacción y el diagrama de clases se materializan en un prototipo operacional desarrollado en lenguaje Java. Según Kendall y Kendall (2011), un prototipo de software consiste en “la creación de un modelo operacional que incluya solo algunas características del sistema final”, permitiendo validar la arquitectura y el comportamiento dinámico del negocio antes del despliegue definitivo.

El prototipo desarrollado para el comercio "JRMARKET" automatiza el núcleo transaccional del establecimiento: el registro de ventas en mostrador, el ingreso de compras de mercadería mayorista, la actualización automática del inventario de stock, la administración de los turnos de caja y la

gestión de usuarios y permisos, cumpliendo con los estándares de diseño de la Programación Orientada a Objetos (POO).

Aplicación de los Pilares de la Programación Orientada a Objetos

En conformidad con lo expuesto en la teoría del módulo y las definiciones de Booch (1996), Pressman (2010) y Deitel y Deitel (2016), el código fuente implementa los cuatro pilares fundamentales del paradigma:

1. Abstracción: Superclase Comprobante

La abstracción permite capturar las características y operaciones esenciales de una entidad del dominio, eliminando los detalles accesorios que causan dispersión (Shaw & Garlan, 2015; Booch, 1996). Una clase es abstracta cuando se declara con la palabra clave abstract, no puede ser instanciada directamente mediante el operador new y cuenta con al menos un método abstracto que define un contrato operacional para sus clases derivadas.

En el sistema, cualquier movimiento comercial del local representa un comprobante contable. Por ello, se definió la superclase abstracta **Comprobante**, la cual reúne los atributos comunes compartidos por cualquier operación de Venta o Compra (identificador numérico, número de comprobante, marca temporal, total monetario y el usuario operador interviniente) y declara el método abstracto contractual calcularTotal().

```
package com.jrmarket.model;

import java.io.Serializable;

/** Datos compartidos por comprobantes de venta y de compra. */
public abstract class Comprobante implements Serializable {
   protected int idComprobante;
   protected String nroComprobante;
   protected String fechaHora;
   protected double total;
   protected Usuario usuario;

   public Comprobante() {}

   public Comprobante(int idComprobante, String nroComprobante, String fechaHora, Usuario
usuario) {
      this.idComprobante = idComprobante;
      this.nroComprobante = nroComprobante;
      this.fechaHora = fechaHora;
      this.usuario = usuario;
      this.total = 0.0;
```

```
    }

    /** Cada tipo de comprobante define cómo sumar los importes de su detalle. */
    public abstract double calcularTotal();

    public double getTotal() { return total; }
    public void setTotal(double total) { this.total = total; }
    // Getters and Setters complementarios...
}
```

2. Herencia y Reutilización: Clases Compra y Venta

La herencia es la capacidad de crear nuevas clases que adquieren automáticamente los atributos y métodos de clases padres, facilitando la reutilización de código y estructurando jerarquías formales mediante la palabra clave extends (Deitel & Deitel, 2016).

En la arquitectura de JRMARKET, tanto la venta de mostrador al cliente (Venta) como el abastecimiento de mercadería (Compra) extienden de Comprobante. En sus constructores se incluye la sentencia **super()** en la primera línea de ejecución, asegurando que la inicialización de los atributos heredados de la superclase se ejecute correctamente en cadena antes de configurar los atributos particulares:

```
// Subclase especializada de compra mayorista
public class Compra extends Comprobante {
    private Proveedor proveedor;
    private List<DetalleCompra> detalles;

    public Compra(int idCompra, String nroComprobante, String fechaHora, Usuario usuario,
Proveedor proveedor) {
        super(idCompra, nroComprobante, fechaHora, usuario); // Invocación a Comprobante
        this.proveedor = proveedor;
        this.detalles = new ArrayList<>();
    }
}

// Subclase especializada de venta de mostrador
public class Venta extends Comprobante {
    private String medioPago;
    private double montoRecibido;
    private double vuelto;
    private String comprobantePago;
    private TurnoCaja turnoCaja;
    private List<DetalleVenta> detalles;
    private EstadoVenta estadoActual;
```

```
    private List<HistorialEstadoVenta> historiales;

    public Venta(int idVenta, String nroTicket, String fechaHora, Usuario usuario,
            String medioPago, TurnoCaja turnoCaja) {
        super(idVenta, nroTicket, fechaHora, usuario); // Invocación a Comprobante
        this.medioPago = medioPago;
        this.turnoCaja = turnoCaja;
        this.detalles = new ArrayList<>();
        this.historiales = new ArrayList<>();
    }
}
```

3. Polimorfismo Dinámico y Sobrescritura de Métodos

El polimorfismo es la propiedad que permite a diferentes clases responder a un mismo mensaje de forma diferenciada a partir de los comportamientos definidos en cada una de ellas (Deitel & Deitel, 2016).

En el sistema, el polimorfismo se concreta mediante la sobreescritura del método (@Override) calcularTotal() en las clases derivadas, adaptando el algoritmo de cálculo a la estructura interna de cada comprobante:

- **En Compra:** Itera sobre la colección List<DetalleCompra>, acumulando el producto de la cantidad adquirida por el costo unitario de compra cobrado por el mayorista.
- **En Venta:** Itera sobre la colección List<DetalleVenta>, totalizando los subtotales calculados a partir del precio unitario de venta al público.

```
// Polimorfismo en Compra.java
@Override
public double calcularTotal() {
    double acumulado = 0.0;
    for (DetalleCompra dc : this.detalles) {
        acumulado += dc.getSubtotal();
    }
    this.total = acumulado;
    return this.total;
}

// Polimorfismo en Venta.java
@Override
public double calcularTotal() {
    double acumulador = 0.0;
    for (DetalleVenta d : this.detalles) {
        acumulador += d.getSubtotal();
    }
```

```
    this.total = acumulador;
    return this.total;
}
```

4. Encapsulamiento, Ocultamiento e Invariantes de Negocio

El encapsulamiento consiste en ocultar la representación interna de un objeto y restringir el acceso directo a sus datos, exponiendo únicamente métodos públicos como interfaz de comunicación (Pressman, 2010; Meza González, 2021).

En **Comprobante** los atributos compartidos se protegen con visibilidad **protected** para permitir su acceso directo en las subclases. En las restantes entidades del modelo (Producto, TurnoCaja, Usuario, etc.), los atributos son estrictamente **private** y se operan mediante **métodos de acceso** (getters) y **mutadores** (setters) que hacen cumplir las reglas del negocio.

Asimismo, los métodos de modificación transaccional utilizan aritmética segura con **Math.addExact()** para impedir anomalías por desbordamiento numérico entero, protegiendo el inventario ante intentos de asignar cantidades negativas:

```
public class Producto implements Serializable {
    private int stockActual;
    private int stockMinimo;
    private boolean estado;

    public void actualizarStock(int cantidad) {
        int nuevoStock = Math.addExact(this.stockActual, cantidad);
        if (nuevoStock < 0) {
            throw new IllegalArgumentException("El stock no puede ser negativo.");
        }
        this.stockActual = nuevoStock;
    }

    public boolean validarDisponibilidad(int cant) {
        return this.estado && this.stockActual >= cant && cant > 0;
    }

    public boolean esStockCritico() {
        return this.estado && this.stockActual <= this.stockMinimo;
    }
}
```

Métodos Constructores e Inicialización de Estados

Para garantizar que las instancias de las entidades nazcan en memoria en un estado plenamente consistente y válido, se definieron constructores parametrizados explícitos. Por ejemplo, en DetalleVenta y DetalleCompra, el constructor exige el producto asociado, las cantidades y los importes, invocando internamente a calcularSubtotal() para garantizar la integridad contable del renglón desde su creación:

```
public class DetalleVenta implements Serializable {
    private int cantidad;
    private double precioCobrado;
    private double costoHistorico;
    private double subtotal;
    private Producto producto;

    public DetalleVenta(int idDetalleVenta, int cantidad, double precioCobrado,
                        double costoHistorico, Producto producto) {
        this.idDetalleVenta = idDetalleVenta;
        this.cantidad = cantidad;
        this.precioCobrado = precioCobrado;
        this.costoHistorico = costoHistorico;
        this.producto = producto;
        this.subtotal = calcularSubtotal();
    }
}
```

Estructuras de Control, Colecciones Dinámicas y Algoritmos de Búsqueda

El procesamiento transaccional del sistema combina eficientemente las principales estructuras de Java:

- **Colecciones Tipadas (List, ArrayList, Map, HashMap):** Utilizadas para conformar el carrito dinámico de ventas **(List<DetalleVenta>)**, los bultos ingresados por compra **(List<DetalleCompra>)** y consolidar existencias por producto mediante mapas asociativos **(Map<Producto, Integer>)** en **confirmarVenta()** y **confirmarIngresoMercaderia()**.
- **Estructuras de Control e Iteración (for-each, if-else):** Empleadas para liquidar subtotales, verificar turnos abiertos y bifurcar el tratamiento de cobro según el medio de pago seleccionado (Efectivo con cálculo de vuelto o Transferencia electrónica con cupón bancario).
- **Algoritmos de Búsqueda y Filtrado en Memoria:**
    - En **VentasController**: Se ejecuta una **búsqueda secuencial** directa por código de barras sobre la lista de productos compartida (catalogoCompartido) para agilizar la carga en mostrador.

- En **ProductoController**: Se implementa filtrado en tiempo real con **FilteredList<Producto>** combinando predicados sobre la descripción, marca y rubro comercial sin recargar la base de datos.
- En **ComprasController**: El método **generarListaSugeridos()** evalúa el catálogo filtrando los artículos con p.esStockCritico() y calculando automáticamente la cantidad sugerida de reposición (stockMinimo * 2 - stockActual).

Gestión de Errores

El sistema incorpora una estrategia de tolerancia a fallos mediante el lanzamiento controlado y la captura de excepciones con bloques try-catch, asegurando que condiciones comerciales anómalas o entradas erróneas no interrumpan la ejecución de la aplicación:

- **Excepciones de Negocio:** Se lanzan **IllegalArgumentException** e **IllegalStateException** cuando se intentan confirmar operaciones con caja cerrada, comprobantes sin ítems, montos en efectivo inferiores al total o compras con costos negativos.
- **Excepciones de Formato y Entrada:** Los controladores capturan **NumberFormatException** cuando el operador tipea caracteres alfanuméricos en casilleros de pago o costo, e **IllegalArgumentException** al validar importes en la clase de utilidad **FormatoImporte**.
- **Seguridad Criptográfica:** En la clase **Usuario**, el método **autenticar(String pass)** implementa un bloque try-catch que captura **NoSuchAlgorithmException** para asegurar la disponibilidad de la función hash SHA-256.

```
// Manejo defensivo en la confirmación de ventas (VentasController.java)
@FXML
private void confirmarVenta() {
   if (!validarTurnoAbierto()) return;
   ventaActual.setMedioPago(cbMedioPago.getValue());
   ventaActual.setComprobantePago(txtNroComprobante.getText().trim());
   if (cbMedioPago.getValue() != null && cbMedioPago.getValue().contains("Efectivo")) {
      try {
         ventaActual.liquidarVuelto(Double.parseDouble(txtPagaCon.getText().trim()));
      } catch (NumberFormatException e) {
         mostrarAlerta("Importe inválido", "Ingrese el importe recibido en efectivo.");
         return;
      }
   }
   try {
      ventaActual.confirmarVenta();
   } catch (IllegalArgumentException | IllegalStateException e) {
      mostrarAlerta("No se pudo confirmar la venta", e.getMessage());
      return;
   }
   // Emisión de confirmación y reinicio del mostrador...
}
```

Menú Principal, Navegación y Control de Acceso (RBAC)

La aplicación implementa una pantalla principal con navegación basada en tarjetas y barras de herramientas (MenuPrincipalView.fxml), resolviendo de forma visual y accesible el requisito de menú de selección:

- **Navegabilidad y Atajos de Teclado:** Permite acceder a las terminales operativas del kiosco mediante clics o teclas rápidas:
    - [F1] Mostrador de Ventas y Cobro.
    - [F2] Cierre de Turno y Arqueo
    - [F3] Catálogo General y Stock
    - [F4] Gestión de Compras y Reposición.
    - [F5] Gestión de Usuarios y Permisos.
- **Seguridad y Control de Acceso basado en Roles (RBAC):** Tras autenticar las credenciales en LoginController, el menú principal invoca al método aplicarPermisosSegunRol(), el cual consulta usuarioActual.tienePermiso(...) y ajusta la habilitación y opacidad visual de los botones de cada módulo:
    - **Restricción de ABM de Productos:** Al acceder al catálogo (ProductoController), el sistema evalúa el rol del usuario mediante setUsuarioActual(). Para roles operativos como Vendedor (sofia) o Repositor (lucas), el panel boxAccionesABML se oculta por completo, dejando el catálogo estrictamente en modo de consulta de precios y existencias y reservando el alta, baja y modificación de artículos de forma exclusiva para el Administrador (cristian), en estricta concordancia con los casos de uso CU018, CU019 y CU020.
    - **Tratamiento de Usuarios Inactivos:** Si un operador **desvinculado o inactivo** (como la cuenta de prueba estefania) intenta autenticarse, LoginController intercepta el estado booleano y emite una alerta modal de error emergente (Alert.AlertType.ERROR), denegando la carga del menú principal y no permitiendo el acceso al Menú Principal.

Verificación Operacional y Capturas del Sistema en Ejecución

Como evidencia empírica de funcionamiento del prototipo en Java, se presentan a continuación las capturas de pantalla de los módulos operando en tiempo real:



Figura 31: Prueba Pantalla Acceso a Sistema funcionando.
Fuente: Elaboración propia.

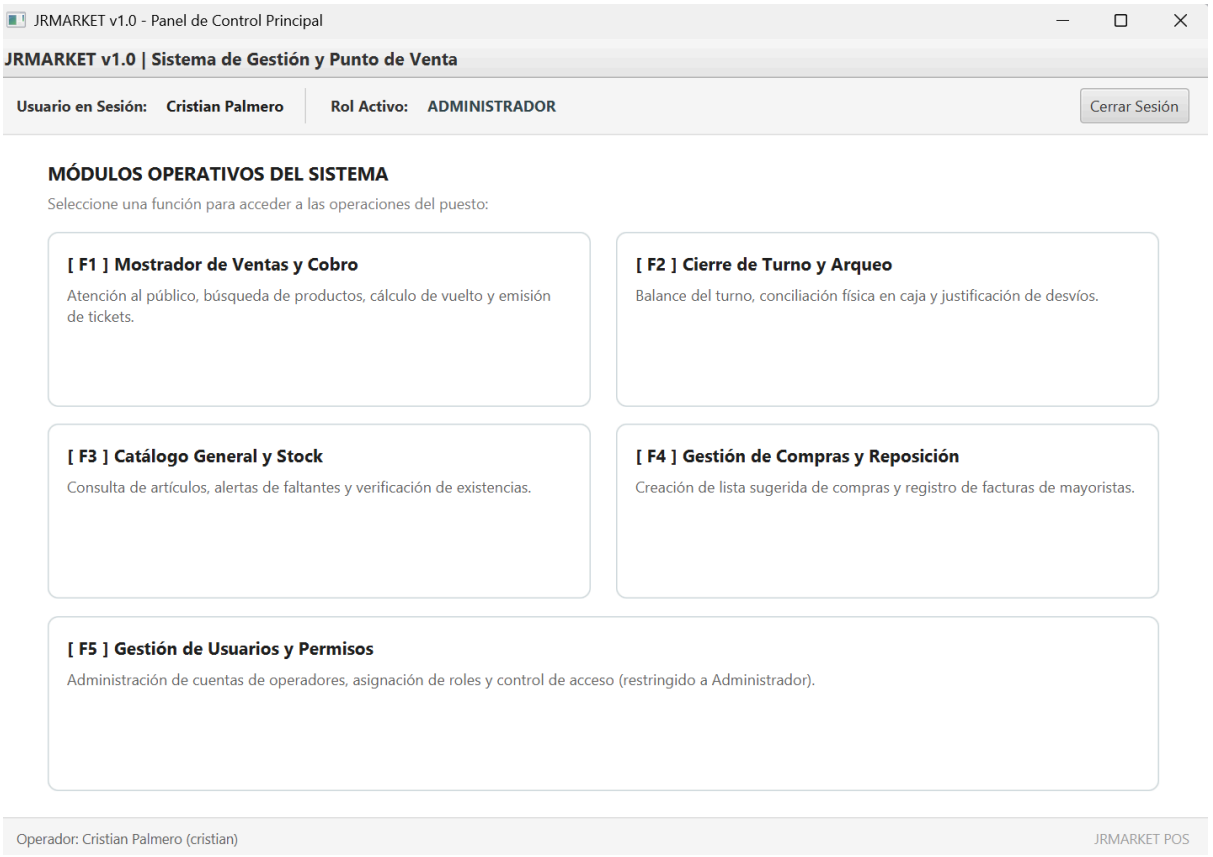


Figura 32: Prueba Menú Principal funcionando.
Fuente: Elaboración propia.

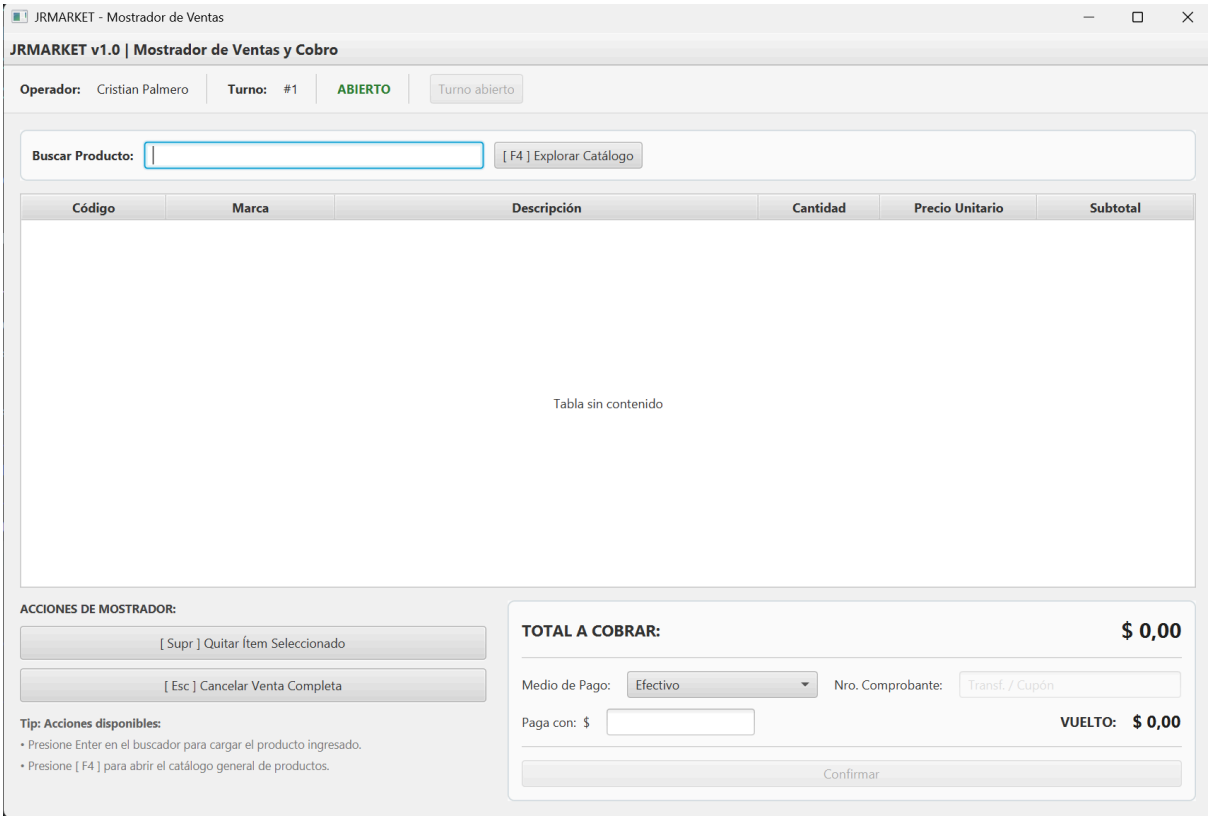


Figura 33: Prueba Módulo de Ventas funcionando.
Fuente: Elaboración propia.

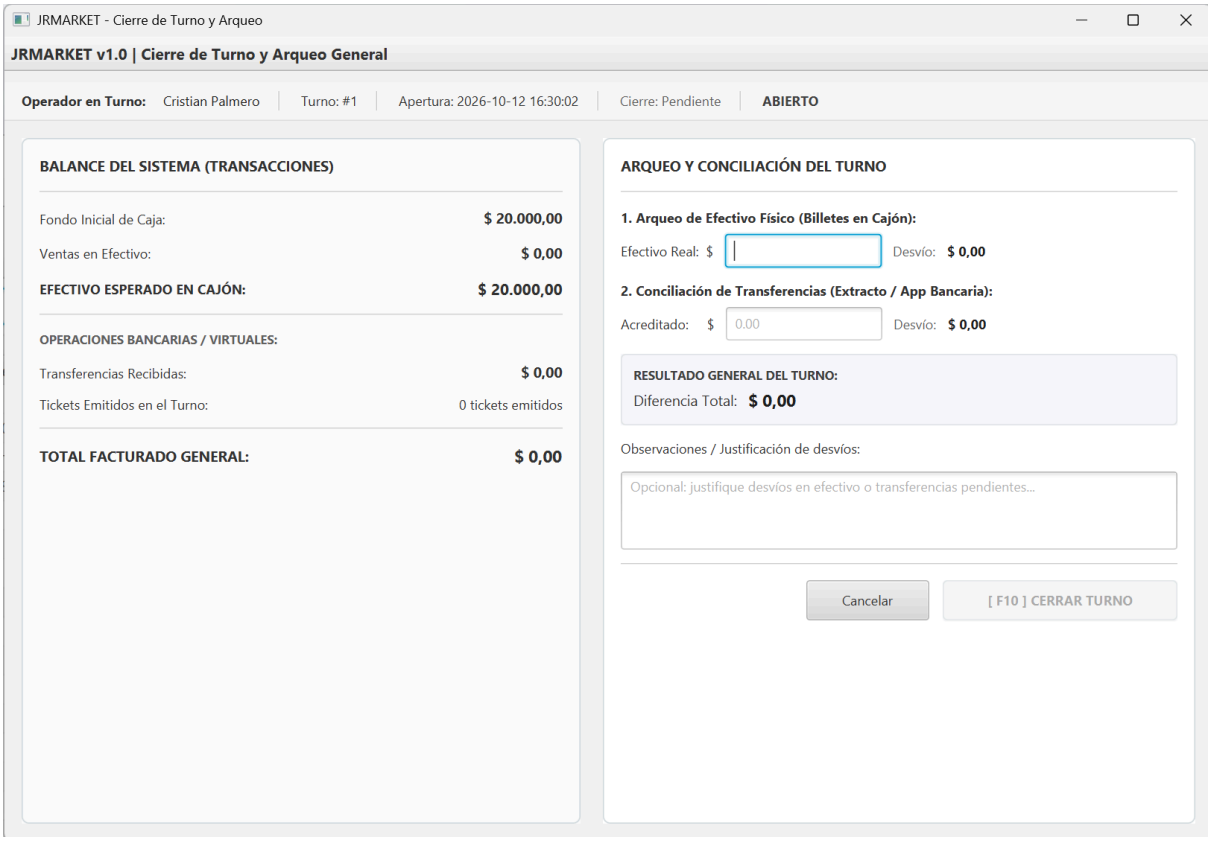


Figura 34: Prueba Módulo de Cierre de Turno funcionando.

Fuente: Elaboración propia.

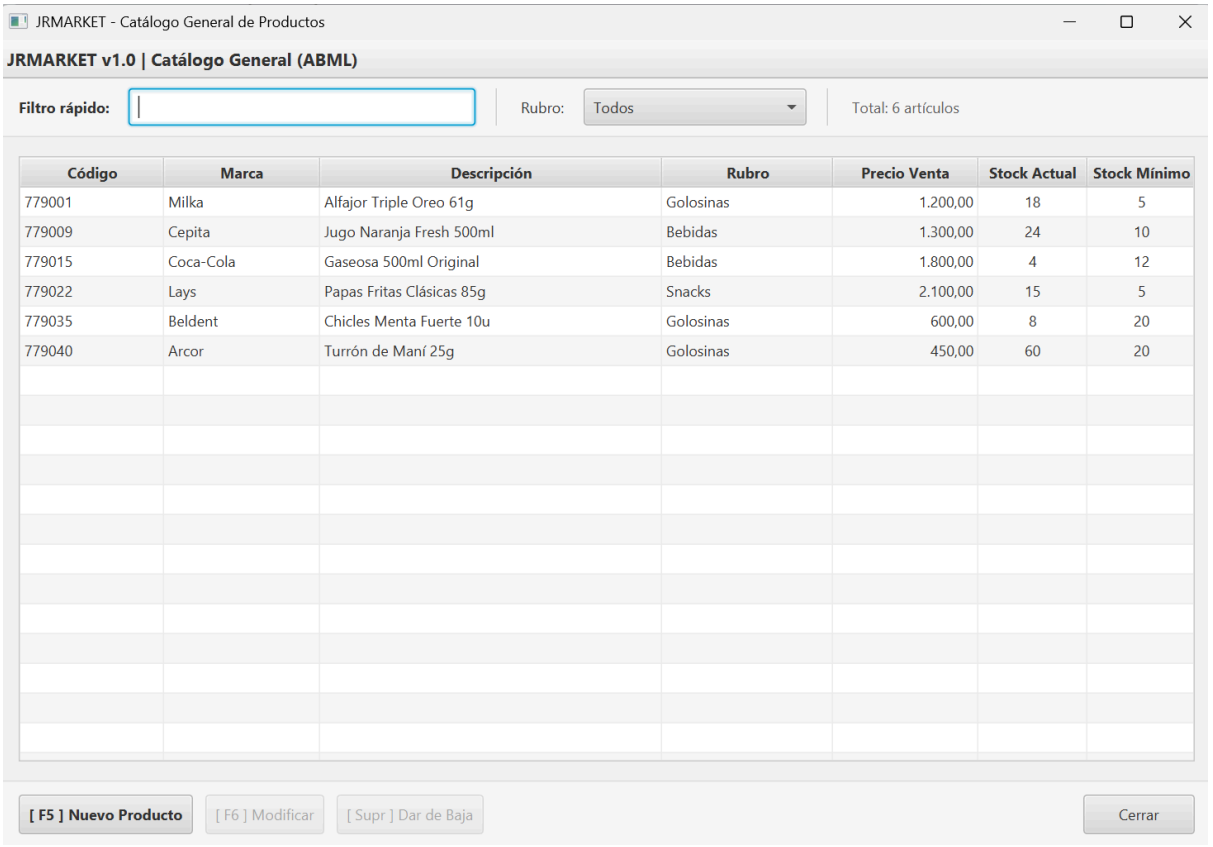


Figura 35: Prueba Catálogo de Productos funcionando.

Fuente: Elaboración propia.

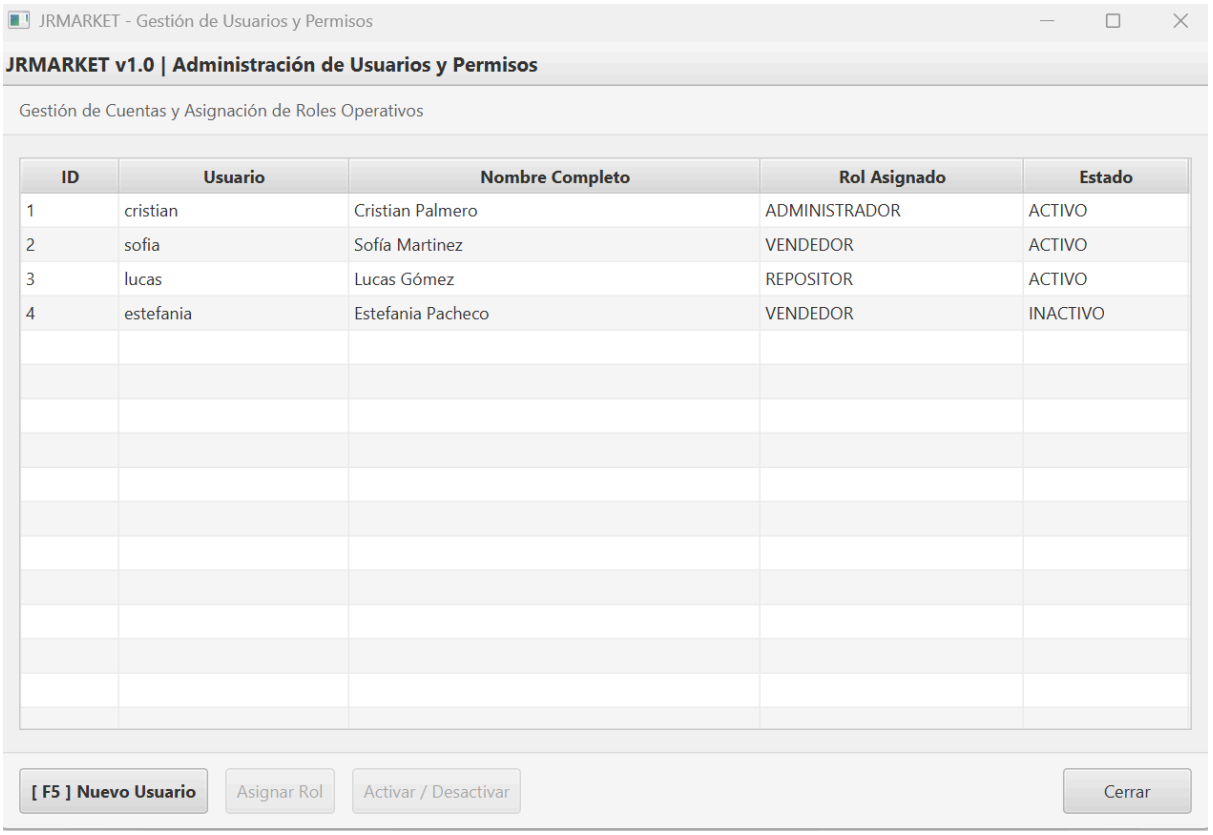


Figura 36: Prueba Módulo de Gestión de Usuarios y Permisos funcionando.
Fuente: Elaboración propia.

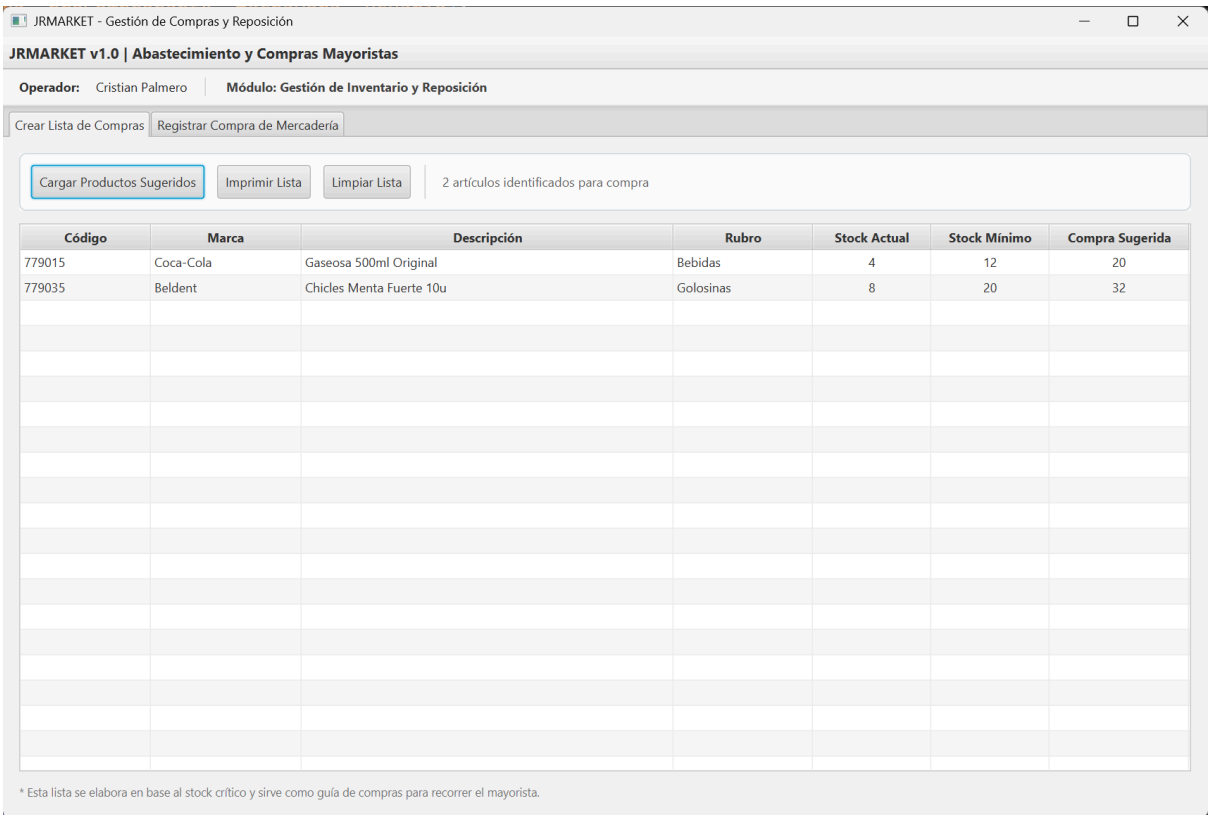


Figura 37: Prueba Módulo de Compras - Crear Lista de Compras funcionando.
Fuente: Elaboración propia.

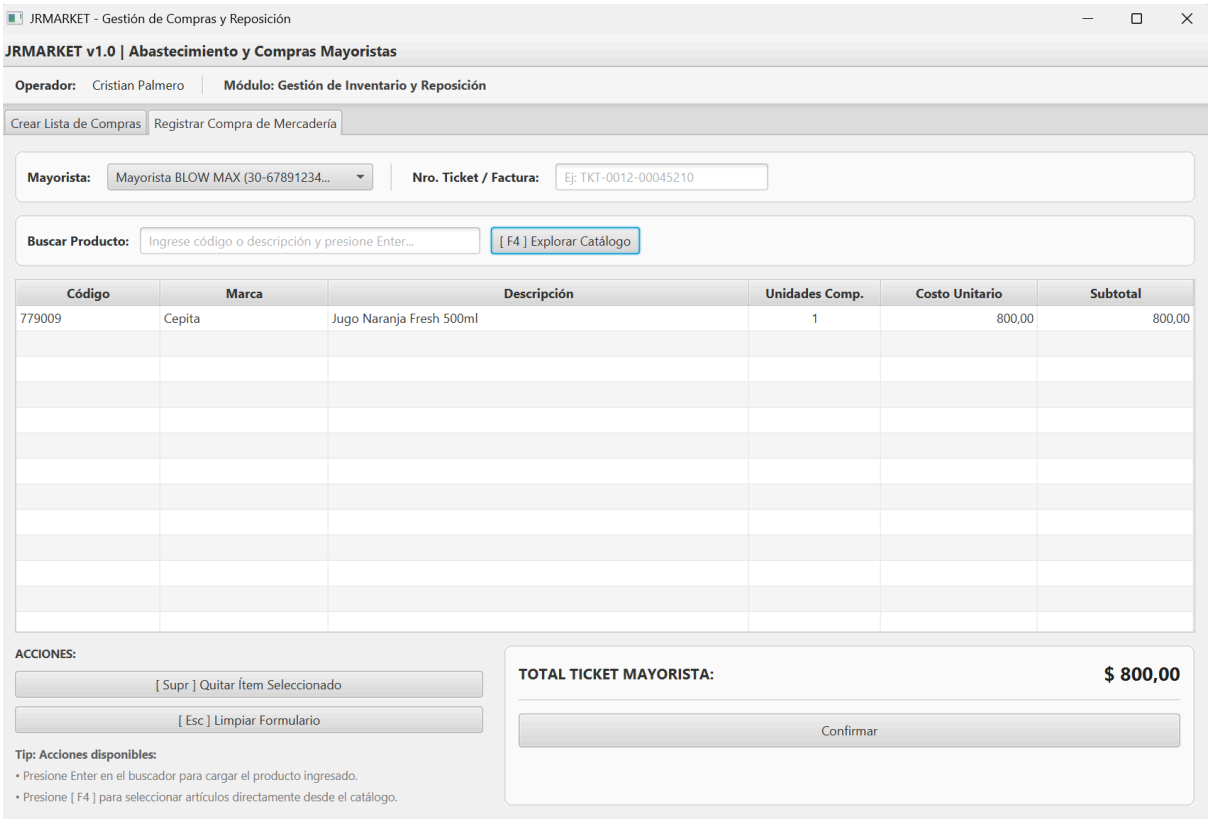


Figura 38: Prueba Módulo de Compras - Registrar Compra funcionando.
Fuente: Elaboración propia.

Asimismo, las reglas transaccionales del modelo (confirmarVenta, confirmarIngresoMercaderia, conciliarSaldo, autenticar con SHA-256 y control de stock) fueron validadas mediante la suite automatizada de pruebas unitarias en JUnit (com.jrmarket.AppTest), superando los 12 casos de prueba con un 100% de éxito (0 fallos, 0 errores).

Patrones de Diseño, Persistencia JDBC y Gestión de Archivos

En esta cuarta y última etapa del ciclo de vida del software para el comercio minorista "JRMARKET", se materializa la integración completa del sistema informático conforme a las directivas del Proceso Unificado de Desarrollo (PUD). En esta instancia se formaliza la arquitectura del prototipo bajo el patrón de diseño Modelo-Vista-Controlador (MVC), la persistencia y manipulación de datos en el motor relacional MySQL mediante la biblioteca estándar JDBC (Java Database Connectivity) prescindiendo de frameworks ORM, el empleo de estructuras de datos estáticas y dinámicas (arreglos, colecciones genéricas ArrayList y cursores Iterator), la implementación de una estrategia robusta para la gestión de excepciones, y la persistencia física de comprobantes y reportes comerciales en archivos de texto plano a través del paquete java.io.

Patrón Arquitectónico Modelo-Vista-Controlador (MVC)

Un patrón de diseño de software provee un esquema estructural abstracto, probado y documentado que soluciona problemas recurrentes de organización y acoplamiento en el desarrollo orientado a objetos (Debrauwer, 2013). En aplicaciones interactivas de escritorio con interfaz gráfica y persistencia relacional, el desafío primordial reside en diseñar una interfaz flexible que pueda evolucionar sin alterar la lógica de negocio ni la estructura de las tablas de la base de datos.

Para el sistema de punto de venta e inventario de **JRMARKET**, se adoptó de manera integral el patrón **Modelo-Vista-Controlador (MVC)**, garantizando el cumplimiento de los requerimientos de modularidad y transaccionalidad (RNF06) y autonomía operativa (RNF02) establecidos en las fases de análisis:

Tabla 6: Correspondencia de requerimientos del sistema con el patrón MVC

Requerimiento	Clasificación	Implementación Arquitectónica en JRMARKET
RNF06	Integridad y Transaccionalidad	Para la persistencia y consulta de datos en la base de datos relacional MySQL, se implementa una arquitectura desacoplada en tres capas mediante el patrón MVC,

		encapsulando la lógica de negocio en el Modelo y aislando los controladores de eventos y las pantallas FXML.
RNF02	Persistencia Local y Autonomía	El acceso a datos se canaliza a través de un componente Data Access Object (JRMarketDAO) dentro del Modelo, comunicándose mediante JDBC con el servidor MySQL local (127.0.0.1:3306), garantizando independencia de la red externa.

Fuente: Elaboración propia.

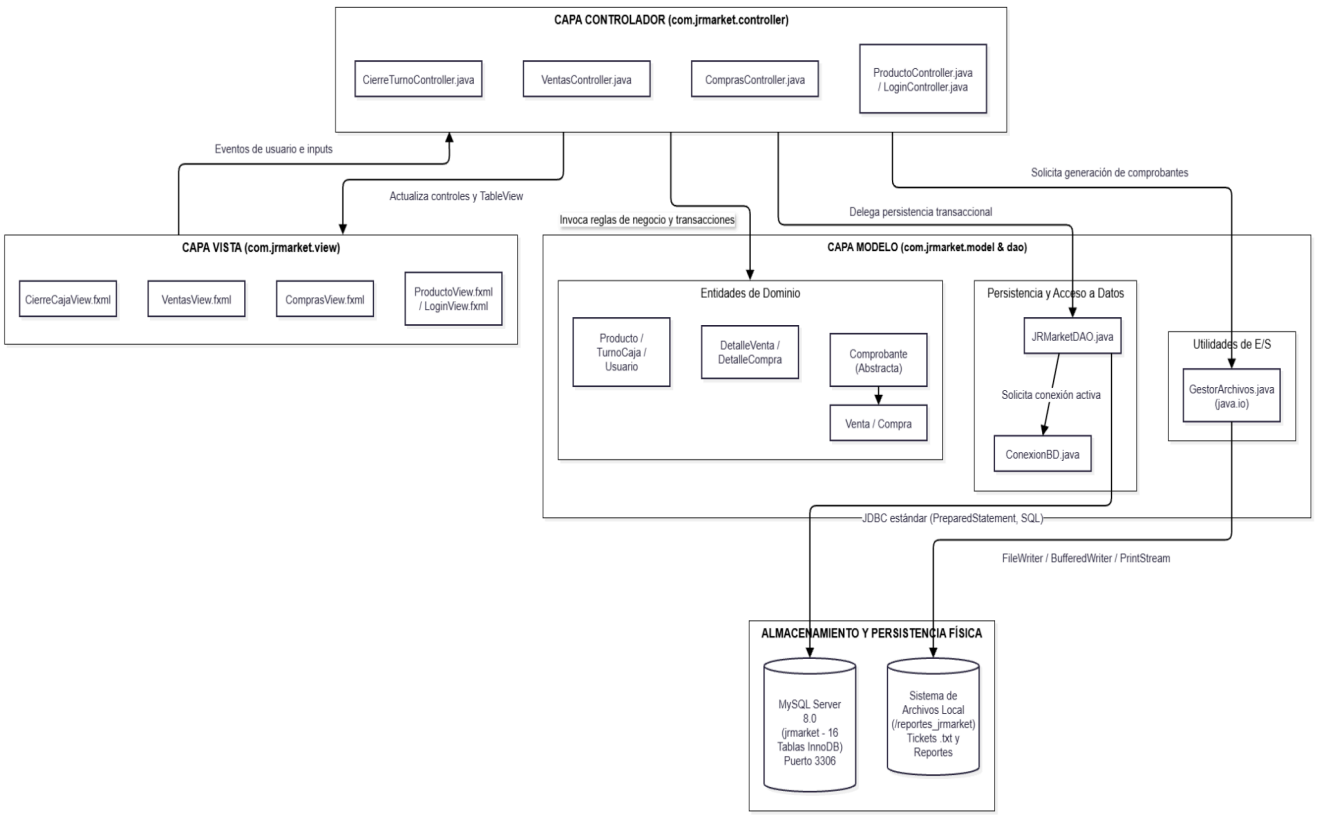


Figura 39: Arquitectura de capas del patrón Modelo-Vista-Controlador aplicada en JRMARKET.
Fuente: Elaboración propia.

Responsabilidades de los Componentes en JRMARKET

- **Modelo** (com.jrmarket.model y com.jrmarket.dao): Constituye el núcleo funcional que representa los datos y las reglas del negocio. Se compone de dos subcapas:

    - **Entidades de Dominio:** Clases fuertemente tipadas que modelan los conceptos transaccionales (Comprobante, Venta, Compra, DetalleVenta, DetalleCompra), de inventario (Producto, Marca, TipoProducto, Proveedor), de caja (TurnoCaja, MedioPago) y de seguridad (Usuario, Rol, Permiso, RolPermiso, EstadoVenta, HistorialEstadoVenta). En estas entidades residen las invariantes críticas: cálculo polimórfico de importes (calcularTotal()), control estricto de existencias con operaciones aritméticas exactas (Math.addExact), validación de vigencia de medios de pago (isVigente()) y cifrado seguro de contraseñas bajo el estándar SHA-256.
    - **Capa de Persistencia y Acceso a Datos (DAO):** Implementada mediante las clases **ConexionBD** y **JRMarketDAO**, las cuales encapsulan la interacción directa con MySQL a través de JDBC puro. Si la estructura física de una tabla relacional cambia o se optimiza una sentencia SQL, las modificaciones se restringen exclusivamente a esta capa sin necesidad de modificar vistas ni controladores de interfaz.
- **Vista (com.jrmarket.view):** Representa el entorno gráfico de interacción directa con el operador. Se implementa mediante archivos declarativos en formato XML estandarizado para **JavaFX** (LoginView.fxml, MenuPrincipalView.fxml, VentasView.fxml, ComprasView.fxml, CierreCajaView.fxml, ProductoView.fxml y UsuarioView.fxml). Cada vista describe visualmente los formularios, tablas (TableView), menús y atajos de teclado, careciendo deliberadamente de lógica de cómputo. Si el comercio requiere modificar la estética visual, la paleta cromática o el tamaño de los paneles, sólo se editan los archivos FXML.
- **Controlador (com.jrmarket.controller):** Actúa como mediador bidireccional entre la vista y el modelo. Se estructura en clases especializadas (LoginController, MenuPrincipalController, VentasController, ComprasController, CierreTurnoController, ProductoController y UsuarioController). Cada controlador captura los eventos disparados por el operador (ingreso de código de barras, confirmación de venta, selección del medio de pago o cierre de caja), valida formalmente los campos de entrada, solicita al modelo la ejecución de las transacciones y coordina la actualización de las tablas visuales y la exportación de comprobantes impresos.

Conexión y Persistencia Relacional con MySQL mediante JDBC Puro

En estricta conformidad con las directivas pedagógicas de la cátedra, la interacción con la base de datos relacional se desarrolló utilizando de forma exclusiva la **biblioteca JDBC** (Java Database Connectivity) provista por la plataforma Java estándar (java.sql), excluyendo cualquier framework de mapeo objeto-relacional como Hibernate, JPA o Spring Data.

Gestión de la Conexión y Disponibilidad del Canal (ConexionBD.java)

La **clase ConexionBD** centraliza la configuración de red y las credenciales de autenticación hacia la **base de datos local *jrmarket*** alojada en el servidor **XAMPP** (127.0.0.1:3306). Sus responsabilidades técnicas abarcan:

- **Carga dinámica del Driver:** Invoca de manera explícita al método **Class.forName("com.mysql.cj.jdbc.Driver")**, registrando el conector oficial en el entorno de ejecución de la Máquina Virtual de Java (JVM).

- **Apertura del Enlace:** Utiliza **DriverManager.getConnection(...)** parametrizado con la cadena de conexión optimizada (useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true), usuario root y clave por defecto del entorno local.
- **Inspección de Disponibilidad y Estado:** Conforme a las recomendaciones de Grousard (2010), una llamada a **isClosed()** que devuelva false no certifica que el canal se encuentre operativo frente al servidor. Por ello, se implementó el método **esConexionValida(...)**, el cual complementa **!conexion.isClosed()** con **conexion.isValid(2)**, enviando un pulso de validación con un tiempo de espera (timeout) de 2 segundos para constatar que el motor MySQL responda activamente.
- **Gestión de Advertencias (SQLWarning):** Mediante **conexion.getWarnings()** se recorre la cadena enlazada de advertencias emitidas por el servidor relacional (**warning.getNextWarning()**), depurándolas con **clearWarnings()** para no degradar el rendimiento.
- **Cierre Seguro de Recursos:** Implementa el método **cerrarConexion(...)** para liberar sockets físicos y desconectar sesiones inactivas.

```
// Paquete: com.jrmarket.dao - Archivo: ConexionBD.java
package com.jrmarket.dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.SQLWarning;

public class ConexionBD {
   private static final String DRIVER = "com.mysql.cj.jdbc.Driver";
   private static final String URL =
"jdbc:mysql://127.0.0.1:3306/jrmarket?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrie
val=true";
   private static final String USUARIO = "root";
   private static final String CLAVE = "";

   private ConexionBD() {}

   public static Connection obtenerConexion() throws SQLException {
      try {
         Class.forName(DRIVER);
      } catch (ClassNotFoundException e) {
         throw new SQLException("No se encontró el driver JDBC de MySQL (" + DRIVER + ").", e);
      }
      Connection conexion = DriverManager.getConnection(URL, USUARIO, CLAVE);
      verificarAdvertencias(conexion);
      return conexion;
   }
```

```
    public static boolean esConexionValida(Connection conexion) {
        try {
            return conexion != null && !conexion.isClosed() && conexion.isValid(2);
        } catch (SQLException e) {
            return false;
        }
    }

    private static void verificarAdvertencias(Connection conexion) {
        try {
            SQLWarning warning = conexion.getWarnings();
            while (warning != null) {
                System.err.println("Advertencia SQL: " + warning.getMessage());
                warning = warning.getNextWarning();
            }
            conexion.clearWarnings();
        } catch (SQLException ignored) {}
    }

    public static void cerrarConexion(Connection conexion) {
        if (conexion != null) {
            try {
                if (!conexion.isClosed()) conexion.close();
            } catch (SQLException e) {
                System.err.println("Error al cerrar conexión: " + e.getMessage());
            }
        }
    }
}
```

Consultas Parametrizadas y Prevención de Inyecciones SQL

El acceso al catálogo de productos, mayoristas, operadores y parametrizaciones se estructura en la clase **JRMarketDAO** empleando la interfaz **PreparedStatement**. Esta interfaz pre-compila las instrucciones SQL en el motor y vincula los parámetros mediante marcadores de posición (?), eliminando cualquier vulnerabilidad frente a ataques de inyección SQL y garantizando un tipado estricto en el intercambio de datos.

Un ejemplo de consulta parametrizada se da en el método **consultarProductosDisponibles()**, el cual ejecuta una instrucción **SELECT** con cláusulas **INNER JOIN** hacia las tablas maestras Marca y TipoProducto. Ésto se hace recorriendo las tuplas del cursor **ResultSet** mediante el iterador **while (resultSet.next())** e instanciando diferentes objetos de **Producto** que se agregan secuencialmente a una colección dinámica **ArrayList<Producto>**:

```java
// Paquete: com.jrmarket.dao - Archivo: JRMarketDAO.java (Fragmento demostrativo)
public List<Producto> consultarProductosDisponibles() {
    inicializarBaseDeDatos();
    List<Producto> productos = new ArrayList<>();
    String consultaSQL = "SELECT p.idProducto, p.codigoBarras, p.descripcion, p.precio, "
            + "p.precioCosto, p.stockActual, p.stockMinimo, p.estado, p.fechaDesde, p.fechaHasta, "
            + "m.idMarca, m.nombre AS nombreMarca, "
            + "t.idTipoProducto, t.nombre AS nombreTipo, t.descripcion AS descTipo "
            + "FROM Producto p "
            + "INNER JOIN Marca m ON p.idMarca = m.idMarca "
            + "INNER JOIN TipoProducto t ON p.idTipoProducto = t.idTipoProducto "
            + "ORDER BY p.idProducto";

    try (Connection conn = obtenerConexionInterna();
         PreparedStatement statement = conn.prepareStatement(consultaSQL);
         ResultSet rs = statement.executeQuery()) {

        while (rs.next()) {
            Marca marca = new Marca(rs.getInt("idMarca"), rs.getString("nombreMarca"),
"2026-01-01", null);
            TipoProducto tipo = new TipoProducto(rs.getInt("idTipoProducto"),
rs.getString("nombreTipo"),
                    rs.getString("descTipo"), "2026-01-01", null);
            String fechaHasta = rs.getString("fechaHasta");
            boolean activo = rs.getBoolean("estado") && (fechaHasta == null);

            Producto producto = new Producto(
                    rs.getInt("idProducto"), rs.getString("codigoBarras"), rs.getString("descripcion"),
                    rs.getDouble("precio"), rs.getDouble("precioCosto"), rs.getInt("stockActual"),
                    rs.getInt("stockMinimo"), activo, rs.getString("fechaDesde"), fechaHasta, marca, tipo);
            productos.add(producto);
        }
    } catch (SQLException e) {
        System.err.println("Aviso JDBC (consultarProductosDisponibles): " + e.getMessage());
    }
    return productos;
}
```

Transaccionalidad Atómica y Consistencia ACID

Para asegurar el requerimiento **RNF06**, las operaciones complejas que impactan en múltiples tablas de forma concurrente, tales como la confirmación de una venta en mostrador o la recepción de una compra mayorista, no pueden ejecutarse de manera aislada.

En el método **registrarVentaTransaccional(Venta venta, MedioPago medioPago)**, el flujo desactiva explícitamente el modo de confirmación automática de sentencias sql mediante **conn.setAutoCommit(false)**. Dentro de un único bloque transaccional atómico se coordinan las siguientes operaciones relacionales:

- Inserción de la cabecera en Venta, recuperando la clave primaria autogenerada (Statement.RETURN_GENERATED_KEYS).
- Inserción iterativa de los renglones en DetalleVenta.
- Actualización física de existencias en Producto (UPDATE Producto SET stockActual = ? WHERE idProducto = ?).
- Asentamiento del movimiento cronológico en HistorialEstadoVenta.
- Actualización acumulativa de saldos teóricos y recaudación por medio de pago en TurnoCaja.

Solo si todas las instrucciones se completan con éxito, se invoca **conn.commit()**, persistiendo definitivamente el lote en el motor InnoDB de MySQL. Si se suscita cualquier error **SQLException**, el bloque catch ejecuta de inmediato **conn.rollback()**, cancelando todas las modificaciones pendientes y restaurando el estado original de la base de datos sin corrupción de datos. En la cláusula **finally**, se restablece **conn.setAutoCommit(true)** y se liberan de forma segura los recursos del servidor.

Estructuras de Datos: Arreglos, Colecciones ArrayList e Iteradores

En el desarrollo de software orientado a objetos, la organización de las estructuras en memoria condiciona la eficiencia algorítmica y la seguridad operativa (Álvarez Caules, 2020). Siguiendo los contenidos del módulo, el proyecto combina el uso de arreglos nativos unidimensionales con colecciones dinámicas genéricas:

Arreglos Unidimensionales Nativos (Array[])

Los arreglos nativos constituyen estructuras contiguas de tamaño estático definido en tiempo de instanciación. En el sistema se aplican en dos escenarios precisos:

- Filtros de Interfaz: En ProductoController.java, la lista desplegable de categorías comerciales se construye poblando un arreglo unidimensional de cadenas de texto (String[] rubrosDisponibles = new String[tiposBD.size() + 1];), inicializando la posición 0 con el valor "Todos" e iterando secuencialmente sobre los tipos de producto recuperados de MySQL.
- Procesamiento Criptográfico de Bloques: En Usuario.java y UsuarioController.java, la función hash de seguridad SHA-256 manipula flujos de bytes mediante arreglos nativos byte[] (MessageDigest.getInstance("SHA-256").digest(...)), asegurando el procesamiento atómico de las credenciales.

Colecciones Dinámicas (ArrayList<E>)

A diferencia de los arreglos tradicionales de dimensión fija, la clase ArrayList<E> del paquete java.util permite modelar conjuntos cuyo número de elementos fluctúa dinámicamente en tiempo de ejecución.
En JRMARKET, se utiliza ArrayList para gestionar:

- El catálogo de productos disponibles (List<Producto> catalogoProductos = new ArrayList<>();).

- El carrito de compras y renglones de venta (List<DetalleVenta> detalles = new ArrayList<>();).
- Los artículos adquiridos en facturas de proveedores (List<DetalleCompra> detalles = new ArrayList<>();).
- El historial de transacciones emitidas en la sesión de caja (List<Venta> ventas = new ArrayList<>();).

Recorrido Seguro Mediante Iteradores (Iterator<E>)

La manipulación de colecciones dinámicas mediante ciclos for indexados clásicos puede generar inconsistencias o excepciones de concurrencia cuando se eliminan o evalúan elementos en tiempo de ejecución (Álvarez Caules, 2020).

Para salvaguardar la integridad de los datos, el sistema implementa la interfaz java.util.Iterator, utilizando sus métodos **hasNext()** y **next()** como cursor de navegación. Esta técnica se aplica en el módulo de compras para filtrar productos con stock crítico, en el mostrador para auditar renglones y en el acceso al sistema para autenticar operadores:

```
// Paquete: com.jrmarket.controller - Archivo: ComprasController.java (Uso de Iterator)
@FXML
private void generarListaSugeridos() {
   listaFaltantesObservable.clear();
   if (catalogoCompartido == null) return;

   // Recorrido seguro mediante Iterator sobre la colección dinámica ArrayList
   Iterator<Producto> it = catalogoCompartido.iterator();
   while (it.hasNext()) {
      Producto p = it.next();
      if (p.esStockCritico()) {
         listaFaltantesObservable.add(p);
      }
   }
   lblResumenFaltantes.setText(listaFaltantesObservable.size() + " artículos identificados para
compra");
}
```

Tratamiento y Gestión Robusta de Excepciones en Java

El tratamiento sistemático de errores constituye un pilar fundamental para certificar la confiabilidad y tolerancia a fallos del software de mostrador. El sistema implementa la arquitectura de excepciones provista por la plataforma Java a través de las sentencias **try, catch, finally, throw y throws**:

```java
// Paquete: com.jrmarket.dao - Archivo: JRMarketDAO.java
// Captura y tratamiento de excepciones verificadas en operaciones transaccionales
try {
    conn = ConexionBD.obtenerConexion();
    conn.setAutoCommit(false); // Inicio de transacción atómica ACID

    // Ejecución de sentencias parametrizadas (cabecera, detalle y actualización de stock)
    // ...
    conn.commit(); // Consolidación en base de datos
    return true;

} catch (SQLException e) {
    System.err.println("Código de error SQL: " + e.getErrorCode() + " - " + e.getMessage());
    if (conn != null) {
        try {
            conn.rollback(); // Marcha atrás obligatoria ante fallo transaccional
        } catch (SQLException rollbackEx) {
            System.err.println("Fallo al revertir transacción: " + rollbackEx.getMessage());
        }
    }
    return false;
} finally {
    ConexionBD.cerrarRecursos(rs, ps, conn); // Liberación determinística de conexiones
}
```

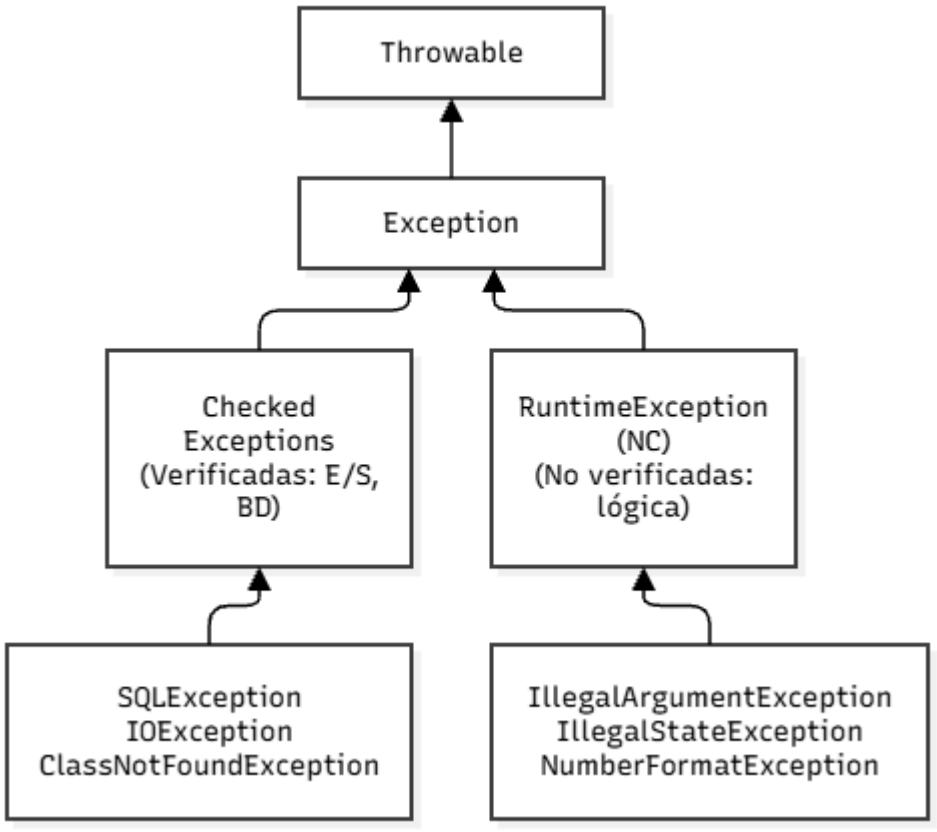


Figura 40: Jerarquía de excepciones aplicada en el sistema JRMARKET.
Fuente: Elaboración propia.

- Excepciones Verificadas (Checked Exceptions): Derivan directamente de java.lang.Exception y exigen por contrato su tratamiento obligatorio en tiempo de compilación. Se administran en las capas de datos y archivos:
    - **SQLException:** Capturada al interactuar con el motor MySQL mediante JDBC, canalizando el registro en consola de errores y disparando la marcha atrás transaccional **conn.rollback()**.
    - **IOException:** Controlada en la clase utilitaria **GestorArchivos** durante la apertura, escritura y cierre de flujos hacia los archivos físicos del disco.
    - **ClassNotFoundException:** Gestionada al cargar el driver **com.mysql.cj.jdbc.Driver** en ConexionBD.
- **Excepciones No Verificadas (Unchecked / RuntimeExceptions):** Derivan de **RuntimeException** y señalan infracciones en la lógica de entrada o transgresiones en las invariantes de negocio:
    - **NumberFormatException:** Interceptada en los controladores VentasController, ComprasController y CierreTurnoController cuando el usuario ingresa caracteres no válidos o letras en campos que requieren montos monetarios o cantidades, notificando el error mediante cuadros de diálogo (Alert) sin interrumpir el proceso de atención.

- **IllegalArgumentException:** Lanzada mediante la cláusula throw new desde las entidades de modelo ante argumentos no permitidos (ejemplo: cantidades menores o iguales a cero, precios de costo negativos o importes en efectivo que no cubren el total de la venta).
- **IllegalStateException:** Emitida cuando una operación viola el estado contextual del dominio, impidiendo registrar ventas con la caja cerrada (turnoCaja.getEstado() != "ABIERTO"), liquidar tickets sin renglones cargados, o confirmar ingresos de compras sin productos asociados.

Manipulación de Archivos de Texto Mediante Flujos java.io

Para satisfacer el requerimiento de intercambio de información hacia almacenamiento externo y emisión física de comprobantes (Sánchez Asenjo, 2010), se diseñó e implementó la clase utilitaria GestorArchivos dentro del paquete com.jrmarket.util. Esta clase procesa flujos de entrada/salida (I/O Streams) orientados a texto plano (.txt) organizados en tres operaciones fundamentales:

Manejo del Sistema de Archivos con la Clase File

La clase **File** se emplea para representar de forma abstracta rutas de archivos y carpetas locales. Antes de emitir cualquier reporte, el método privado **asegurarDirectorio()** comprueba la existencia de la carpeta **reportes_jrmarket (dir.exists())** y, si no ha sido creada, genera la estructura física en el sistema de archivos mediante **dir.mkdirs()**.

Flujos de Bytes con Salida Formateada (FileOutputStream y PrintStream)

Para la emisión física de comprobantes de mostrador, el método **exportarTicketVenta(Venta venta)** vincula un flujo de salida de bajo nivel a nivel de bytes (FileOutputStream) con un flujo de filtrado de alto nivel PrintStream configurado bajo el estándar de codificación StandardCharsets.UTF_8.

Esta combinación permite estructurar tickets comerciales alineados mediante métodos println() y cadenas con formato printf(...):

```
// Paquete: com.jrmarket.util - Archivo: GestorArchivos.java (Emisión de tickets)
public static File exportarTicketVenta(Venta venta) throws IOException {
   File directorio = asegurarDirectorio();
   String nombreArchivo = "ticket_" + venta.getNroTicket().replaceAll("[^a-zA-Z0-9_-]", "_") +
".txt";
   File archivoTicket = new File(directorio, nombreArchivo);

   try (FileOutputStream fos = new FileOutputStream(archivoTicket);
        PrintStream ps = new PrintStream(fos, true, StandardCharsets.UTF_8)) {

      ps.println("==============================================");
      ps.println("         JRMARKET - COMPROBANTE POS          ");
      ps.println("==============================================");
```

```
        ps.println("Ticket Nro : " + venta.getNroTicket());
        ps.println("Fecha/Hora : " + venta.getFechaHora());
        ps.println("Operador   : " + (venta.getUsuario() != null ?
venta.getUsuario().getNombreCompleto() : "-"));
        ps.println("Medio Pago : " + venta.getMedioPago());
        ps.println("------------------------------------------------");
        ps.println("CANT  DESCRIPCION                    SUBTOTAL ");
        ps.println("------------------------------------------------");

        Iterator<DetalleVenta> it = venta.getDetalles().iterator();
        while (it.hasNext()) {
            DetalleVenta dv = it.next();
            String desc = dv.getProducto().getDescripcion();
            if (desc.length() > 28) desc = desc.substring(0, 28);
            ps.printf("%-5d %-30s $ %8s%n", dv.getCantidad(), desc,
FormatoImporte.formatear(dv.getSubtotal()));
        }

        ps.println("------------------------------------------------");
        ps.println("TOTAL COBRADO: $ " + FormatoImporte.formatear(venta.getTotal()));
        if (venta.getMedioPago() != null &&
venta.getMedioPago().toUpperCase().contains("EFECTIVO")) {
            ps.println("PAGA CON     : $ " + FormatoImporte.formatear(venta.getMontoRecibido()));
            ps.println("SU VUELTO    : $ " + FormatoImporte.formatear(venta.getVuelto()));
        } else {
            ps.println("COMPROBANTE  : " + venta.getComprobantePago());
        }
        ps.println("================================================");
    }
    return archivoTicket;
}
```

Flujos de Caracteres con Buffer (FileWriter y BufferedWriter)

A diferencia de la emisión de comprobantes individuales de mostrador (donde se prioriza la salida formateada en bytes con **PrintStream**), para la generación de reportes voluminosos de auditoría e inventario se adoptó una arquitectura basada en flujos de caracteres orientados a texto plano mediante la jerarquía **java.io.Writer**.

Conforme a las pautas de Sánchez Asenjo (2010), el empleo directo de **FileWriter** interactúa con el sistema de almacenamiento en cada operación de escritura elemental, lo que genera una sobrecarga

innecesaria de llamadas al sistema y operaciones de entrada/salida a nivel de disco. Para optimizar el rendimiento y minimizar el consumo de recursos de la máquina local del comercio, el sistema desacopla el flujo físico envolviendo el canal en un búfer de memoria intermedia a través de la clase **BufferedWriter**.

El método **exportarListaFaltantes(List<Producto> catalogo)** materializa este patrón técnico: evalúa el inventario consolidado, filtra aquellos artículos que han alcanzado el umbral crítico mediante el método de dominio **p.esStockCritico()**, computa la sugerencia de reposición **(stockMinimo * 2 - stockActual)** y vuelca la información en disco garantizando la independencia de plataforma mediante saltos de línea universales **newLine()**:

```
// Paquete: com.jrmarket.util - Archivo: GestorArchivos.java (Reporte de reposición)
public static File exportarListaFaltantes(List<Producto> productos) throws IOException {
   File directorio = asegurarDirectorio();
   File archivoReporte = new File(directorio, "lista_compras_sugerida.txt");

   // Implementación de Try-with-resources para garantizar el cierre determinístico de flujos
(java.io.AutoCloseable)
   try (FileWriter fw = new FileWriter(archivoReporte, StandardCharsets.UTF_8);
        BufferedWriter bw = new BufferedWriter(fw)) {


bw.write("================================================================================
==========");
      bw.newLine();
      bw.write("         JRMARKET - REPORTE DE MERCADERÍA SUGERIDA PARA REPOSICIÓN
");
      bw.newLine();
      bw.write("Fecha de Emisión: " +
LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")));
      bw.newLine();

bw.write("================================================================================
==========");
      bw.newLine();
      bw.write(String.format("%-14s %-32s %-8s %-8s %-10s", "CÓDIGO", "DESCRIPCIÓN", "STOCK",
"MÍNIMO", "SUGERIDO"));
      bw.newLine();
      bw.write("--------------------------------------------------------------------------------");
      bw.newLine();

      int articulosCriticos = 0;
      for (Producto p : productos) {
         if (p.isEstado() && p.esStockCritico()) {
```

```
            int cantidadSugerida = (p.getStockMinimo() * 2) - p.getStockActual();
            String desc = p.getDescripcion().length() > 30 ? p.getDescripcion().substring(0, 30) :
p.getDescripcion();

            bw.write(String.format("%-14s %-32s %-8d %-8d %-10d",
                  p.getCodigoBarras(), desc, p.getStockActual(), p.getStockMinimo(),
cantidadSugerida));
            bw.newLine();
            articulosCriticos++;
          }
      }

      bw.write("-------------------------------------------------------------------------------");
      bw.newLine();
      bw.write("Total de productos en umbral crítico: " + articulosCriticos);
      bw.newLine();

bw.write("================================================================================
=========");
      bw.newLine();

      bw.flush(); // Vaciado explícito del búfer hacia el almacenamiento físico
    }
    return archivoReporte;
}
```

Entrega del Proyecto Integrador, Repositorios y Presentación Audiovisual

El proyecto informático para el comercio minorista "JRMARKET" cumple de manera exhaustiva con la totalidad de los requisitos formales, conceptuales y prácticos establecidos por la cátedra para la instancia final de evaluación:

- **Persistencia Relacional Completa:** Se provee el script SQL unificado que reconstruye el esquema de base de datos relacional en MySQL conteniendo las 16 tablas vinculadas bajo el motor transaccional InnoDB, con restricciones de integridad referencial e inserción completa de datos semilla de prueba.
- **Código Fuente Funcional y Estructurado:** El sistema en lenguaje Java se entrega sin errores de compilación, logrando el 100% de la suite de pruebas unitarias automatizadas (com.jrmarket.AppTest), organizado modularmente bajo el patrón MVC e integrando el controlador JDBC oficial de MySQL.
- **Evidencia Audiovisual de Operación:** Se acompaña un video demostrativo, exhibiendo la arquitectura del código en el entorno de desarrollo, la ejecución del sistema en JavaFX para el módulo de ventas y la verificación del impacto en tiempo real sobre las tablas en MySQL.

Enlaces de Verificación del Proyecto Integrador:

- Repositorio de Código Fuente Completo en GitHub: Repositorio Proyecto
- Repositorio de Scripts SQL de Base de Datos: Repositorio SQL
- Repositorio de Google Drive: EFIP I
- Video de Presentación del Proyecto Integrador en Youtube: Video en Youtube
- Video de Presentación del Proyecto Integrador en Drive: Video Proyecto.mp4

Bibliografía

- Álvarez Caules, C. (2020). Java ArrayList for y sus opciones. Arquitectura Java. https://www.arquitecturajava.com/java-arraylist-for-y-sus-opciones/
- Booch, G. (1996). Análisis y diseño orientado a objetos, con aplicaciones (2da ed.). Addison-Wesley Iberoamericana.
- Debrauwer, L. (2013). Patrones de diseño en Java. Ediciones ENI.
- Deitel, P. y Deitel, H. (2016). Cómo programar en Java (10ma ed.). Pearson Educación.
- Dux Software. (2024a). Planes y precios del sistema de gestión comercial para pymes. Dux Software Argentina. https://duxsoftware.com.ar/precios
- Dux Software. (2024b). Soluciones de punto de venta y control de stock para comercios minoristas. Dux Software Argentina. https://duxsoftware.com.ar/
- FácilVirtual. (2024a). Software Kioscos y facturación comercial en Argentina. FácilVirtual Tienda Oficial. https://www.facilvirtual.com.ar/software/software-kioscos-factura-electronica
- FácilVirtual. (2024b). Software para Kioscos y Almacenes: Características, funcionalidades y requerimientos técnicos. FácilVirtual Argentina. https://www.facilvirtual.com/ar/soluciones/software-kioscos
- Forouzan, B. A. (2007). Transmisión de datos y redes de comunicaciones (4ta ed.). McGraw-Hill Interamericana.
- Kendall, K. E. y Kendall, J. E. (2011). Análisis y diseño de sistemas (8va ed.). Pearson Educación.
- Ley N° 18.284. Código Alimentario Argentino. Congreso de la Nación Argentina, 8 de abril de 1971.
- Ley N° 24.240. Ley de Defensa del Consumidor. Congreso de la Nación Argentina, 22 de septiembre de 1993.
- Ley N° 24.788. Ley Nacional de Lucha contra el Alcoholismo. Congreso de la Nación Argentina, 5 de marzo de 1997.
- Ley N° 25.326. Protección de los Datos Personales. Congreso de la Nación Argentina, 30 de octubre de 2000.
- Ley N° 26.687. Regulación de la publicidad, promoción y consumo de los productos elaborados con tabaco. Congreso de la Nación Argentina, 11 de junio de 2011.
- Ley N° 7.646. Regulación de la venta y consumo de bebidas alcohólicas en el territorio provincial. Honorable Legislatura de Mendoza, 10 de enero de 2007.
- Loucopoulos, P. (2009). Requirements Engineering. En Handbook of Software Engineering and Knowledge Engineering. World Scientific.
- Meza González, J. D. (2021). Modificadores de acceso public, protected, default y private en Java. Programar Ya.
- Pressman, R. S. (2010). Ingeniería del Software: Un enfoque práctico (7ma ed.). McGraw-Hill Interamericana.
- Reinosa, E. J., Maldonado, C. A., Muñoz, R., Damiano, L. y Abrutsky, M. (2012). Bases de datos. Ediciones Granica.
- Sánchez Asenjo, N. (2010). Iniciación y programación en Java. Ediciones Paraninfo / Ministerio de Educación.
- Saroka, R. H. (2002). Sistemas de información en la era digital. Fundación OSDE.

- Shaw, M. y Garlan, D. (2015). Software Architecture: Perspectives on an Emerging Discipline. Pearson India.
- Zoo Logic. (2021). Conocé las características y beneficios de Dragonfish: Sistema de gestión para puntos de venta minoristas. Zoo Logic Software Empresarial. https://www.zoologic.com.ar/caracteristicas-beneficios-dragonfish/
- Zoo Logic. (2024). Dragonfish: Programa de gestión comercial para locales y comercios. Zoo Logic Software Empresarial. https://www.zoologic.com.ar/conoce-dragonfish/